Chapter 1.

Nonlinear and transcendental algebraic equations

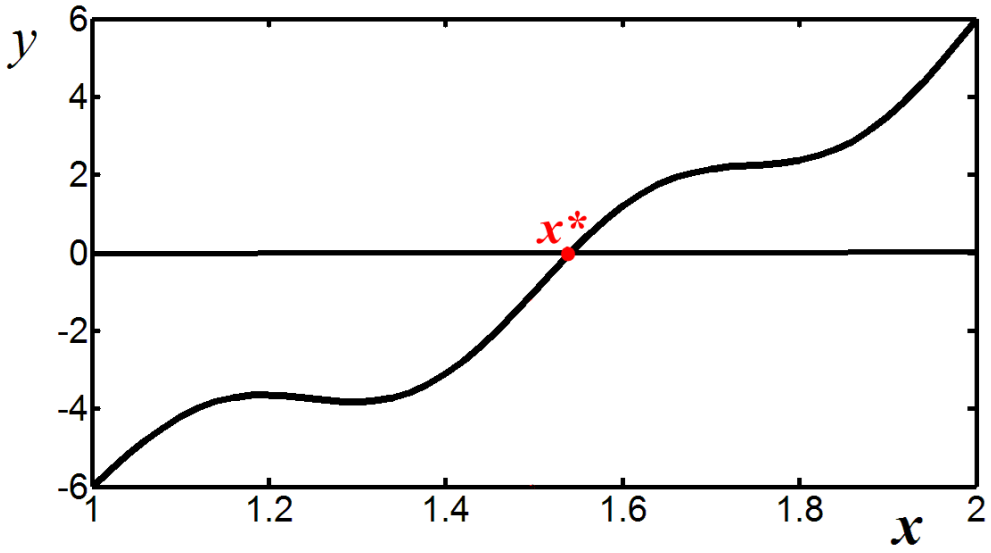
Problem:

if there exists a true/exact solution x^*, such that $f(x^*)=0$, then which way can we calculate an approximate value of x^* , admitting a small error, say ± 0.0001 ?

x^* is called a root of the equation

Theorem. If the function $y=f(x)$ is continuous on $a \le x \le b$, and the signs of $f(a)$ and $f(b)$ are opposite, then there exists x^* such that $f(x^*)=0$.

(for a proof, see course of Math. Analysis)



$$4x^2 + \sin(4\pi x) - 10 = 0, \quad 1 \le x \le 2$$

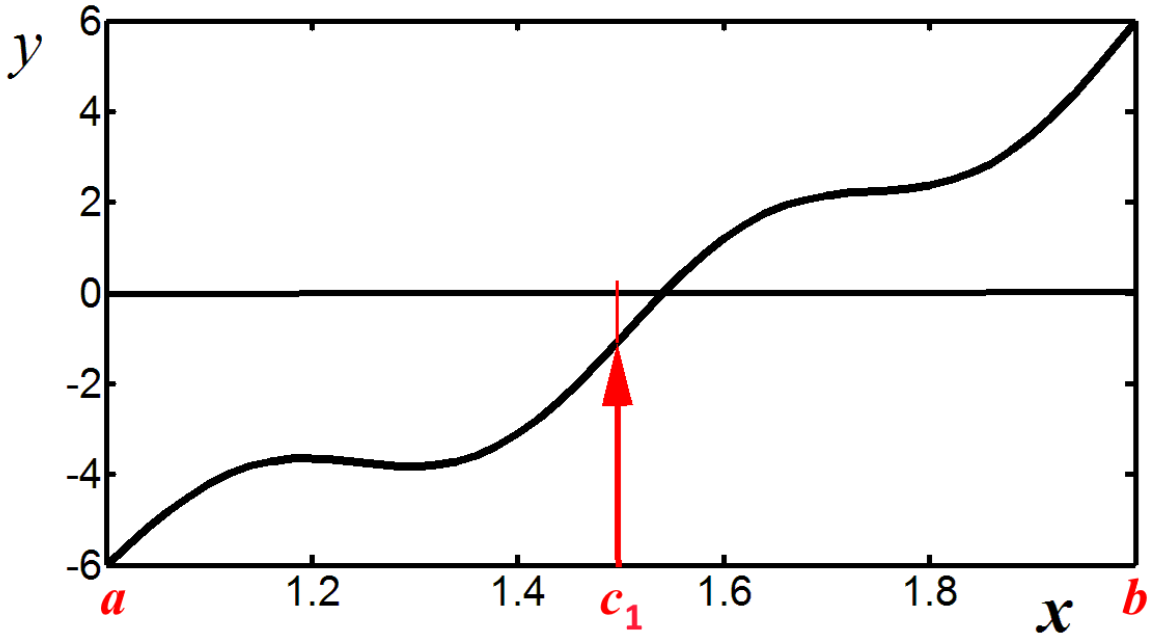
The solution exists!

1) Bysection method for calculation of x^*

It consists of repeatedly bisecting the given interval, and then selecting subinterval in which the function changes sign, and therefore must contain a root.

The method is also called the **interval halving** method, and the **dichotomy method**.

At each step the method divides the interval in two parts by computing the midpoint $c_1 = (a+b)/2$ and then selecting the part in which function $f(x)$ changes sign.



Algorithm of calculations: Suppose $f(a)<0,\ f(b)>0.$

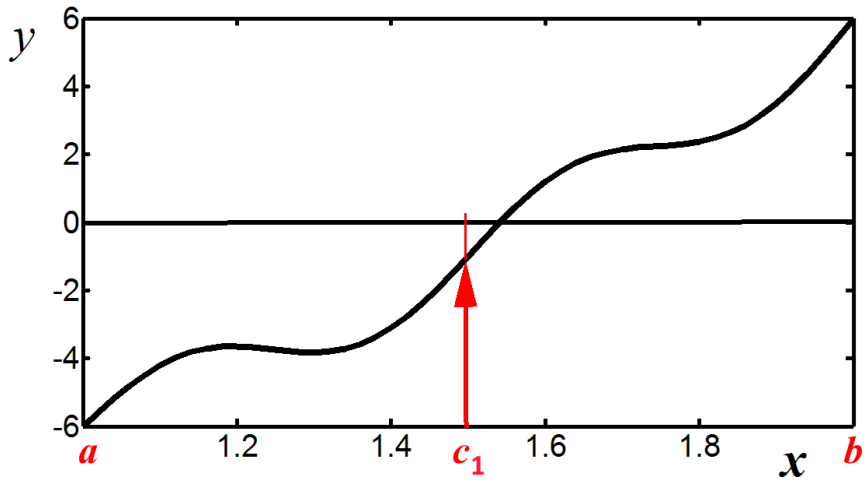
Calculate $c_1=(a+b)/2$ *and* $f(c_1)$

If $f(c_1)<0$, then root is in the right subsegment,

therefore we denote $a_2=c_1$, $b_2=b$

If $f(c_1)>0$, then we denote

$a_2= a$, $b_2= c_1$



Then procedure repeats:

$c_k=(a_k+b_k)/2$, $k= 2,\ldots$

If $f(c_k)<0$, then $a_{k+1}= c_k$, $b_{k+1}= b_k$

If $f(c_k)>0$, then $a_{k+1}= a_k$, $b_{k+1}= c_k$

(If $f(c_k)=0$, then c_k *is a root*).

At each step, $f(a_k)<0$, $f(b_k)>0$. The procedure continues until the subinterval is sufficiently small.

Length of the subinterval $b_k - a_k = (b - a) / 2^{k-1}$

shows a maximum error in the calculated approximate solution c_k **:**

$$| c_k - x^* | \leq (b - a) / 2^k$$

where x^* **is the "true" solution.**

k**=10 =>** $(b-a)/1024$

k**=20 =>** $(b-a)/1048576$

In engineering problems, typically, the error (tolerance) of 0.0001 or 0.00001 is O.K.

That is, calculations can be stopped if in successive approximations, c_k **and** c_{k+1} **, 4 or 5 digits to the right of decimal point remain the same after rounding.**

Now we consider another method, which often needs smaller number of steps for obtaining a solution of the same accuracy

2) Method of chords

Suppose that the curve

$y=f(x)$ is convex: $f''(x)>0$

Draw a straight segment (chord) connecting its endpoints:

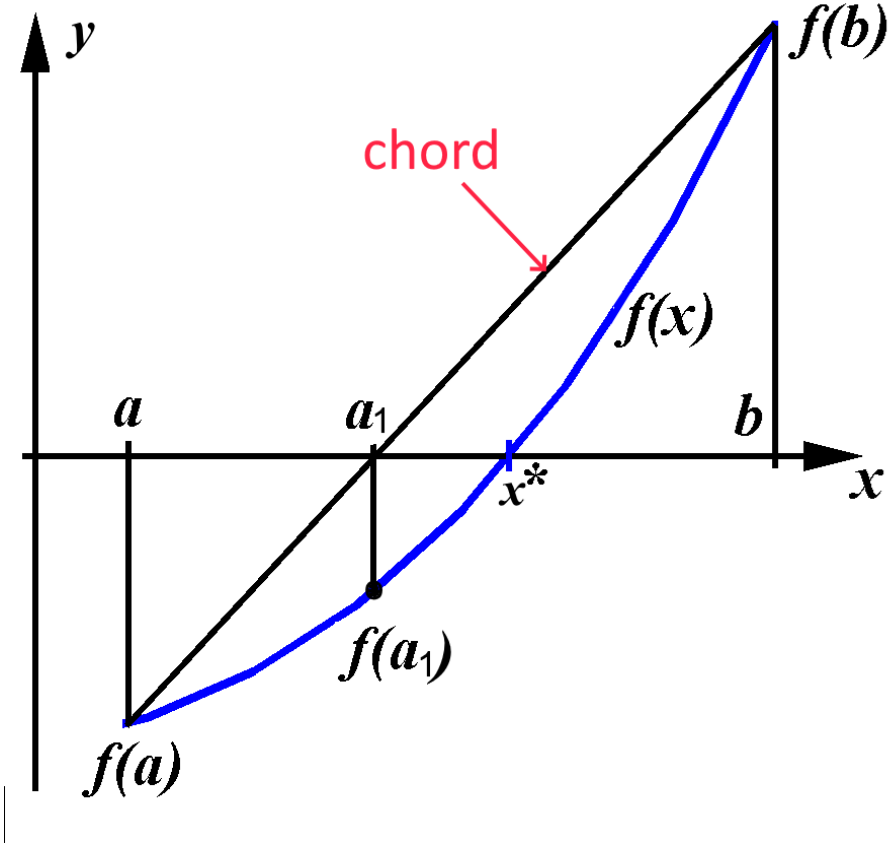
$$y=f(a)+(x-a)[f(b)-f(a)]/(b-a)$$

Find intersection of the segment with the axis $y=0$:

$$0=f(a)+(a_1-a)[f(b)-f(a)]/(b-a)$$

$$-(b-a)f(a)=(a_1-a)[f(b)-f(a)]$$

$$a_1-a=-(b-a)f(a)/[f(b)-f(a)]$$



$a_1 = a - (b-a)f(a)/[f(b)-f(a)]$

- first step towards the solution

$a_2 = a_1 - (b-a_1)f(a_1)/[f(b)-f(a_1)]$ **- second step**

$a_{k+1} = a_k - (b-a_k)f(a_k)/[f(b)-f(a_k)]$ $k=1,2,\ldots$

We get a sequence of approximate solutions

$$a_k \rightarrow x^*$$

the sequence is monotonously increasing as $f(a_k)<0$

Example: let's solve the equation

$$e^x + 2x^2 = 2, \quad 0 \le x \le 1$$

3) Method of tangent lines (Newton's method)

The same problem: solve

$$f(x)=0\ , \quad a \le x \le b$$

Suppose curve $y=f(x)$ *is convex:* $f''(x)>0$

Line tangent to curve at $x=b, y=f(b)$:

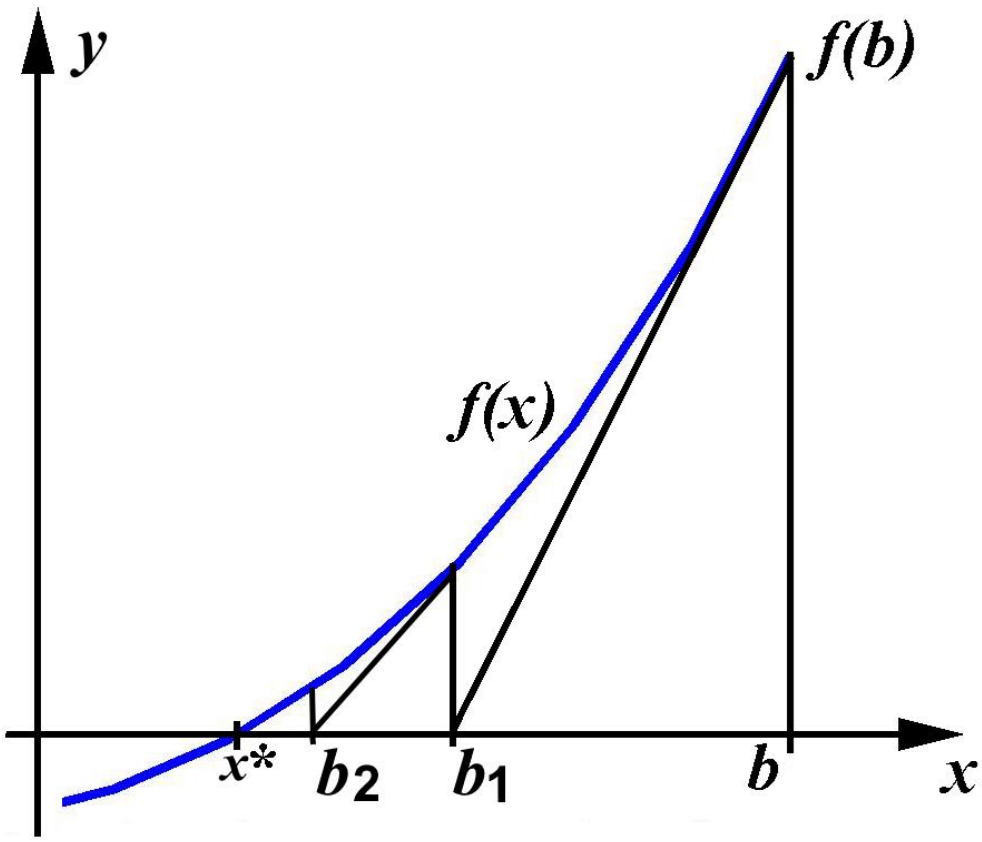
$$y=f(b)+(x-b)f'(b)$$

intersection: $0=f(b)+(b_1-b)f'(b)$

$$-f(b)=(b_1-b)f'(b)$$

$$-f(b)/f'(b)=(b_1-b)$$

$b_1=b-f(b)/f'(b)$ - *first step towards solution,*



$b_1 = b - f(b)/f'(b)$ - *first step towards solution,*

$b_2 = b_1 - f(b_1)/f'(b_1)$ $\qquad$ $b_{k+1} = b_k - f(b_k)/f'(b_k),\ k=2, \ldots$

sequence is monotonously decreasing

Example: finding square root

$$x^2 = d$$

$$f(x) = x^2 - d \qquad f'(x) = 2x$$

$$b_{k+1} = b_k - (b_k^2 - d)/(2b_k)$$

$$b_{k+1} = b_k - b_k/2 + d/(2b_k) \qquad b_{k+1} = (b_k + d/b_k)/2$$

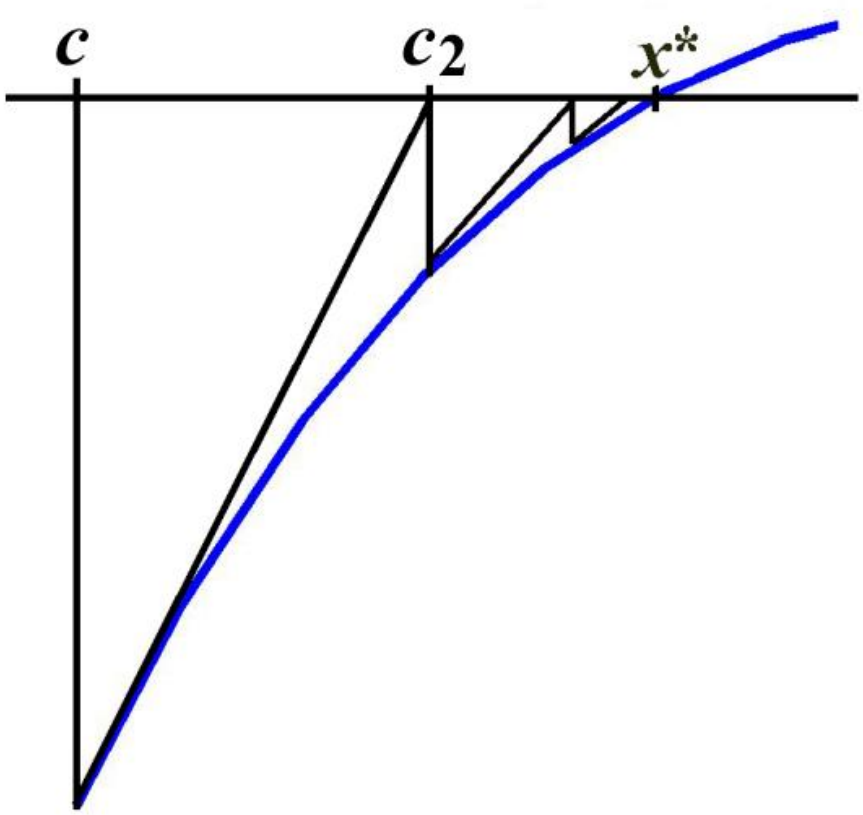
initial b – arbitrary value

Example: $e^x + 2x^2 - 2 = 0$

(same as in the method of chords)

If curve y=$f(x)$ is concave:

$f''(x)<0$



then the left endpoint a of the given segment is recommended as initial point for the start of iterations.

We have the same formula

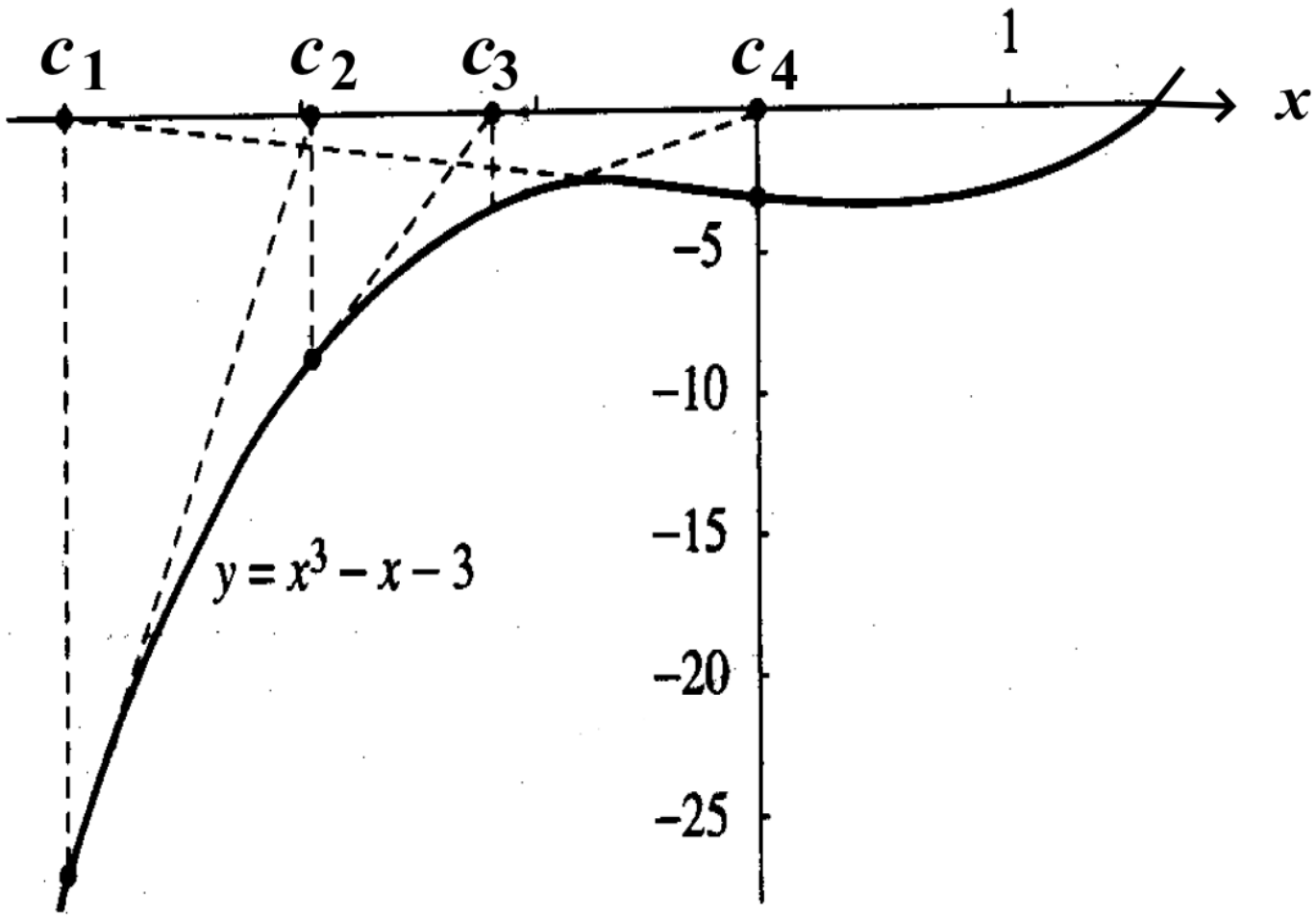
$$c_{k+1}=c_k - f(c_k)/ f'(c_k),$$

where $f(c_k)<0$; therefore, the sequence c_k is monotonously increasing.

Sometimes, the method does not work (bad cases) :

(*) If $f'(c_k)=0$ for some k, then the method can no longer be applied.

(**) Iterations can stuck in a cycle:



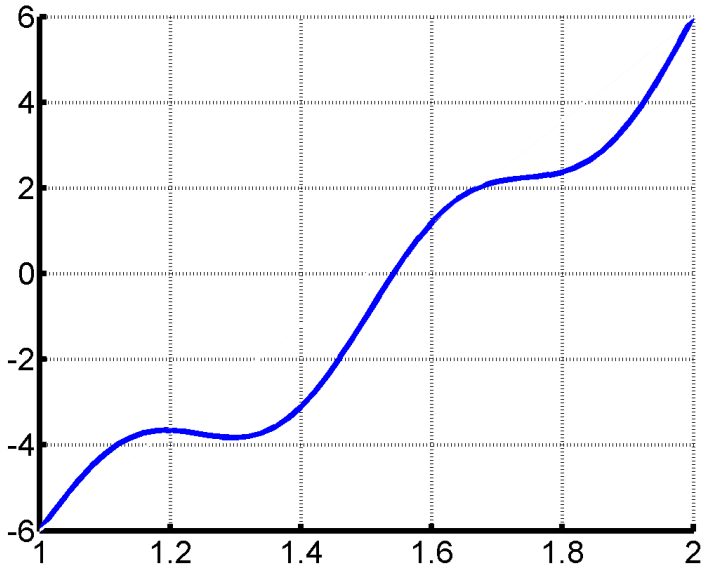
The case f(a)>0 and f(b)<0 can be reduced to the previous one by multiplying the equation by -1 :

$$-f(x)=0$$

Notice on the number of roots:

$$4x^2 + \sin(4\pi x) - 10 = 0 \,, \qquad 1 \le x \le 2$$

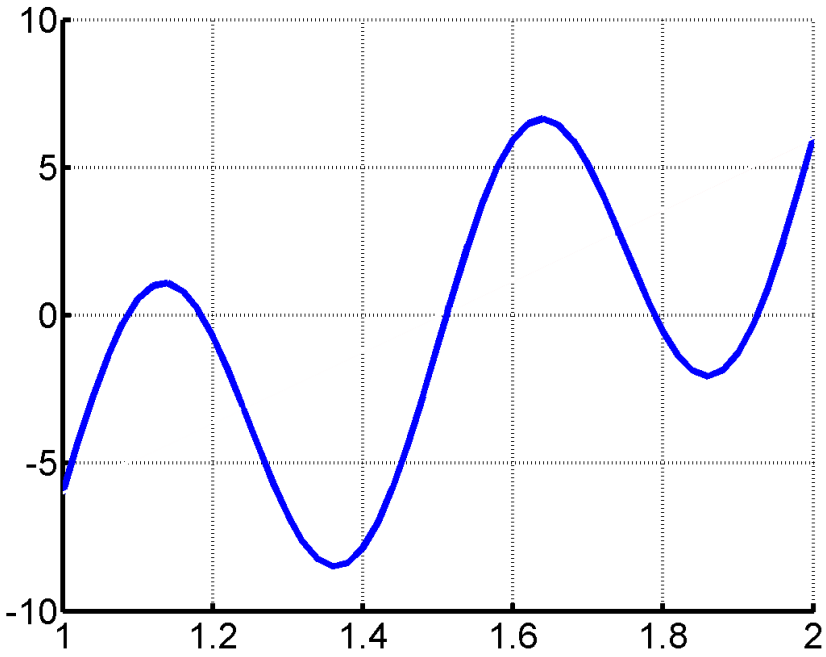
1 root:



Possibly, there exist non-unique roots

$$4x^2 + 6*\sin(4\pi x) - 10 = 0, \quad 1 \le x \le 2$$

5 roots:



It is recommended to “separate” roots by plotting a graph

4) Iteration method

Example 1.

$x - 0.1 \sin x - 2 = 0$

$x = 0.1 \sin x + 2$

$c_1 = 2$ - initial approximation

$c_2 = 0.1 \sin c_1 + 2 = 2.0909297$

$c_3 = 0.1 \sin c_2 + 2 = 2.0867753$

$c_4 = 0.1 \sin c_3 + 2 = 2.0869810$

$c_5 = 0.1 \sin c_4 + 2 = 2.0869709$

$c_6 = 0.1 \sin c_5 + 2 = 2.0869714$

$c_7 = 0.1 \sin c_6 + 2 = 2.0869713$

$c_8 = 0.1 \sin c_7 + 2 = 2.0869713$

- approximate solutions

In general: The equation $f(x)=0$ can be transformed to the form $x = \varphi(x)$ by a simple addition of x to both sides:

$$x = x+f(x)$$

$$x = \varphi(x)$$

Then choose an initial value c_1 and start iterations:

$$c_{k+1} = \varphi(c_k)\,, \qquad k=1, 2,...$$

If c_{k+1} and c_k become very close to each other, then

$$c_k \approx x^* \text{ - solution}$$

Example 2. $x + 0.3(1+x^4) - 0.4 = 0$

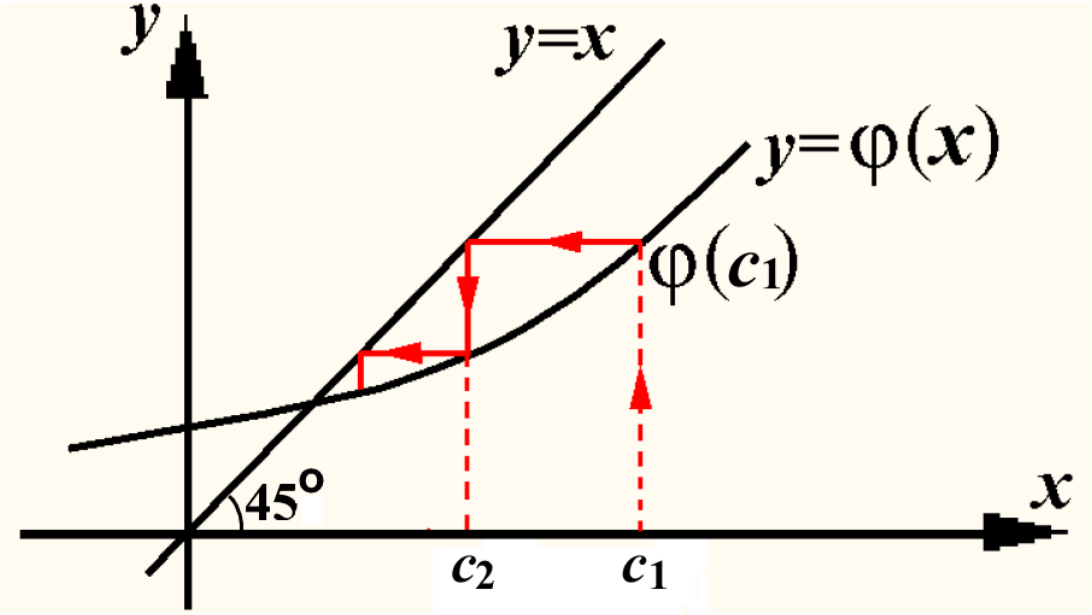
clear

$0 \leq x \leq 1.4$

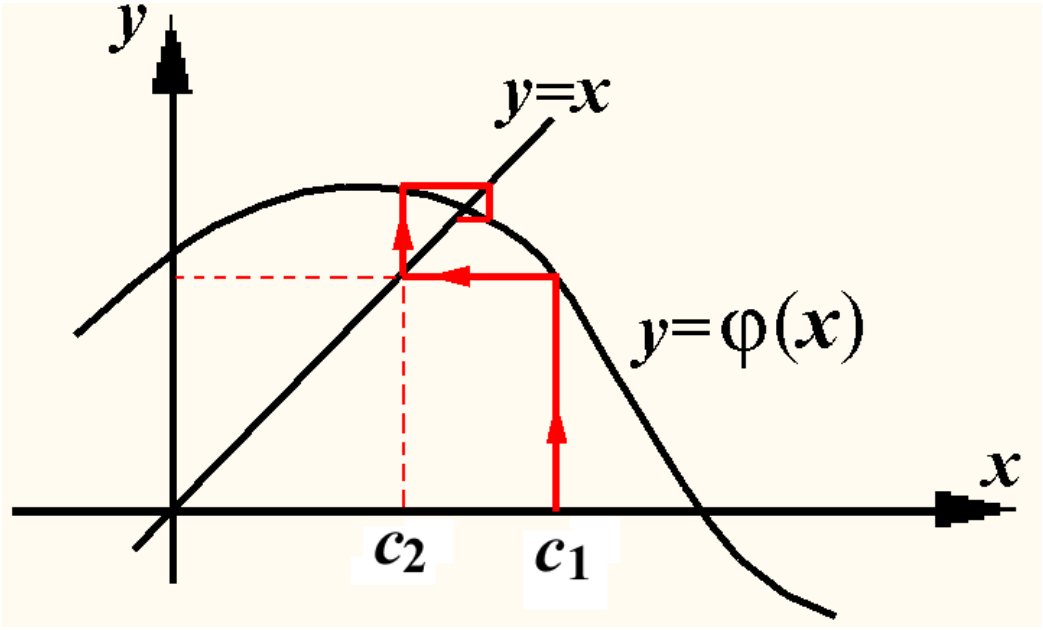
```
> c=0
> c= 0.4- 0.3*(1+c^4)
```

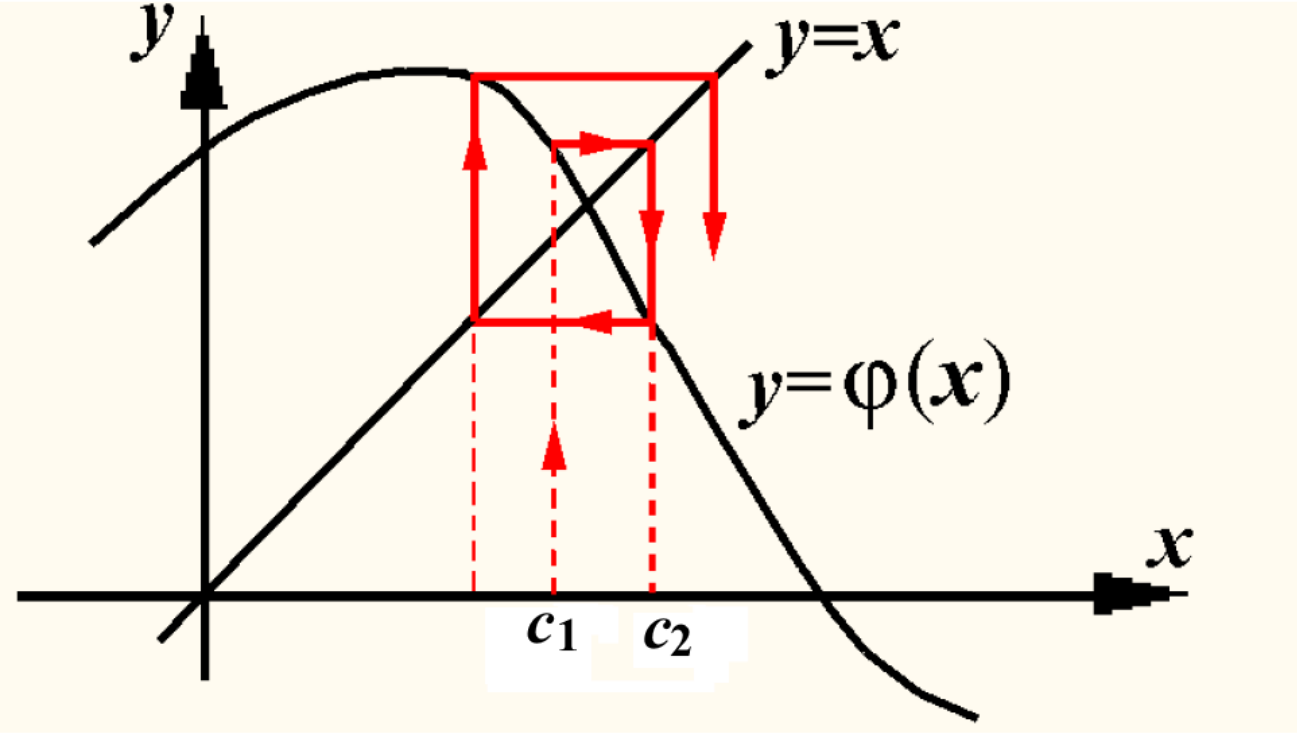
Geometric illustration of solving $x=\varphi(x)$:

Let us plot $y=x$, $y=\varphi(x)$



$$c_2 = \varphi(c_1)$$





In this figure, we see a divergence of successive approximations c_k , which move away from the exact solution x^* .

The convergence or divergence of the sequence c_k

depends on the slope of curve $y=\varphi(x)$ *to the x-axis, that is on the module of the first derivative :*

$$\left|\varphi'(x)\right|$$

Theorem *(sufficient condition for the convergence of iterations):*

If $\left|\varphi'(x)\right| < 1$ at $a \le x \le b$, *then*

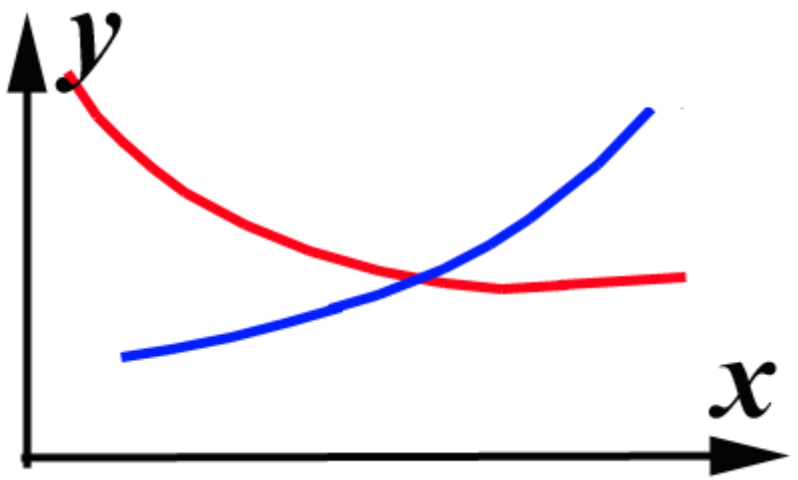
$c_k \to x^*$ *at* $k \to \infty$, *and*

x^* *is the unique root of the equation* $x=\varphi(x)$.

Chapter 2. Systems of nonlinear equations

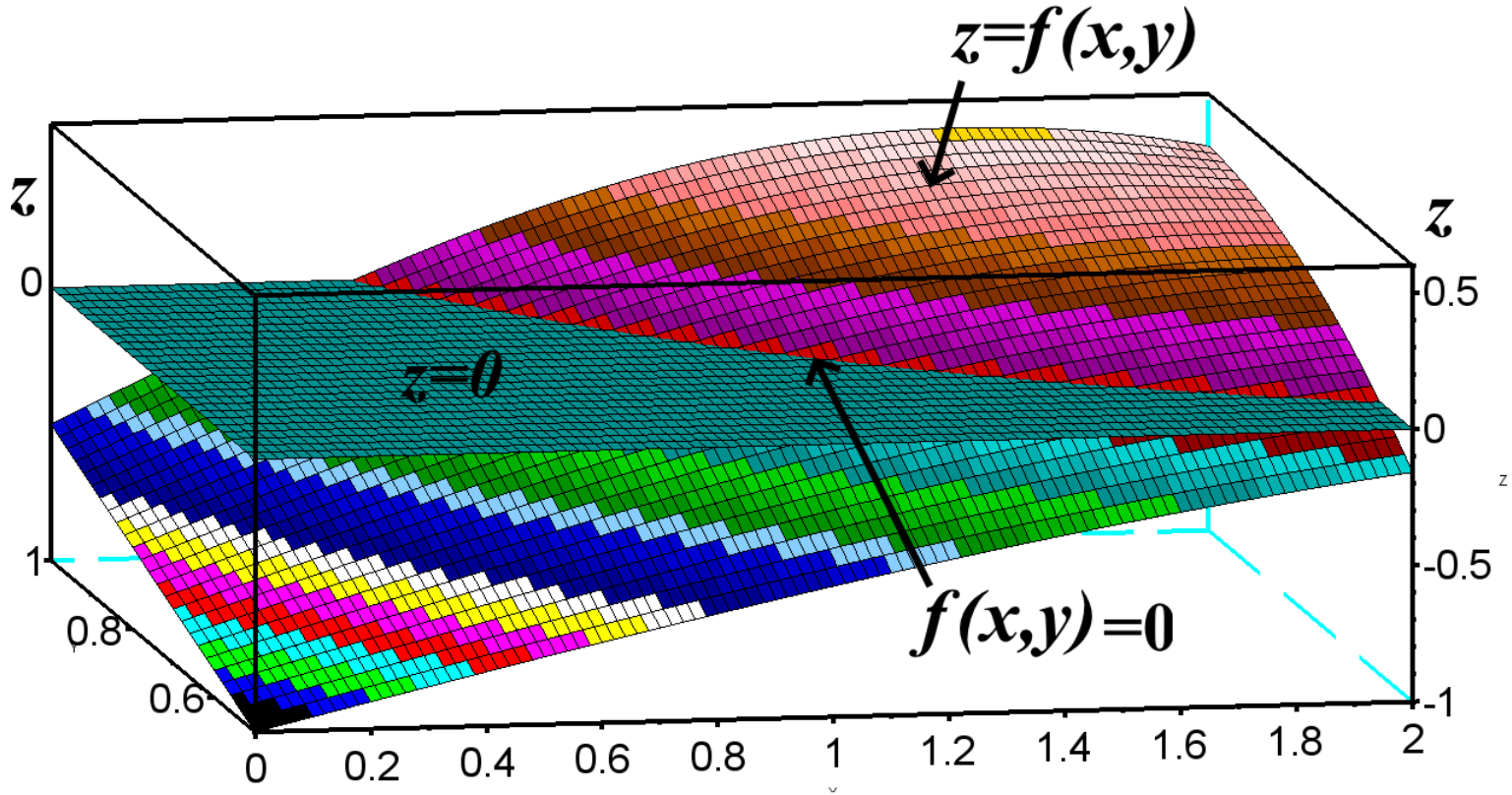
Now we turn to a system of 2 nonlinear equations with 2 unknowns x and y :

$$\begin{cases} f(x, y) = 0 \\ g(x, y) = 0 \end{cases}$$



Geometric interpretation in 2D:
typically each function determines a curve in (x, y) plane.
Intersection of the curves gives a solution.

Interpretation in 3D:



another surface illustrates function $z=g(x,y)$

Method of tangents (Newton's method)

Let us recall the method of tangent lines for a single equation

$$f(x)=0 \qquad a \le x \le b \ .$$

Algorithm for calculation of successive approximations to a solution:

$$c_{k+1}=c_k - f(c_k)/ f'(c_k) \qquad k=1,2,...$$

This was obtained by plotting a line which is tangent to curve $y= f(x)$ at an initial point $x=c_1$, $y= f(c_1)$ on the curve.

Method of tangents (Newton's method)

Let us recall the method of tangent lines for a single equation

$$f(x)=0 \qquad a \le x \le b\ .$$

Algorithm for calculation of successive approximations to a solution:

$$c_{k+1}=c_k - f(c_k)/f'(c_k) \qquad k=1,2,...$$

This was obtained by plotting a line which is tangent to curve $y=f(x)$ at an initial point $x=c_1$, $y=f(c_1)$ on the curve.

Similarly, in the case of 2 equations, we should obtain 2 planes which are tangent to surfaces $z=f(x, y)$, $z=g(x, y)$ at initial points $x_1, y_1, z_{1f}=f(x_1, y_1)$ and $x_1, y_1, z_{1g}=g(x_1,y_1)$.

Then we should:

1) **find intersections of the tangent planes with plane $z=0$; such intersections are 2 straight lines,**

2) **find intersection of the 2 lines in the plane $z=0$. It gives us a point x_2, y_2 which is the next approximation to the root.**

Formula for calculation of x_2, y_2 :

[in case of 1 equation it was $c_2 = c_1 - f(c_1)/f'(c_1)$]

$$\begin{pmatrix} x_2 \\ y_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ y_1 \end{pmatrix} - \mathrm{inv}(F'_1) \times \begin{pmatrix} f(x_1,y_1) \\ g(x_1,y_1) \end{pmatrix}$$

column vectors — 2 x 2 matrix — vector

$$F'_1 = \begin{pmatrix} \partial f/\partial x & \partial f/\partial y \\ \partial g/\partial x & \partial g/\partial y \end{pmatrix} \Bigg|_{\text{at } x_1, y_1}$$

matrix

inv – inverse of the matrix

$$F' = \begin{pmatrix} \partial f/\partial x & \partial f/\partial y \\ \partial g/\partial x & \partial g/\partial y \end{pmatrix}$$

$$\text{inv } F' = \frac{1}{f_x g_y - g_x f_y} \begin{pmatrix} \partial g/\partial y & -\partial f/\partial y \\ -\partial g/\partial x & \partial f/\partial x \end{pmatrix}$$

see Algebra

$f_x = \partial f/\partial x, \quad f_y = \partial f/\partial y, \ldots.$

Proof of the formula.

If a surface in space *(x,y,z)* is given by expression *G(x,y,z)*=0, then equation of the tangent plane is

$$G_x \cdot (x-x_0) + G_y \cdot (y-y_0) + G_z \cdot (z-z_0) = 0$$

(partial derivatives)

In the case under consideration $z=f(x,y)$ ➔ $f(x,y)-z=0$**:**

$$f_x \cdot (x-x_1) + f_y \cdot (y-y_1) - (z- z_{1f}) = 0$$

$g(x,y)-z=0$**:**

$$g_x \cdot (x-x_1) + g_y \cdot (y-y_1) - (z- z_{1g}) = 0$$

Find intersections with plane $z=0$**:**

$$f_x \cdot (x-x_1) + f_y \cdot (y-y_1) + z_{1f} = 0$$

$$g_x \cdot (x-x_1) + g_y \cdot (y-y_1) + z_{1g} = 0$$

Now find intersection of the lines:

$$\left.\begin{aligned} f_x \cdot (x_2-x_1) + f_y \cdot (y_2-y_1) + z_{1f} = 0 \\ g_x \cdot (x_2-x_1) + g_y \cdot (y_2-y_1) + z_{1g} = 0 \end{aligned}\right\}$$

$$\left.\begin{aligned} f_x\cdot(x_2-x_1) + f_y\cdot(y_2-y_1) + z_{1f}=0 \\ g_x\cdot(x_2-x_1) + g_y\cdot(y_2-y_1) + z_{1g}=0 \end{aligned}\right\}$$

All derivatives are calculated at $x=x_1, y=y_1$

In matrix form:

$$\begin{pmatrix} f_x & f_y \\ g_x & g_y \end{pmatrix} \times \begin{pmatrix} x_2-x_1 \\ y_2-y_1 \end{pmatrix} + \begin{pmatrix} z_{1f} \\ z_{1g} \end{pmatrix} = 0$$

$$\mathbf{F'_1} \times \begin{pmatrix} x_2-x_1 \\ y_2-y_1 \end{pmatrix} + \begin{pmatrix} z_{1f} \\ z_{1g} \end{pmatrix} = 0$$

multiply by $\mathbf{inv(F'_1)}$**:**

$$\begin{pmatrix} x_2 - x_1 \\ \\ y_2 - y_1 \end{pmatrix} + \mathbf{inv(F'_1)} \times \begin{pmatrix} z_{1f} \\ \\ z_{1g} \end{pmatrix} = 0$$

$$\begin{pmatrix} x_2 \\ \\ y_2 \end{pmatrix} = \begin{pmatrix} x_1 \\ \\ y_1 \end{pmatrix} - \mathbf{inv(F'_1)} \times \begin{pmatrix} f(x_1,y_1) \\ \\ g(x_1,y_1) \end{pmatrix}$$

General formula:

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \mathrm{inv}(F'_k) \times \begin{pmatrix} f(x_k,y_k) \\ g(x_k,y_k) \end{pmatrix}$$

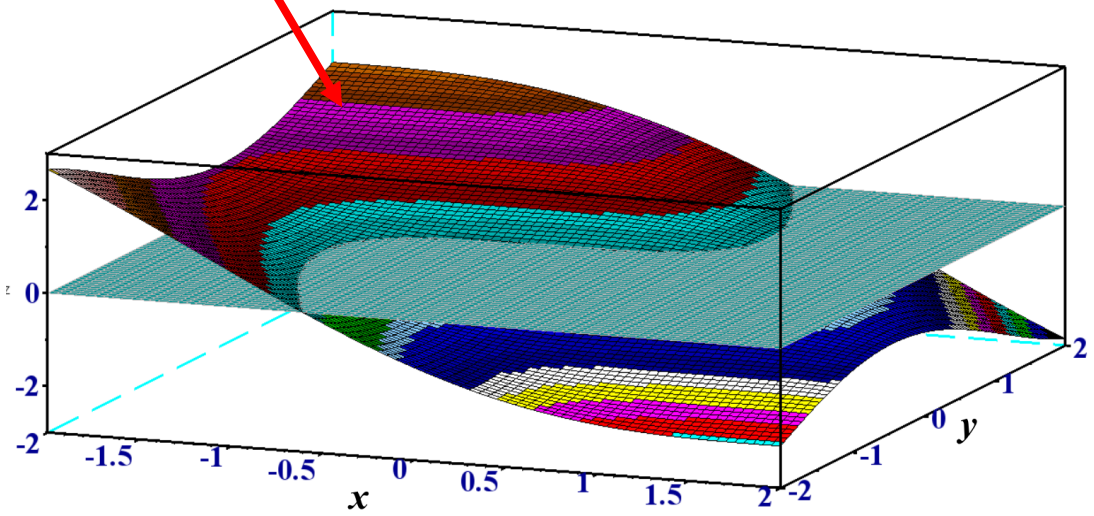
column vectors — matrix — column vector

k=1, 2, 3, . . .

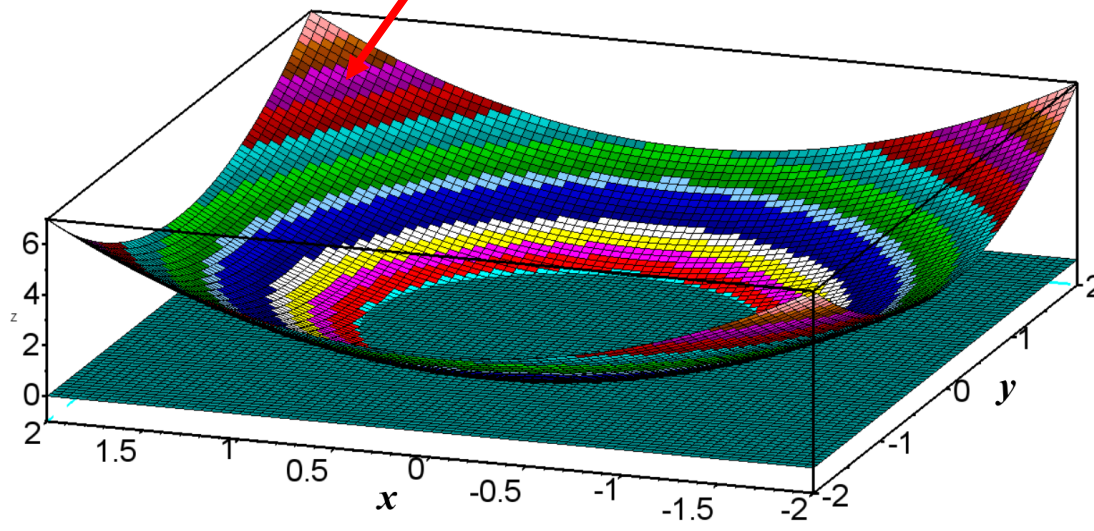
Example: $\sin(x+y) - x - 0.1 = 0$

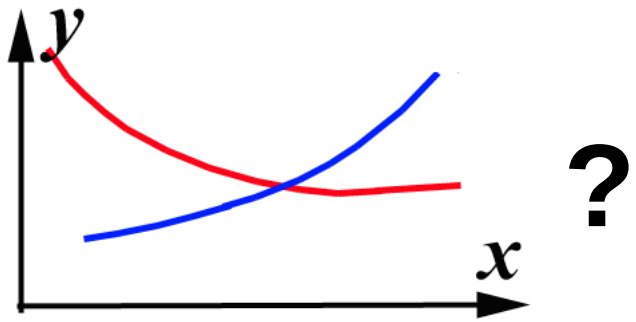
$x^2 + y^2 - 1 = 0$

$z=f(x,y)$



$z=g(x,y)$





?

Scilab:

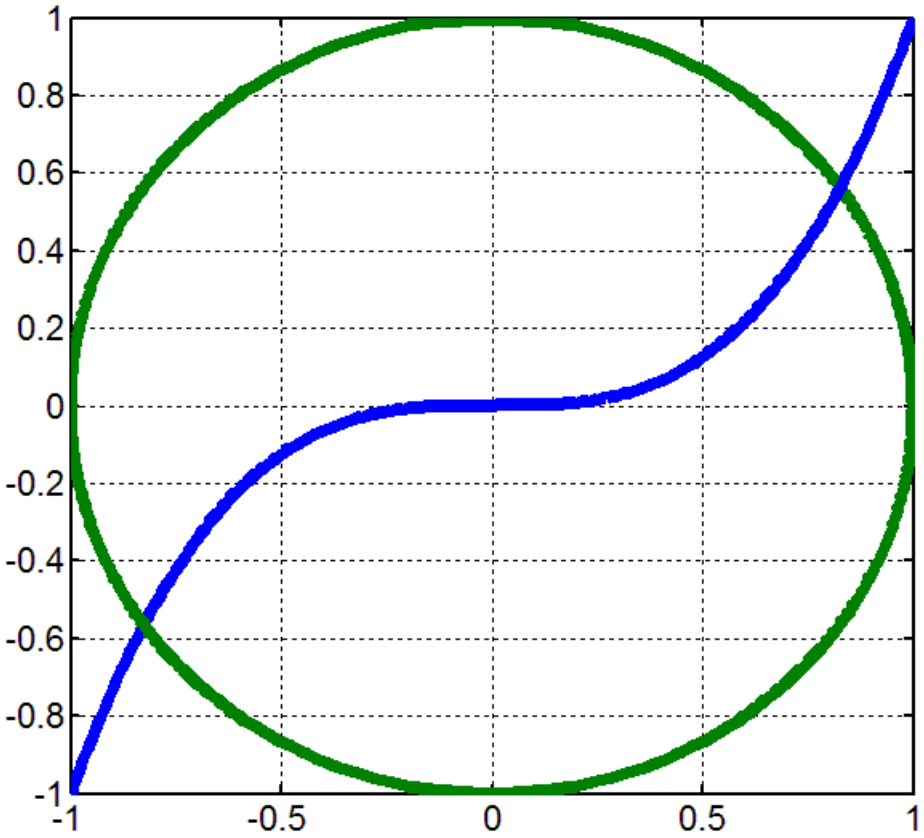
```
clear
 x=0
 y=1
 X=[x y]'
 for i=1:50
     f=sin(x+y)-x-0.1
     g=x*x+y*y-1
     F=[f  g]'
     dfdx=cos(x+y)-1
     dfdy=cos(x+y)
     dgdx=2*x
     dgdy=2*y
     Fderivat=[dfdx  dfdy;  dgdx  dgdy]
     X=X-inv(Fderivat)*F
     x=X(1)
     y=X(2)
     disp(X)
 end
```

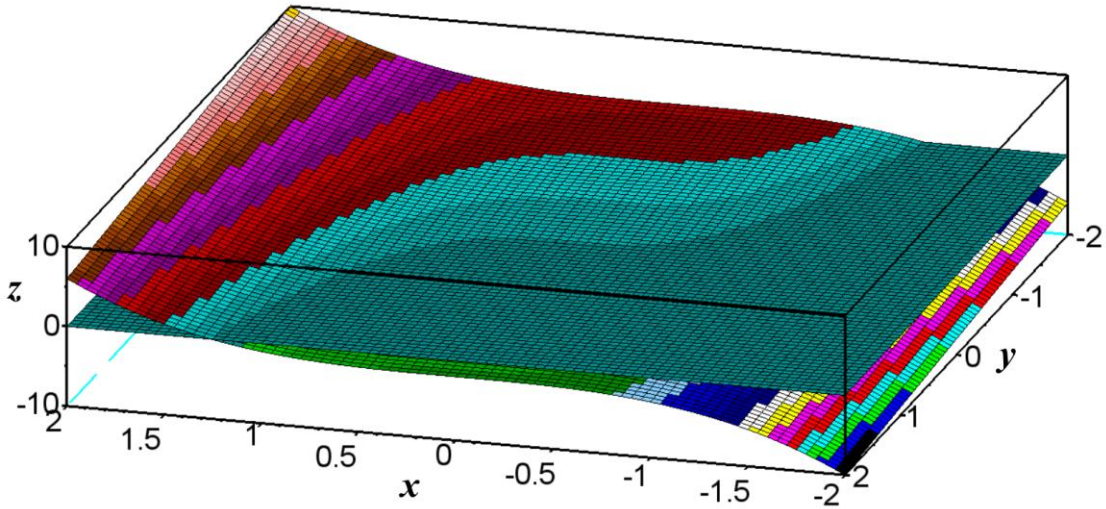
Answer: 0.8772863 0.4799675

Example of non-unique solution:

$$x^2 + y^2 - 1 = 0$$

$$x^3 - y = 0$$





Method of solving a system of equations by a transition to a minimization problem:

$$\begin{cases} f(x,y)=0 \\ g(x,y)=0 \end{cases}$$

$$F(x,y) = f^2 + g^2 \rightarrow \min$$

Example: $x^2 + y^2 - 1 = 0$

$x^3 - y = 0$

$0.5<x<1, \quad 0.4<y<0.6$

Scilab:

```
clear     // Method of transition to minimization problem
x=0.5 : 0.001 : 1
y=0.4 : 0.001 : 0.6
min=10
for i=1:501
for j=1:201
f1= x(i)^2 + y(j)^2 -1
f2= x(i)^3 -y(j)
F=f1*f1 + f2*f2
if (F<min)
min=F
xmin=x(i)
ymin=y(j)
end
end
end
disp(min, xmin, ymin)
```

Same example using EXCEL: $x^2 + y^2 - 1 = 0$

$x^3 - y = 0$

	A	B	C
1	0.5	0.5	
2	=A1*A1+B1*B1-1	=A1^3-B1	=A2*A2+B2*B2

Solve (the extension of EXCEL)

Target cell: C2

Search of minimum

Changing cells: A1; B1

Answer:

1) A1= 0.82603144 B1= 0.56362407

2) A1=−0.82603144 B1=−0.56362407

Using EXCEL, solve the example already solved with Scilab:

$$\sin(x+y) - x - 0.1 = 0$$

$$x^2 + y^2 - 1 = 0$$

Three equations

$$x^2 + y^2 + z^2 - 1 = 0$$

$$2x^2 + y^2 - 4z = 0$$

$$3x^2 - 4y + z^2 = 0$$

```
x= 1
y= 1
z= 1
X=[x y z]'
for i=1:15
    f=x*x+y*y+z*z-1
    g=2*x*x +y*y -4*z
    h=3*x*x-4*y+z*z
    F=[f  g  h]'
    dfdx=2*x
    dfdy=2*y
    dfdz=2*z
    dgdx=4*x
    dgdy=2*y
    dgdz=-4
    dhdx=6*x
    dhdy=-4
    dhdz=2*z
    Fderivat=[dfdx  dfdy dfdz;  dgdx  dgdy dgdz; dhdx dhdy dhdz]
    X=X-inv(Fderivat)*F
    x=X(1)
    y=X(2)
    z=X(3)
    disp(X)
end
```

Chapter 3
Systems of linear algebraic equations

$$a x + b y + c z = e$$
$$f x + g y + h z = l$$
$$p x + q y + s z = t$$

x, y, z *are unknowns*

Chapter 3
Systems of linear algebraic equations

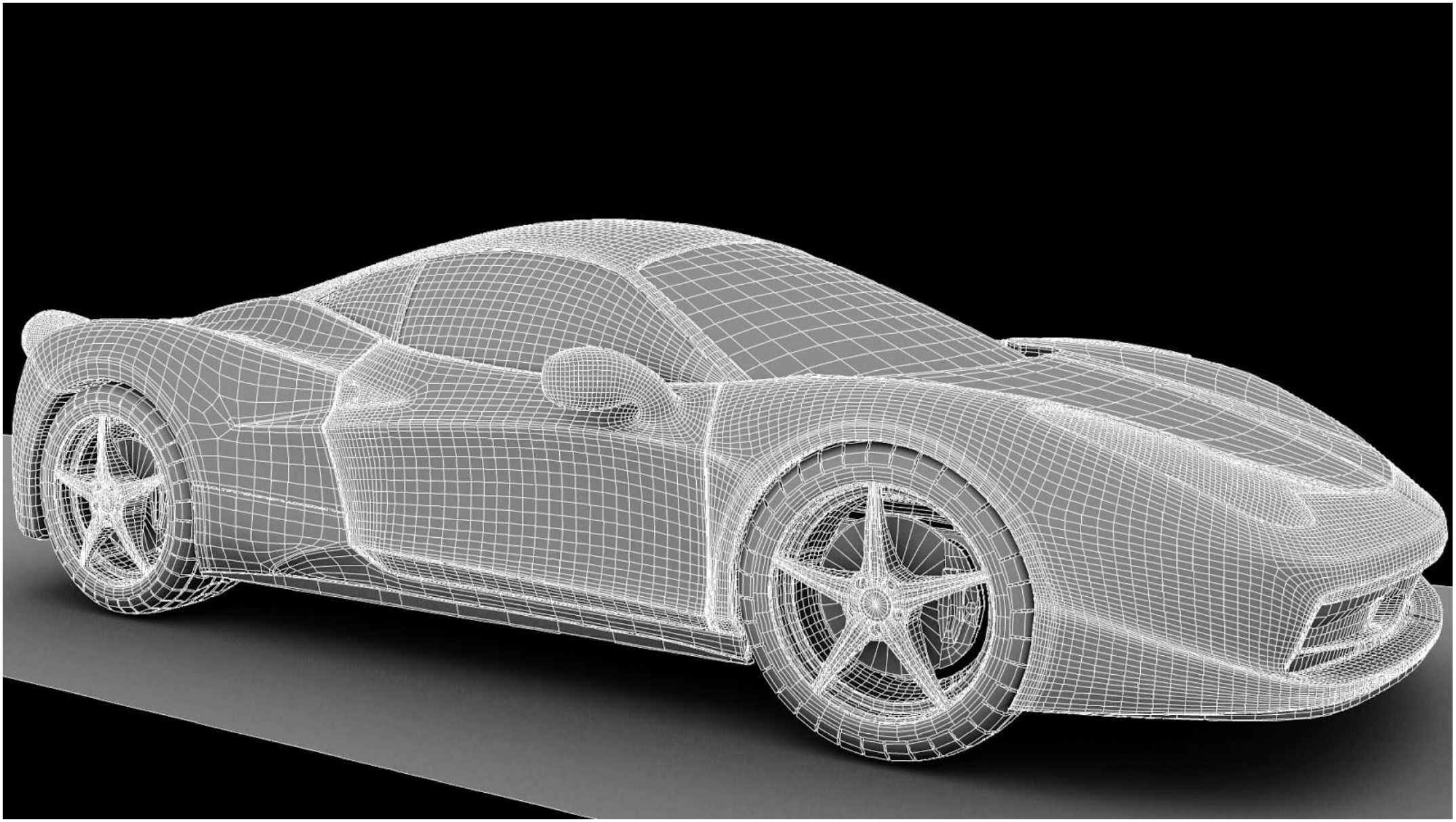
$$a x + b y + c z = e$$
$$f x + g y + h z = l$$
$$p x + q y + s z = t$$

x, y, z *are unknowns*

Change the notations:

$$a x_1 + b x_2 + c x_3 = b_1$$
$$f x_1 + g x_2 + h x_3 = b_2$$
$$p x_1 + q x_2 + s x_3 = b_3$$

x_1, x_2, x_3 *are unknowns*



$$a_{11}\,x_1 + a_{12}\,x_2 + a_{13}\,x_3 + \ldots + a_{1n}\,x_n = b_1\,,$$

$$a_{21}\,x_1 + a_{22}\,x_2 + a_{23}\,x_3 + \ldots + a_{2n}\,x_n = b_2\,,$$

- -

$$a_{i1}\,x_1 + a_{i2}\,x_2 + a_{i3}\,x_3 + \ldots + a_{in}\,x_n = b_i\,,$$

- -

$$a_{n1}\,x_1 + a_{n2}\,x_2 + a_{n3}\,x_3 + \ldots + a_{nn}\,x_n = b_n\,.$$

where a_{ij} - given real numbers

x_i - unknowns to be found

The system can be written in matrix form:

$$Ax=b$$

$$A=\begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ - & - & - & - & - \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} \\ - & - & - & - & - \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{pmatrix}$$

$$x=\begin{pmatrix} x_1 \\ x_2 \\ - \\ x_i \\ - \\ x_n \end{pmatrix}$$ column vector

$$b=\begin{pmatrix} b_1 \\ b_2 \\ - \\ b_i \\ - \\ b_n \end{pmatrix}$$ column vector

$Ax=b$

If $\det A \neq 0$,

$$\begin{vmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{vmatrix} \neq 0$$

then there exists a unique solution x **of the system.**

Methods for computation of the solution:

1. Using the inverse $\mathrm{inv}(A)$ of matrix A
2. Gaussian elimination of unknowns
3. LU factorization
4. Iteration method
5.
6.

Scilab (www.scilab.org):

```
-->A= [2 3 4 5 ; 2 1 3 4; 5 6 8 9 ; 5 6 4 3]
 A  =
    2.    3.    4.    5.
    2.    1.    3.    4.
    5.    6.    8.    9.
    5.    6.    4.    3.

-->A(4,2)=60

 A  =
    2.    3.     4.    5.
    2.    1.     3.    4.
    5.    6.     8.    9.
    5.    60.    4.    3.
```

--> b= [3; 2; 5; 7] column vector

```
 b  =
    3.
    2.
    5.
    7.
```

--> b= [4 3 0   9]' column vector

```
  b  =
    4.
    3.
    0.
    9.
```

Product of a matrix and column vector:

--> A*b ➔ column vector

```
--> x=0 : 0.2 : 0.6
 x  =   0.    0.2    0.4    0.6
```

row vector (1 row, 4 columns)

Product of a row vector and a column vector:

```
--> w=x*b
      w = x(1)*b(1)+x(2)*b(2)+x(3)*b(3)+x(4)*b(4)=
```

result is a number

det(A)

1) Method of the inverse of matrix A

$$Ax=b$$

we denote by $\mathrm{inv}(A)$ or A^{-1} the inverse of matrix A

How to compose A^{-1} : by computing its cofactors, see Algebra

If we got A^{-1}, then multiplying the system by A^{-1}

$$A^{-1}Ax = A^{-1}b$$

$$A^{-1}A = I = \begin{pmatrix} 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & 0 & \dots & 0 \\ 0 & 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 0 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 0 & \dots & 1 \end{pmatrix}$$

identity matrix

$Ix = A^{-1}b$ **and we obtain the solution:** $x = A^{-1}b$

Scilab:

```
A=
b=
det(A)
x=inv(A)*b
```

`x=A\b` is an equivalent way of using `inv(A)`

1)

A				b
- 2.	4.	2.	- 2.	14
1.	2.	3.	1.	-21
0.	2.	3.	1.	19
1.	4.	3.	1.	-11

3)

A				b
3.	4.	2.	-2.	14
1.	-2.	3.	0.	-22
0.	2.	3.	0.	18
1.	4.	3.	1.	14

2)

A				b
-6.	4.	2.	- 2.	24
1.	2.	3.	5.	-21
-4.	2.	3.	1.	19
1.	4.	3.	1.	-11

4)

A				b
4.	4.	2.	-2.	4
1.	-2.	6.	2.	-12
-2.	2.	3.	0.	18
1.	4.	3.	-2.	14

2. Gaussian elimination

Carl Friedrich Gauss 1777-1855

$$a_{11}\,x_1 + a_{12}\,x_2 + a_{13}\,x_3 + \ldots + a_{1n}\,x_n = b_1\,,$$
$$a_{21}\,x_1 + a_{22}\,x_2 + a_{23}\,x_3 + \ldots + a_{2n}\,x_n = b_2\,,$$
$$- -$$
$$a_{i1}\,x_1 + a_{i2}\,x_2 + a_{i3}\,x_3 + \ldots + a_{in}\,x_n = b_i\,,$$
$$- -$$
$$a_{n1}\,x_1 + a_{n2}\,x_2 + a_{n3}\,x_3 + \ldots + a_{nn}\,x_n = b_n\,.$$

Suppose that $a_{11} \neq 0$. Then

$$x_1 = (b_1 - a_{12} x_2 - a_{13} x_3 - \ldots - a_{1n} x_n)/ a_{11}$$

we multiply this by a_{i1}:

$$a_{i1} x_1 = a_{i1}(b_1 - a_{12} x_2 - a_{13} x_3 - \ldots - a_{1n} x_n)/a_{11} \qquad (*)$$

and replace $a_{i1} x_1$ in i^{th} equation by (*):

$$a_{i1} x_1 + a_{i2} x_2 + \ldots + a_{ij} x_j + \ldots + a_{in} x_n = b_i \qquad i^{th} \text{ equation}$$

$$a_{i1}(b_1 - a_{12} x_2 - a_{13} x_3 - \ldots - a_{1n} x_n)/a_{11}$$
$$+ a_{i2} x_2 + \ldots + a_{ij} x_j + \ldots + a_{in} x_n = b_i$$

We now sum up the coefficients in front of the same x_j

$$a_{i2}^{(1)} x_2 + \ldots + a_{ij}^{(1)} x_j + \ldots + a_{in}^{(1)} x_n = b_i^{(1)}$$

where $a_{i2}^{(1)} = a_{i2} - a_{i1}a_{12}/a_{11}$

$a_{i3}^{(1)} = a_{i3} - a_{i1}a_{13}/a_{11}$

$a_{ij}^{(1)} = a_{ij} - a_{i1}a_{1j}/a_{11}$

$b_i^{(1)} = b_i - a_{i1}b_1/a_{11}$ $\qquad i=2,3,\ldots, n$

After elimination of x_1 , the system becomes

$$a_{11}\,x_1 + a_{12}\,x_2 + \ldots + a_{1j}\,x_j + \ldots + a_{1n}\,x_n = b_1$$

$$a_{22}^{(1)}\,x_2 + \ldots + a_{2j}^{(1)}\,x_j + \ldots + a_{2n}^{(1)}\,x_n = b_2^{(1)}$$

- -

$$a_{i2}^{(1)}\,x_2 + \ldots + a_{ij}^{(1)}\,x_j + \ldots + a_{in}^{(1)}\,x_n = b_i^{(1)}$$

- -

$$a_{n2}^{(1)}\,x_2 + \ldots + a_{nj}^{(1)}\,x_j + \ldots + a_{nn}^{(1)}\,x_n = b_n^{(1)}$$

Suppose that $a_{22}^{(1)} \neq 0$

Keeping on the elimination of unknowns, in the same way we obtain

$$a_{11}\,x_1 + a_{12}\,x_2 + \ldots + a_{1j}\,x_j + \ldots + a_{1n}\,x_n = b_1$$

$$a_{22}^{(1)}\,x_2 + \ldots + a_{2j}^{(1)}\,x_j + \ldots + a_{2n}^{(1)}\,x_n = b_2^{(1)}$$

$$- -$$

$$a_{n-1,\,n-1}^{(n-2)}\,x_{n-1} + a_{n-1,n}^{(n-2)}\,x_n = b_{n-1}^{(n-2)}$$

$$a_{nn}^{(n-1)}\,x_n = b_n^{(n-1)}$$

If $a_{nn}^{(n-1)} \neq 0$, then $x_n = b_n^{(n-1)} / a_{nn}^{(n-1)}$

$x_{n-1} =$

- It could happen that $a_{ii}^{(i-1)}=0$.

Example. $A=\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$

$$0\cdot x_1 + x_2 = b_1$$

$$x_1 + 0\cdot x_2 = b_2$$

Theorem The system of n linear algebraic equations with $\det A \neq 0$ can be transformed to an equivalent system with nonzero $a_{ii}^{(i-1)}$ by transposition (interchange) of columns or rows.

Scilab:

C=rref([A b]) ; - **Gaussian elimination**

(Reduced Row Echelon Form)

```
-->C=rref([A b])
C =
1. 0. 0. 0.   2.5028409
0. 1. 0. 0.  - 0.1382955
0. 0. 1. 0.  - 0.72625
0. 0. 0. 1.  - 0.0971591
```

x_1

x_2

x_3

x_4

x=C(:,5)

The number of arithmetic operations necessary for obtaining a solution with Gaussian elimination is

$$2n(n+1)(n+2)/3 + n(n-1)$$

(a proof is available in textbooks).

3) LU factorization method

Let *Ax=b* denote the linear system to be solved, where *A* is *n*x*n* size matrix. In Gaussian elimination, the system was reduced to the upper triangular system *Ux=g* with

$$U = \begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ . & & \cdots & . \\ . & & \cdots & . \\ . & & \cdots & . \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}$$

Let us introduce an auxiliary lower triangular matrix *L* :

$$L = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ m_{21} & 1 & \cdots & 0 \\ . & & \cdots & . \\ . & & \cdots & . \\ . & & \cdots & . \\ m_{n1} & \cdots & m_{nn-1} & 1 \end{bmatrix}$$

The relationship of the matrices L and U to the original A is given by the following theorem:

Theorem. Let A be a matrix with $det\ A \neq 0$. Then if U is produced as Gaussian elimination without interchange of rows/columns, then there exists triangular matrix L such that $LU=A$ and this is called factorization of A.

The factorization leads to a slightly different way of solving the system $Ax=b$. It can be rewritten as $LUx=b$. We denote $Ux=g$ and obtain the two simple systems

$$Lg=b \quad \text{and} \quad Ux=g \ .$$

Both L and U are triangular, therefore solutions can be easily calculated by substitution. The computational cost is here reduced drastically as compared to Gaussian one.

Instead of constructing L and U by using the elimination steps, it is possible to solve directly elements of these matrices.
We will illustrate the direct computation of L and U in the case n=3:

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1 & -1 \\ 1 & 2 & -2 \\ -2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} & u_{13} \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

1st row of A: $1=1\cdot u_{11}$ $1=1\cdot u_{12}$ $-1=1\cdot u_{13}$

$$\begin{bmatrix} 1 & 1 & -1 \\ 1 & 2 & -2 \\ -2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ m_{21} & 1 & 0 \\ m_{31} & m_{32} & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & -1 \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

$1= m_{21}\cdot 1$

$-2= m_{31}\cdot 1$

$$\begin{bmatrix} 1 & 1 & -1 \\ 1 & 2 & -2 \\ -2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ -2 & m_{32} & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & -1 \\ 0 & u_{22} & u_{23} \\ 0 & 0 & u_{33} \end{bmatrix}$$

$-2 = -1 + u_{23}$

$2 = 1 + u_{22}$

$$\begin{bmatrix} 1 & 1 & -1 \\ 1 & 2 & -2 \\ -2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ -2 & m_{32} & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & u_{33} \end{bmatrix}$$

$1 = 2 - m_{32} + u_{33}$

$1 = -2 + m_{32}$

$$\begin{bmatrix} 1 & 1 & -1 \\ 1 & 2 & -2 \\ -2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ -2 & 3 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 2 \end{bmatrix}$$

L U

Take, for example, $b=\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

$$LUx=b$$

$$Lg = b$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ -2 & 3 & 1 \end{pmatrix} \begin{pmatrix} g_1 \\ g_2 \\ g_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} \quad \begin{array}{l} \Rightarrow \quad g_1=1 \\ \Rightarrow \quad g_2=0 \\ \Rightarrow \quad g_3=3 \end{array}$$

Finally, we solve $Ux=g$:

$$\begin{pmatrix} 1 & 1 & -1 \\ 0 & 1 & -1 \\ 0 & 0 & 2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 3 \end{pmatrix}$$

$$\Rightarrow \quad x_1 + x_2 - x_3 = 1 \quad \Rightarrow x_1 = 1$$

$$\Rightarrow \quad x_2 = 3/2$$

$$\Rightarrow \quad x_3 = 3/2$$

x=linsolve(A, d) *solves the system* **Ax+d=0** **with LU factorization**

x=linsolve(A,-b) *solves the system* **Ax=b**

Chapter 3-2. Systems of linear algebraic equations

4) Iteration method $Ax=b$ $\det A \neq 0$

$$A=\begin{pmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ - & - & - & - & - \\ a_{i1} & a_{i2} & a_{i3} & \dots & a_{in} \\ - & - & - & - & - \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{pmatrix}$$

$$x=\begin{pmatrix} x_1 \\ x_2 \\ - \\ x_i \\ - \\ x_n \end{pmatrix} \qquad b=\begin{pmatrix} b_1 \\ b_2 \\ - \\ b_i \\ - \\ b_n \end{pmatrix}$$

Iteration method defines a sequence of approximate solutions $x^{(k)}$ that converge to the exact solution $x^{(k)} \to x^*$ as $k \to \infty$

Let us transform $Ax=b$ to the form $x=Cx+d$, then choose some $x^{(1)}$ and organize iterations:

$$x^{(k+1)}=Cx^{(k)}+d\,, \qquad k=1,2,\ldots$$

Example: $1.01\,x_1 + 0.2\,x_2 = 3 \;\Rightarrow\; (1+0.01)\,x_1 + 0.2\,x_2 = 3$

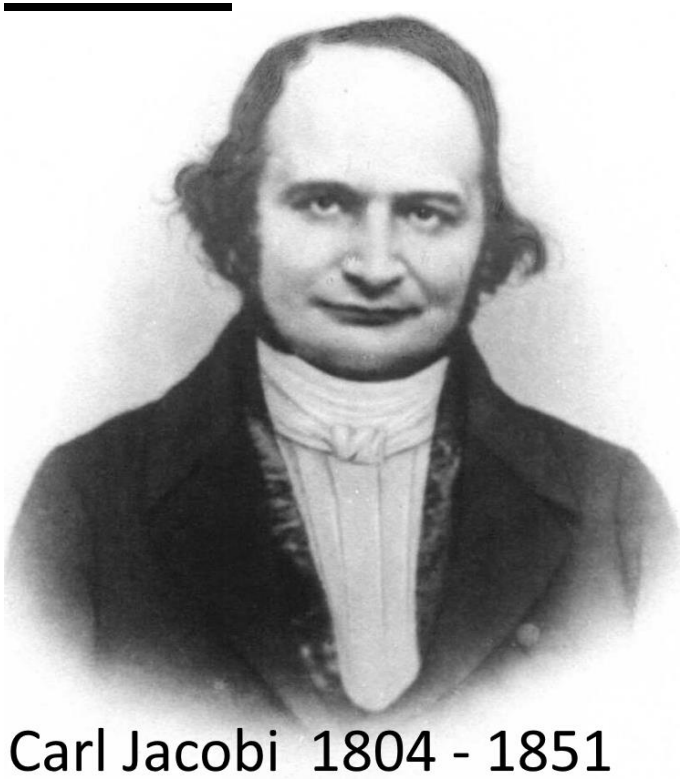
$0.05\,x_1 + 1.08\,x_2 = 2 \;\Rightarrow\; 0.05\,x_1 + (1+0.08)\,x_2 = 2$

$$x_1 = 3 - 0.01\,x_1 - 0.2\,x_2$$
$$x_2 = 2 - 0.05\,x_1 - 0.08\,x_2$$

choose $x_1^{(1)}=3\,,\; x_2^{(1)}=2$

$$x_1^{(k+1)} = 3 - 0.01\,x_1^{(k)} - 0.2\,x_2^{(k)}$$
$$x_2^{(k+1)} = 2 - 0.05\,x_1^{(k)} - 0.08\,x_2^{(k)}$$

A way of transforming $Ax=b$ to the form $x=Cx+d$ is <u>Jacobi iterative method</u>



Carl Jacobi 1804 - 1851

We illustrate it for the case n=3 :

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$

$$a_{21}x_1 + a_{22}x_2 + a_{23}x_3 = b_2$$

$$a_{31}x_1 + a_{32}x_2 + a_{33}x_3 = b_3$$

Suppose the diagonal elements a_{ii} are non-zero and divide the first equation by a_{11}, the second by a_{22} and third by a_{33}:

$$x_1 = \frac{1}{a_{11}}(b_1 - a_{12}x_2 - a_{13}x_3)$$

$$x_2 = \frac{1}{a_{22}}(b_2 - a_{21}x_1 - a_{23}x_3)$$

$$x_3 = \frac{1}{a_{33}}(b_3 - a_{31}x_1 - a_{32}x_2)$$

We choose

$$x^{(1)} = \begin{pmatrix} x_1^{(1)} \\ x_2^{(1)} \\ x_3^{(1)} \end{pmatrix}$$

and define iterations:

$$x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)})$$

$$x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k)} - a_{23}x_3^{(k)})$$

$$x_3^{(k+1)} = \frac{1}{a_{33}}(b_3 - a_{31}x_1^{(k)} - a_{32}x_2^{(k)})$$

$$x^{(k+1)} = Cx^{(k)} + \beta \ , \qquad \beta_i = b_i / a_{ii}$$

diagonal elements of matrix C are zeros in the Jakobi method

$$x^{(k+1)} = Cx^{(k)} + \beta$$

$$C = \begin{pmatrix} c_{11} & c_{12} & c_{13} & \dots & c_{1n} \\ c_{21} & c_{22} & c_{23} & \dots & c_{2n} \\ - & - & - & - & - \\ c_{i1} & c_{i2} & c_{i3} & \dots & c_{in} \\ - & - & - & - & - \\ c_{n1} & c_{n2} & c_{n3} & \dots & c_{nn} \end{pmatrix}$$

Theorem. A sufficient condition for convergence of $x^{(k)}$ to exact solution $x^{(k)} \to x^*$ is $\|C\| < 1$, where $\|C\|$ is a norm of matrix C.

(Proof is omitted).

The convergence means $x_i^{(k)} \to x_i^*$ for any i

For the properties of a norm, see textbook by S.Sastry.

There are 3+ types of matrix norms:

1) $$\|C\|_1 = \max_{1 \le j \le n} \sum_{i=1}^{n} |c_{ij}|$$ **"column" norm**

2) $$\|C\|_\infty = \max_{1 \le i \le n} \sum_{j=1}^{n} |c_{ij}|$$ **"row" norm**

$$x_1 = 0.01\, x_1 - 0.2\, x_2 + 3$$
$$x_2 = -0.05\, x_1 + 0.08\, x_2 + 2$$

3) **Euclidean norm:**

$$\|C\|_e = \left(\sum_{i,j=1}^{n} |c_{ij}|^2 \right)^{1/2}$$

Column vector norms:

For the vector

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

some useful norms are

$$\| x \|_1 = |x_1| + |x_2| + \cdots + |x_n| = \sum_{i=1}^{m} |x_i|$$

$$\| x \|_2 = \sqrt{|x_1|^2 + |x_2|^2 + \cdots + |x_n|^2} = \left[\sum_{i=1}^{n} |x_i|^2 \right]^{1/2} = \| x \|_e$$

$$\| x \|_\infty = \max_i |x_i| .$$

The norm $\|\bullet\|_2$ is called the *Euclidean* norm since it is just the formula for distance in the [ ] Euclidean space.

$$A=\begin{pmatrix} 0 & 1 & 0 \\ 1.1 & -0.5 & 0 \\ 0 & 1 & 1.2 \end{pmatrix}$$

Scilab :

norm(A,1)

norm(A,'inf') **(Matlab : norm(A,'infty')**

Note: condition $\|C\| < 1$ is analogous to the condition $|\varphi'(x)| < 1$ in the section of Chapter 1 addressing the nonlinear equation $f(x)=0$ $x=\varphi(x)$

Theorem (on the accuracy of successive approximations $x^{(k)}$) : $x^{(k+1)}=Cx^{(k)}+\beta$

1)

$$\|x^{(k)} - x^*\| \le \frac{\|C\|^k}{1-\|C\|} \|\beta\|$$

2)

$$\|x^{(k)} - x^*\| \le \frac{\|C\|}{1-\|C\|} \|x^{(k+1)}-x^{(k)}\|$$

P. Seidel 1821-1896

A modified version of the Jacobi iteration method is Seidel method:

Seidel suggested to immediately insert the calculated $x_i^{(k+1)}$ ***into the right-hand sides of next equations. This accelerates the convergence*** $x^{(k)} \rightarrow x^*$

$$x_1^{(k+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(k)} - a_{13}x_3^{(k)})$$

$$x_2^{(k+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(k+1)} - a_{23}x_3^{(k)})$$

$$x_3^{(k+1)} = \frac{1}{a_{33}}(b_3 - a_{31}x_1^{(k+1)} - a_{32}x_2^{(k+1)})$$

The number of arithmetic operations per iteration is $\approx n^2$

Therefore, the effectiveness of iterative methods essentially depends on the required number of iterations k

Ill-conditioned systems of linear equations

Sometimes small changes in the coefficients of the system produce large changes in the solution. Such systems are called ill-conditioned. This can usually be expected when det(A) is small.

Example:

$$\left.\begin{aligned} 2.01x + y &= 4 \\ 2x + y &= 2 \end{aligned}\right\}$$

For ill-conditioned systems, the errors of approximate solutions $x^{(k)}$ are typically large.

Meanwhile the accuracy of approximate solutions $x^{(k)}$ can be improved using an iterative procedure and substitutions.

<u>Notice</u>:
If small changes in the coefficients of the system produce small changes in the solution, then the system is called well-conditioned.

Chapter 4. Trapezoids method for calculation of definite integrals

$$\int_a^b f(x)\,dx$$

When do you need to use a numerical method?

1st case: Function $f(x)$ is given by a formula; however, the integral cannot be expressed in terms of elementary functions $sin(x)$, $cos(x)$, $tan(x)$, $exp(x)$,...

For example,

$$\int_a^b e^{x \sin(\cos(\sin x))}\,dx$$

2nd case: values of function $f(x)$ are only given at finite number of points of the segment $[a,b]$:

$$y_0, y_1, y_2, \ldots, y_n$$

$$at \qquad a=x_0, x_1, x_2, \ldots, x_n=b$$

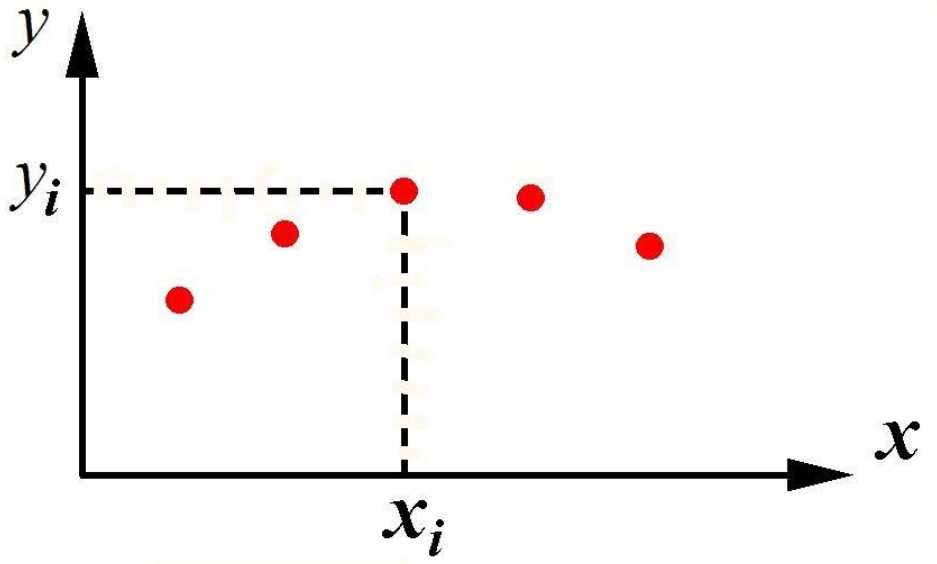
(for example, y_i are results of some experiments).

$$I=\int_{x_0}^{x_n} f(x)\,dx = \ ?$$

In case 1, when $f(x)$ is given by a formula, we can choose points x_i ourselves and calculate $y_i=f(x_i)$

Assume that $h_i = x_{i+1} - x_i = \text{const} = h$

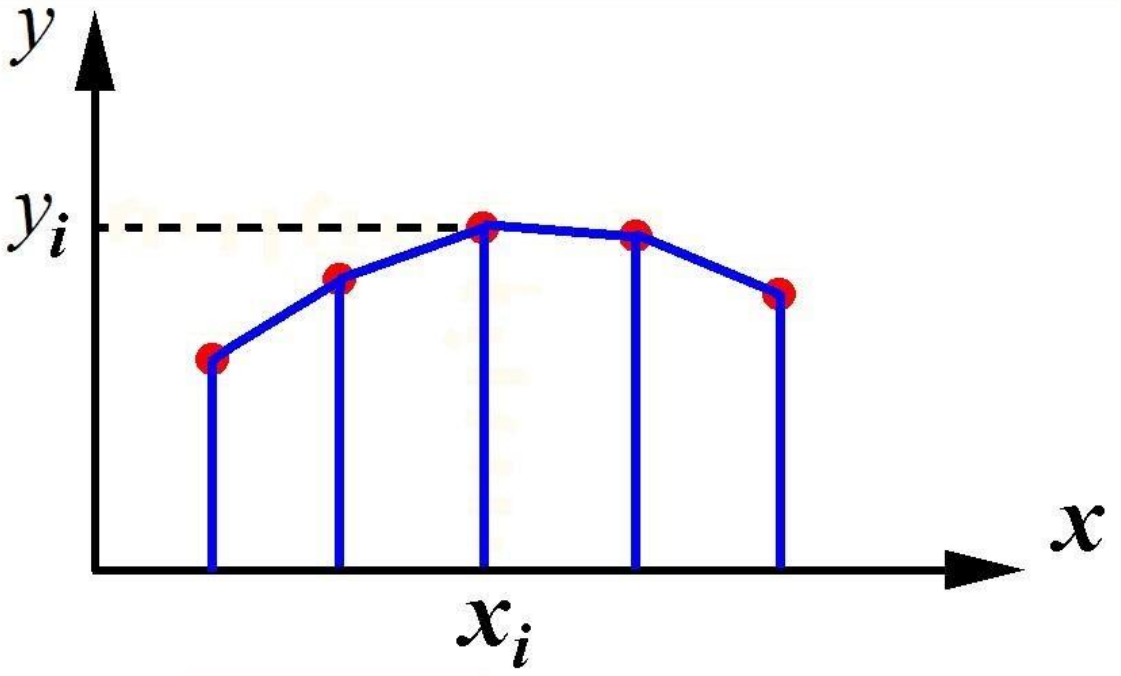
$$I = \int_{x_0}^{x_n} f(x)\,dx = \ ?$$



Trapezoids formula:

$$I = \int_{x_0}^{x_n} f(x)\,dx \approx h\left(y_0/2 + y_1 + y_2 + \ldots + y_{n-1} + y_n/2 \right)$$

Derivation of the formula:



Suppose that $f(x)$ is well approximated on $[x_i, x_{i+1}]$ by the linear function

$$f(x) \approx y_i + (y_{i+1} - y_i)(x - x_i)/h$$

$$\int_{x_i}^{x_{i+1}} [\, y_i + (y_{i+1} - y_i)(x - x_i)/h \,]\, dx =$$

$$= y_i h + (y_{i+1} - y_i) \int (x - x_i)\, dx / h =$$

$$= y_i h + (y_{i+1} - y_i) [(x_{i+1} - x_i)^2 - (x_i - x_i)^2]/2h =$$

$$= y_i h + (y_{i+1} - y_i)\, h / 2 =$$

$$= (y_i + y_{i+1})\, h/2$$

Actually, this expression is evident, as an integral is known to be the area below the plot of function *f(x)* ; *in the case of linear function the area is halved sum of bases* × *height of trapezoid.*

$$\int_{x_0}^{x_1} f(x)\,dx \approx h\,(y_0 + y_1)/2$$

$$\int_{x_0}^{x_2} f(x)\,dx \approx h\,(y_0+y_1+y_1+y_2)/2 = h\,(y_0+2y_1+y_2)/2$$

$$\int_{x_0}^{x_3} f(x)\,dx \approx h\,(y_0+2y_1+2y_2+y_3)/2$$

Summation over all segments $[\,x_i\,,\,x_{i+1}]$, $i = 0, 1, 2, \ldots$ **gives**

$$\int_{x_0}^{x_n} f(x)\,dx \approx h\left(y_0/2 + y_1 + y_2 + \ldots + y_{n-1} + y_n/2 \right)$$

***Theorem* on the error of the trapezoid formula:**

$$\int_{x_0}^{x_n} f(x)\,dx = h\left(y_0/2 + y_1 + y_2 + \ldots + y_{n-1} + y_n/2\right) - f''(c)\,h^2\,(x_n - x_0)/12$$

↑

Error

where c is some point in the interval $x_0 < x < x_n$

(Proof is omitted).

Sometimes $f''(c)$ **can be estimated clearly:**

$$f(x) = \sin x^2 \qquad \int_0^1 \sin x^2 \, dx$$

$$f'(x) = \qquad f''(x) =$$

$$|f''(x)| \leq$$

Therefore

$$|f''(c)| \, h^2 \, (x_n - x_0) / 12 \leq$$

Example: calculate the integral

$$\int_0^1 e^{x \sin(\cos(\sin x))}\, dx$$

Way 1: write a short Scilab code

```
n=100
h=1/n
x= 0: h :1
y= exp(x.*sin(cos(sin(x))))
Int=(y(1)+y(n+1))/2
for i=2: n
Int=Int+y(i)
end
Int=h*Int
disp(Int)  ;    plot(x,y)
```

Way 2. Scilab

```
x= 0 : 0.01 : 1

y=exp(x.*sin(cos(sin(x))))
Int=inttrap(x,y)
```

Another example

$$\int_{5}^{13} \sqrt{2x-1}\ dx$$

```
clear
x=5: 0.5 : 13
y=sqrt(2*x-1)
Int=inttrap(x,y)
32.663890
```

```
x=5: 0.1 : 13
y=sqrt(2*x-1)
Int=inttrap(x,y)
32.666556
```

In fact,

$$\int_{5}^{13} \sqrt{2x-1}\ dx$$

can be expressed in an analytic form:

$$= (1/3)\,(2x-1)^{3/2}\Big|_{x=5}^{x=13} =$$

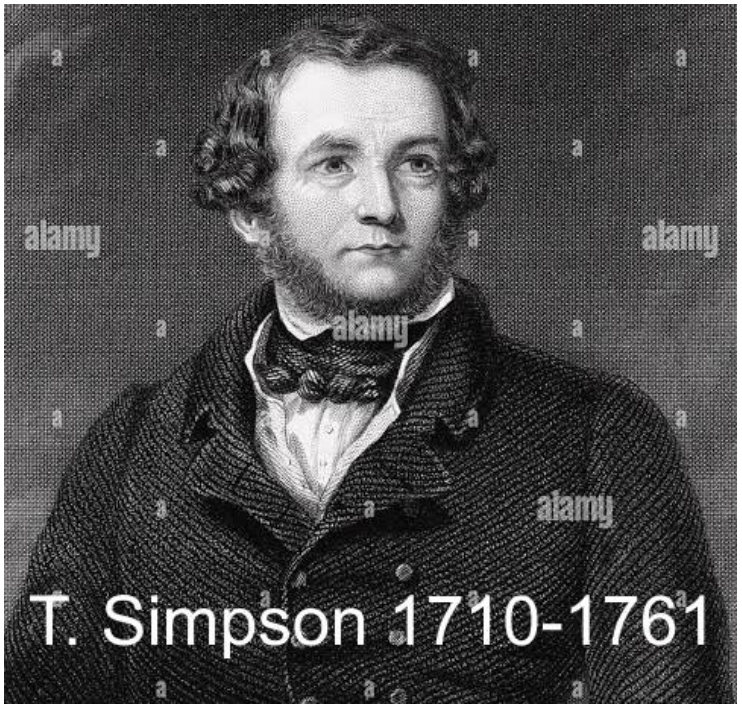
$$= (1/3)\,(26-1)^{3/2} - (1/3)\,(10-1)^{3/2}$$

$$\text{Int*} = (1/3)(25^{3/2} - 9^{3/2}) =$$

Matlab:

```
Int= trapz(x,y)
```

Chapter 4. Simpson's method for calculation of definite integrals



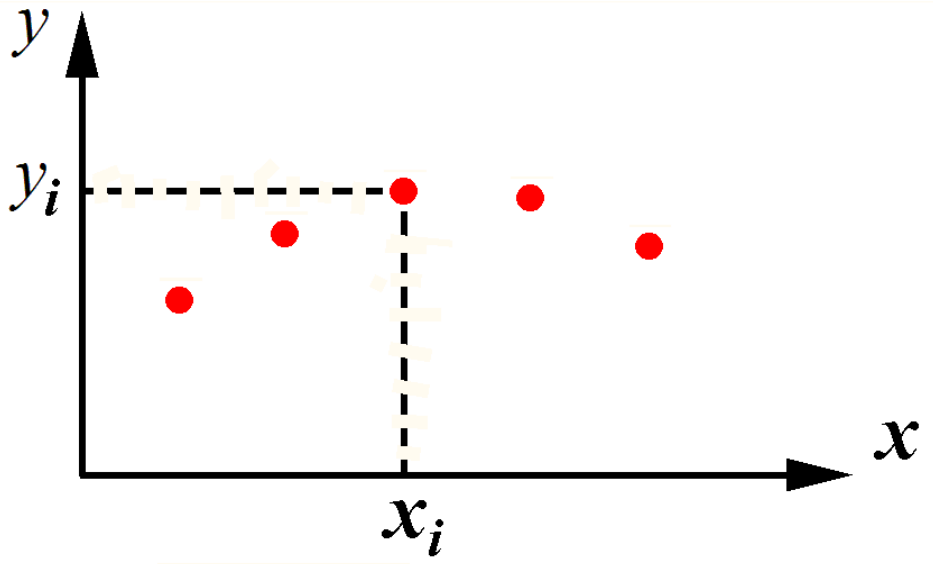
$$\int_a^b f(x)dx = ?$$

Again, we assume that values of $f(x)$ are given at a finite number of points of the segment [a,b]:

$x_0, x_1, x_2, \ldots, x_n$ $\qquad$ $y_0, y_1, y_2, \ldots, y_n$

In addition, we assume for simplicity that

$$x_{i+1} - x_i = \text{const} = h$$



$$\int_{x_0}^{x_n} f(x)\,dx = \ ?$$

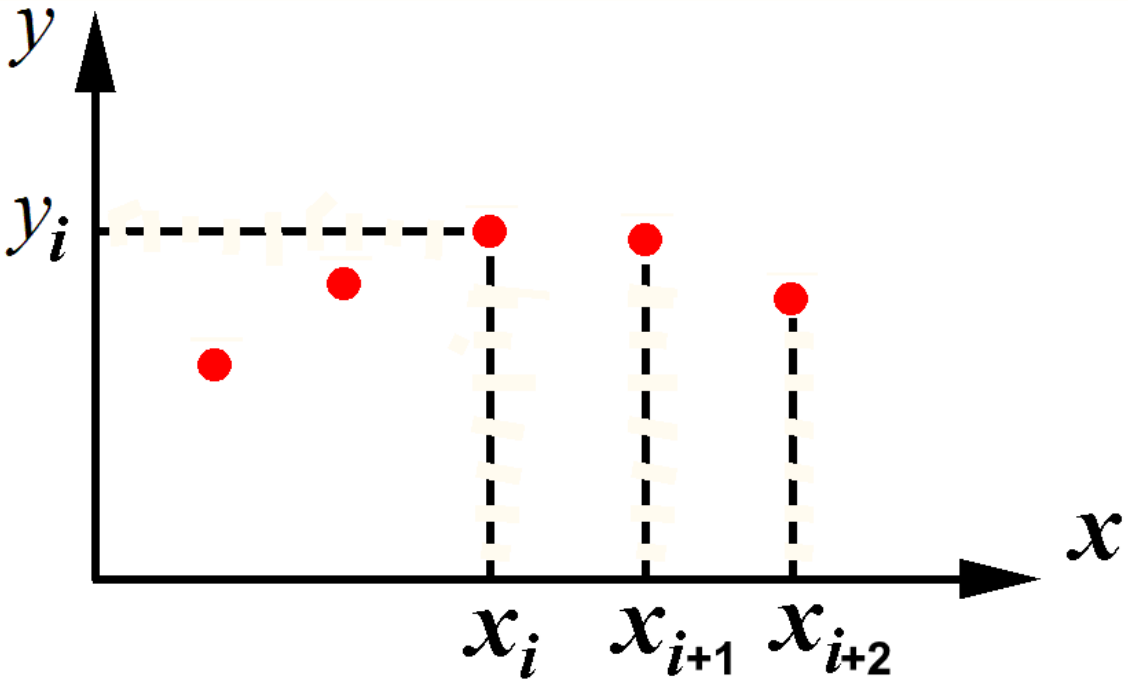
Thomas Simpson's formula:

$$\int_{x_0}^{x_n} f(x)\,dx \approx h[\,y_0 + y_n + 2(y_2 + y_4 + \ldots + y_{n-2}) + 4(y_1 + y_3 + \ldots + y_{n-1})\,] / 3$$

It provides a higher accuracy (smaller error) of integral calculation than trapezoid formula.

The way of obtaining Simpson's formula:

we approximate $f(x)$ by pieces of <u>parabolas</u> which pass through three neighboring points (x_i, y_i) (x_{i+1}, y_{i+1}) (x_{i+2}, y_{i+2}) (not by straight segments as in trapezoids method).



$$f(x) \approx y_i (x - x_{i+1})(x - x_{i+2}) / (2h^2) -$$
$$- y_{i+1} (x - x_i)(x - x_{i+2}) / h^2 +$$
$$+ y_{i+2} (x - x_i)(x - x_{i+1}) / (2h^2)$$

$$f(x) \approx y_i (x - x_{i+1})(x - x_{i+2})/(2h^2) -$$
$$- y_{i+1} (x - x_i)(x - x_{i+2})/h^2 +$$
$$+ y_{i+2} (x - x_i)(x - x_{i+1})/(2h^2)$$

Suppose $x_0 = 0$, then on segment [0, 2h]:

$$f(x) \approx y_0 (x - h)(x - 2h)/(2h^2) -$$
$$- y_1 (x - 0)(x - 2h)/h^2 +$$
$$+ y_2 (x - 0)(x - h)/(2h^2)$$

Integral of parabola can be expressed in analytical form

$$\int_{x_0}^{x_2} f(x)\,dx \approx h(y_0 + 4y_1 + y_2)/3$$

$$\int_{x_0}^{x_2} f(x)\,dx \approx h(y_0 + 4y_1 + y_2)/3$$

$$\int_{x_0}^{x_4} f(x)\,dx = \int_{x_0}^{x_2} f(x)\,dx + \int_{x_2}^{x_4} f(x)\,dx \approx$$

$$\approx h(y_0 + 4y_1 + y_2)/3 + h(y_2 + 4y_3 + y_4)/3 =$$

$$= h(y_0 + 4y_1 + 2y_2 + 4y_3 + y_4)/3$$

$$\int_{x_0}^{x_6} f(x)\,dx \approx h(y_0 + 4y_1 + 2y_2 + 4y_3 + 2y_4 + 4y_5 + y_6)/3$$

and so on. Eventually:

$$\int_{x_0}^{x_n} f(x)\,dx \approx h[\, y_0 + y_n + 2(y_2 + y_4 + \ldots + y_{n-2}) + 4(y_1 + y_3 + \ldots + y_{n-1})\,] / 3$$

Next expression shows an error of Simpson's formula

$$\int_{x_0}^{x_n} f(x)\,dx = h[\, y_0 + y_n + 2(y_2 + y_4 + \ldots + y_{n-2}) + 4(y_1 + y_3 + \ldots + y_{n-1})\,] / 3 - f^{(4)}(c)\, h^4 (x_n - x_0) / 180$$

where $x_0 < c < x_n$

Practical Runge's rule for estimation of the error:

Suppose that an integral

$$\int_a^b f(x)dx$$

is calculated using the splitting of [a, b] into n **subsegments, and denote the result by** I_n **.**
Then calculate the same integral using the splitting into $2n$ **subsegments, denote the result by** I_{2n}**.**

Runge's rule: the estimate of the error is

$$|I^* - I_{2n}| \le q\,|I_{2n} - I_n|$$

where I^* **is the exact value of integral,**

q= 1/3 in case of trapezoids method,

q= 1/15 in case of Simpson's method.

Example of calculation using Simpson's formula.

$$\int_0^1 e^{x \sin(\cos(\sin x))}\, dx$$

$$\int_{x_0}^{x_n} f(x)\, dx \approx h[\, y_0 + y_n + 2(y_2 + y_4 + \ldots + y_{n-2}) + 4(y_1 + y_3 + \ldots + y_{n-1})\,] / 3$$

Choose n=100, h=0.01:

$$\int_{x_0}^{x_{100}} f(x)\, dx \approx h[\, y_0 + y_{100} + 2(y_2 + y_4 + \ldots + y_{98}) + 4(y_1 + y_3 + \ldots + y_{99})] / 3$$

For convenience of writing the code, we renumerate the points:

$$\int_{x_1}^{x_{101}} f(x)\,dx \approx h[\, y_1 + y_{101} + 2(y_3 + y_5 + \ldots + y_{99}) + 4(y_2 + y_4 + \ldots + y_{100})\,] / 3$$

Scilab:

```
clear
x= 0:0.01:1
y= exp(x.*sin(cos(sin(x))))
h=0.01
s= y(1)+y(101)
for i=1:49
s=s+2*y(2*i+1)
s=s+4*y(2*i)
end
s=s+4*y(100)
Int=h*s/3
disp(Int)
```

Answers: 1.4569240 Simpson, h=0.01
1.4569217 – trapezoid method, h=0.01;
1.4569240 – trapezoid methods, h=0.001

Scilab built-in subroutines:

integrate and intg

```
Int=integrate('exp(x*sin(cos(sin(x))))','x',0,1)      // error< 1e-13
printf("%1.12f",Int)

Int2=integrate('exp(x*sin(cos(sin(x))))','x',0,1,1e-15)      // error< 1e-15
printf("%1.12f",Int2)

clear
function [f]=myfun(x)
f=exp(x*sin(cos(sin(x))))
endfunction
[ Int3, error] = intg(0,1, myfun )     // intg !
```

Matlab:

Int=SIMPS(x,y)

Int=quad(function, a, b);
default tol 10e-6

Int2=quad(function, a, b, tol);

Calculation of double integrals

Formulae for the evaluation of a double integral can be obtained by repeatedly applying the trapezoidal and Simpson's rules

$$I = \int_{y_j}^{y_{j+1}} \int_{x_i}^{x_{i+1}} f(x, y)\, dx\, dy,$$

where

$$x_{i+1} = x_i + h \quad \text{and} \quad y_{j+1} = y_j + k.$$

$$I = \int_{y_j}^{y_{j+1}} \left(\int_{x_i}^{x_{i+1}} f(x, y)\, dx \right) dy,$$

Let us apply the trapezoid formula with respect to x and then with respect to y :

$$I = \frac{h}{2} \int_{y_j}^{y_{j+1}} [\, f(x_i, y) + f(x_{i+1}, y)] \, dy$$

$$= \frac{hk}{4} [\, f(x_i, y_j) + f(x_{i+1}, y_j) + f(x_i, y_{j+1}) + f(x_{i+1}, y_{j+1})]$$

final trapezoid formula

Similarly, applying Simpson's formula to integral

$$I = \int_{y_{j-1}}^{y_{j+1}} \left(\int_{x_{i-1}}^{x_{i+1}} f(x, y)\, dx \right) dy,$$

we obtain

$$I = \frac{h}{3} \int_{y_{j-1}}^{y_{j+1}} [\, f(x_{i-1}, y) + 4\, f(x_i, y) + f(x_{i+1}, y)]\, dy$$

$$= \frac{hk}{9} [\, f(x_{i-1}, y_{j-1}) + 4\, f(x_{i-1}, y_j) + f(x_{i-1}, y_{j+1})$$

$$+ 4\{\, f(x_i, y_{j-1}) + 4\, f(x_i, y_j) + f(x_i, y_{j+1})\}$$

$$+ f(x_{i+1}, y_{j-1}) + 4\, f(x_{i+1}, y_j) + f(x_{i+1}, y_{j+1})]$$

A numerical example is given here.

Example 6.20 Evaluate

$$I = \int_0^1 \int_0^1 e^{x+y}\, dx\, dy,$$

using the trapezoidal and Simpson's rules. With $h = k = 0.5$, we have the following table of values of e^{x+y}.

	x		
y	0	0.5	1.0
0	1	1.6487	2.7183
0.5	1.6487	2.7183	4.4817
1.0	2.7183	4.4817	7.3891

Using the 'trapezoidal rule' from Eq. (6.94) repeatedly, we obtain

$$I = \frac{0.25}{4}[1.0 + 4(1.6487) + 6(2.7183) + 4(4.4817) + 7.3891]$$

$$= \frac{12.3050}{4}$$

$$= 3.0762.$$

Using 'Simpson's rule' given in Eq. (6.96) repeatedly, we obtain

$$I = \frac{0.25}{9}[1.0 + 2.7183 + 7.3891 + 2.7183 + 4(1.6487 + 4.4817 + 4.4817 + 1.6487) + 16(2.7183)]$$

$$= \frac{26.59042}{9}$$

$$= 2.9545.$$

The exact value I^* of the integral is $(e-1)^2 = 2.9524924\ldots$
It is seen that the error in result given by Simpson's rule is 0.002, the error in result given by trapezoid rule is 0.123, i.e., about 60 times larger.

Scilab subroutine: int2d( )

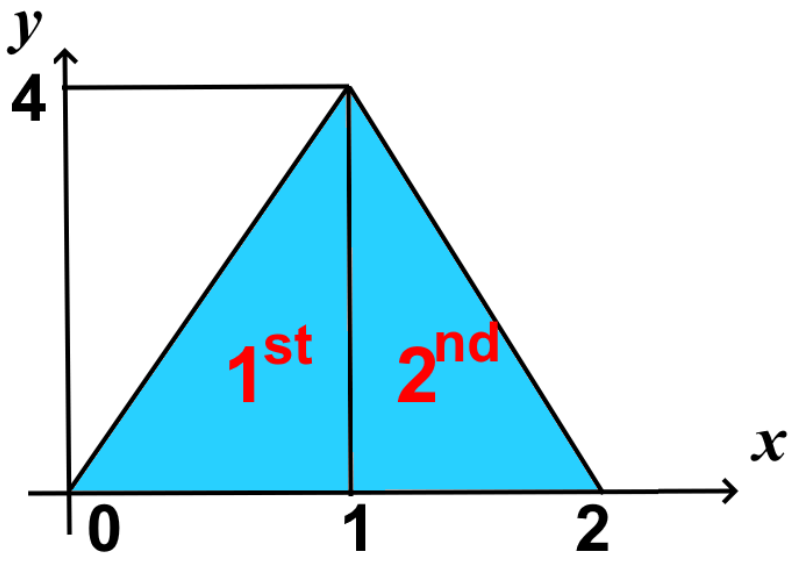
Example:

[Int, err] = int2d(X,Y,'exp(x+y)')

X,Y – coordinates of the vertices of triangles, which constitute the domain

Coordinates:

X		Y	
0	1	0	0
1	2	0	0
1	1	4	4
1st triangle	2nd triangle	1st triangle	2nd triangle



Command window:

```
X = [0  1; 1  2 ;  1  1]
Y = [0  0; 0  0 ;  4  4]
deff('z=f(x,y)', 'z=cos(x+y)')
[I,e] = int2d(X, Y, f)
```

Matlab subroutine: dblquad ( )

dblquad(fun,xmin,xmax,ymin,ymax,tol,method)

Calculates the integral over the rectangle

$$x_{min} \leq x \leq x_{max}$$

$$y_{min} \leq y \leq y_{max}$$

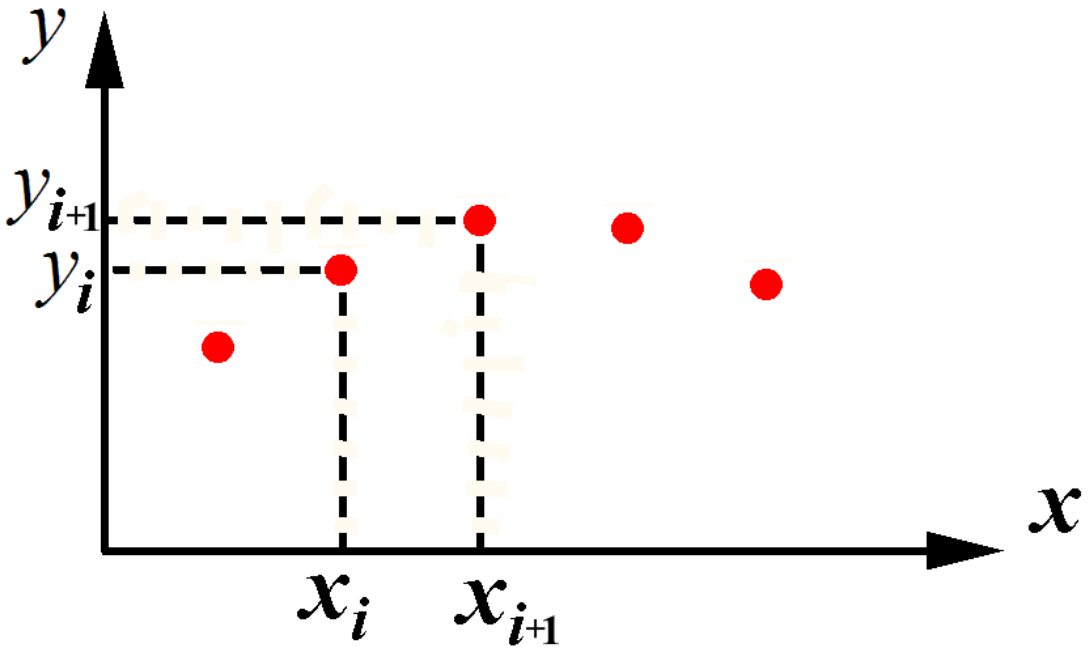
Chapter 5. Interpolation by polynomials

Suppose that values of a function $f(x)$ are given at finite number of points/nodes $x_0, x_1, x_2, \ldots, x_n$ (the function is given by table/array): $y_0, y_1, y_2, \ldots, y_n$.

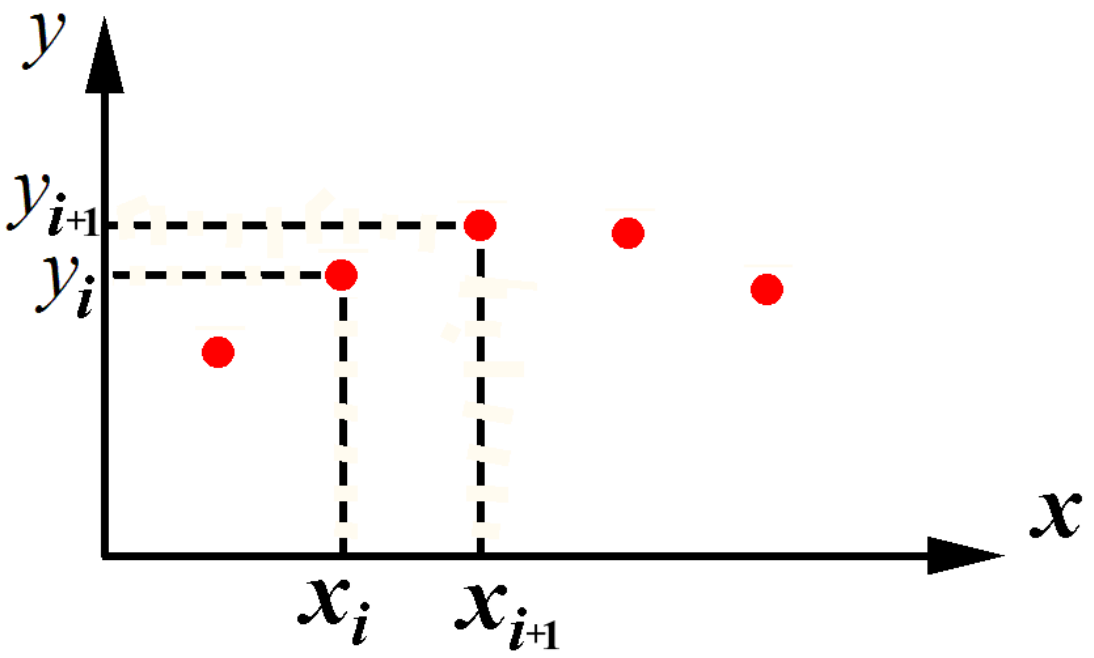
What value can be adopted as approximate value of f at a point x lying between nodes?

If $x_i < x < x_{i+1}$, then $f(x) = ?$ approximately ?

Term "interpolation" means approximation between nodes



An evident approach is to plot a polynomial $P(x)$ that passes through the given points (x_i , y_i), and then to assume that $f(x) \approx P(x)$.



(We already used a linear function $P_1(x)$ to obtain trapezoids formula, as well as a parabola $P_2(x)$ to obtain Simpson's formula).

Is it possible to plot a single polynomial that passes through all $n+1$ points (x_i , y_i) in the plane (x, y) ?

A general form of n^{th} degree polynomial is:

$$P_n(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_n x^n$$

It involves $n+1$ coefficient a_i .

We can impose the condition $P_n(x_i) = y_i$, $i=0,1, \ldots n$

$$a_0 + a_1 x_i + a_2 x_i^2 + \ldots + a_n x_i^n = y_i$$

and obtain the system of $n+1$ algebraic equations with respect to a_i . The system can be solved with methods described in Chapter 2.

Unfortunately, this way of getting $P_n(x)$ needs a great deal of computations.

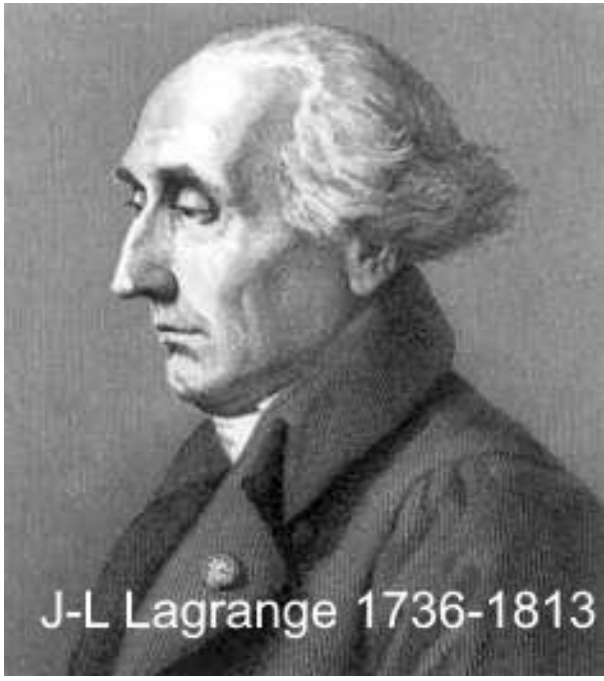
Lagrange suggested a form of the required polynomial which does not need solving any algebraic equations.

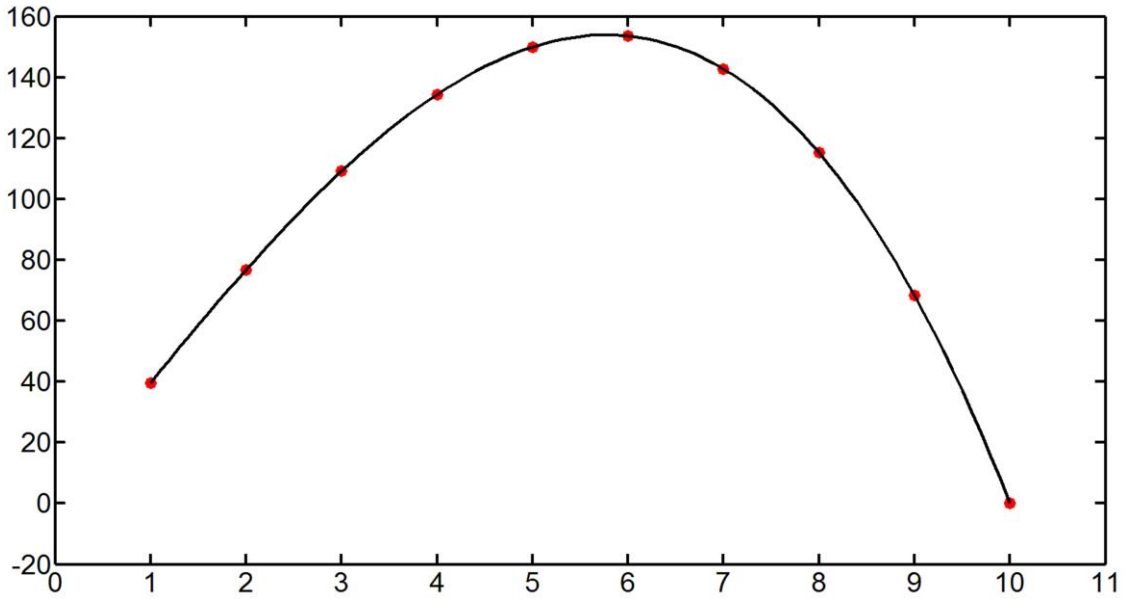
Lagrange's polynomial:

$$P(x) = y_0 \frac{(x-x_1)(x-x_2)\ldots(x-x_n)}{(x_0-x_1)(x_0-x_2)\ldots(x_0-x_n)} +$$

$$+ y_1 \frac{(x-x_0)(x-x_2)\ldots(x-x_n)}{(x_1-x_0)(x_1-x_2)\ldots(x_1-x_n)} +$$

$$\ldots + y_n \frac{(x-x_0)(x-x_2)\ldots(x-x_{n-1})}{(x_n-x_0)(x_n-x_2)\ldots(x_n-x_{n-1})}$$


J-L Lagrange 1736-1813



Regarding the error of approximation $f(x) \approx P(x)$ **at** $x_i < x < x_{i+1}$ **:**

Theorem

If there exist derivatives $f^{n+1}(x)$ of function $f(x)$ up to the order $n+1$ on segment $[x_0, x_n]$, then

$$f(x) - P(x) = f^{n+1}(c) \cdot (x-x_0)(x-x_1) \ldots (x-x_n)/(n+1)!$$

where $x_0 \le c \le x_n$, ! is the factorial

(proof is omitted)

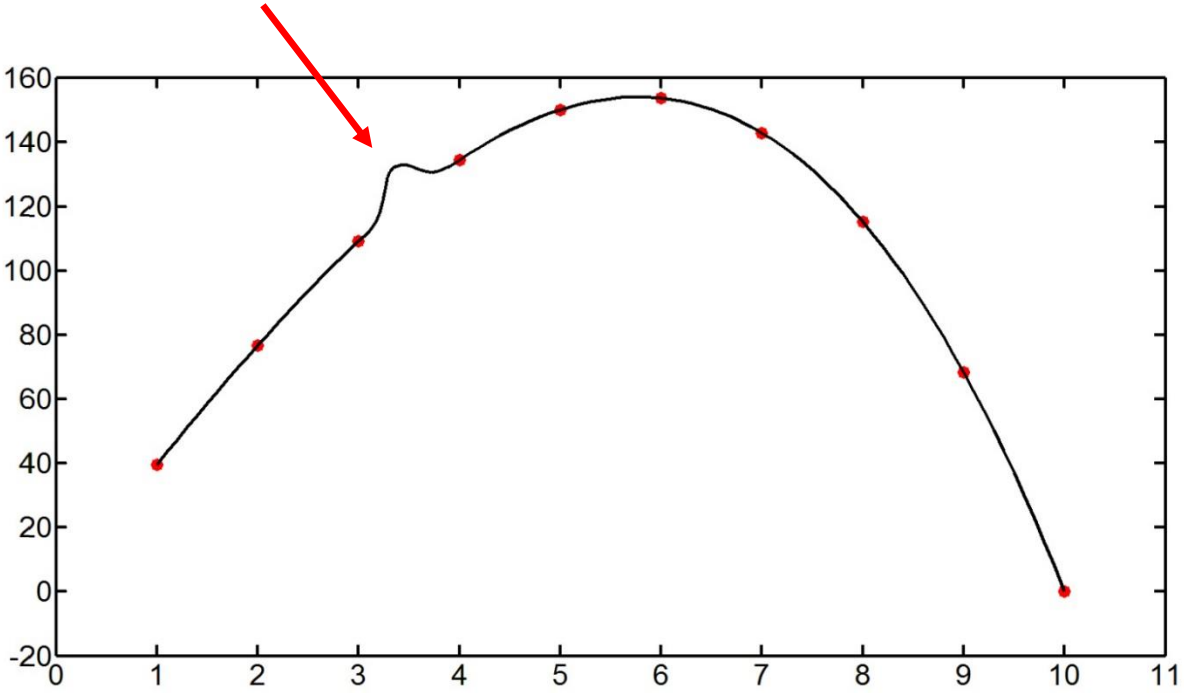
Example

Estimate the error of approximation of the function $y=\ln x$ by Lagrange's polynomial of degree 2. Use the points:

x_i	$y_i=\ln x$
2.0	0.69315
2.5	0.91629
3.0	1.09861

```
x0= 1
x1= 2
x2= 3
x3= 5
      y0= 2
      y1= 2.9
      y2= 4.2
      y3= 6
plot(x0,y0,'o',x1,y1,'o',x2,y2,'o',x3,y3,'o')        // nodal points
      for i=1:101
      x= 1+ (i-1)*4*0.01
      y=y0* (x-x1)*(x-x2)*(x-x3)   / ((x0-x1)*(x0-x2)*(x0-x3) ) +...
         y1* (x-x0)*(x-x2)*(x-x3)   / ((x1-x0)*(x1-x2)*(x1-x3) ) + ...
         y2* (x-x0)*(x-x1)*(x-x3)   / ((x2-x0)*(x2-x1)*(x2-x3) ) +...
         y3* (x-x0)*(x-x1)*(x-x2)   / ((x3-x0)*(x3-x1)*(x3-x2))
         xp(i)=x
         yp(i)=y
      end
plot(xp,yp,'k','LineWidth',3)                          // polynomial
            ypp=interpln([x0 x1 x2 x3; y0 y1 y2 y3],xp)
            plot(xp,ypp,'r','LineWidth',3)
```

Sometimes, Lagrange polynomial exhibits an unusual behavior:



This may happen when degree *n* of the polynomial is large

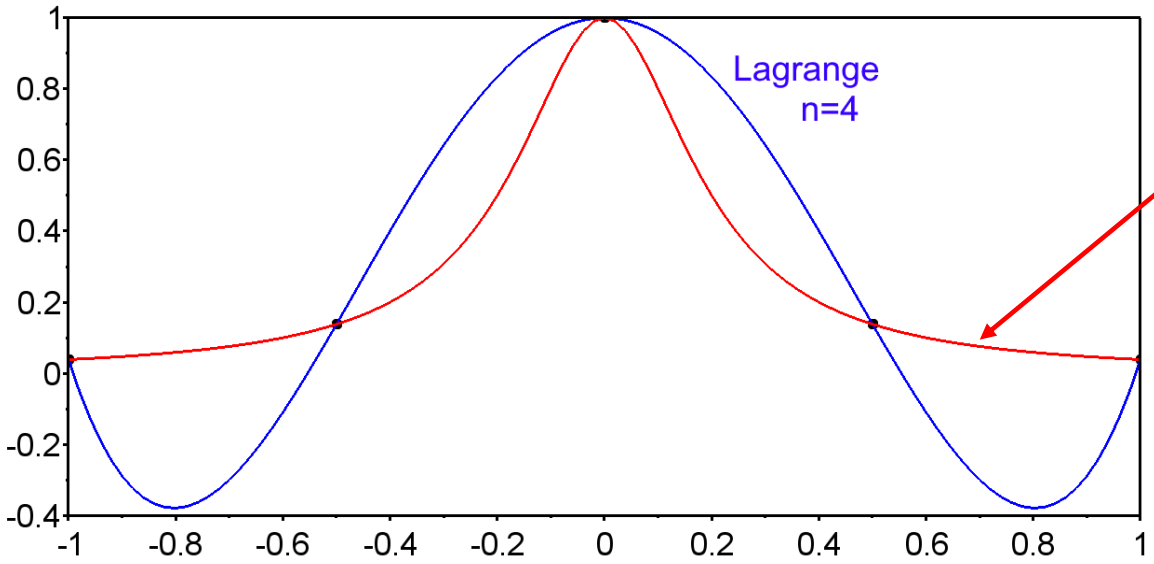
This is a specific property of polynomials.

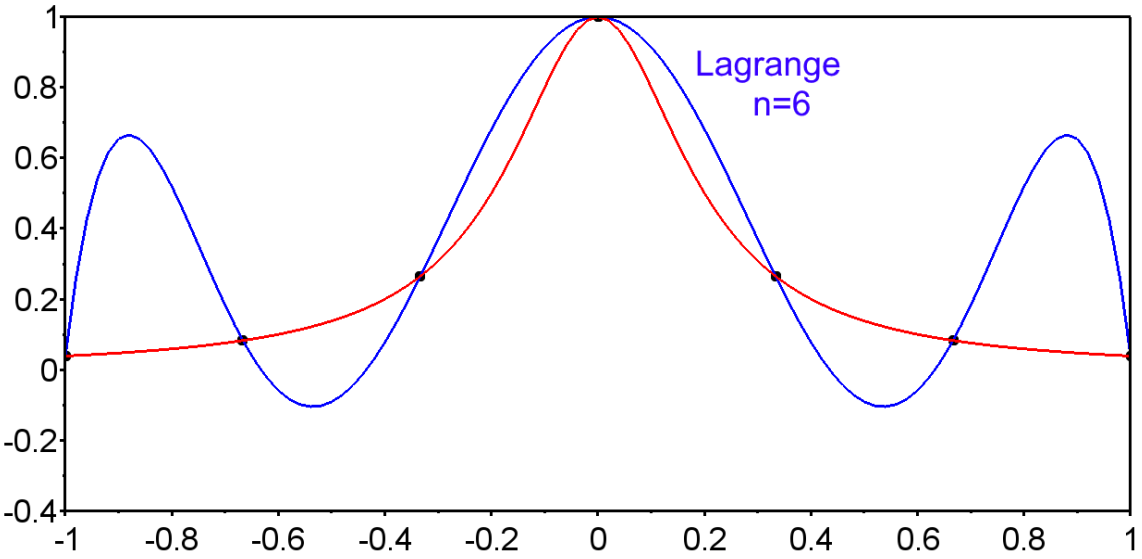
As a result, we get a bad approximation of function *f(x)*.

Even at *n*=4 and *n*=5 Lagrangian's polynomial happens to show inappropriate behavior:

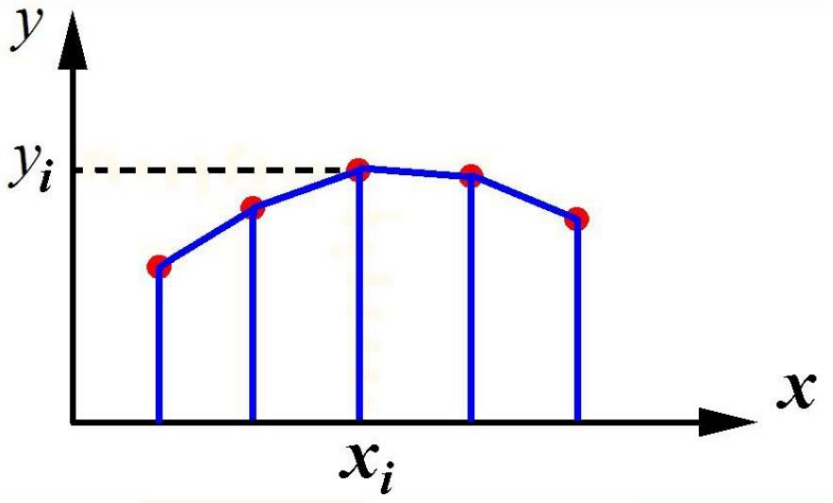
Example. Let us approximate the function $f(x)= 1/(1+25x^2)$

$-1 \le x \le 1$





Chapter 6. Interpolation by splines



recall approximation/interpolation by the linear function:

$$f(x) \approx y_i + (y_{i+1} - y_i)(x - x_i)/h$$

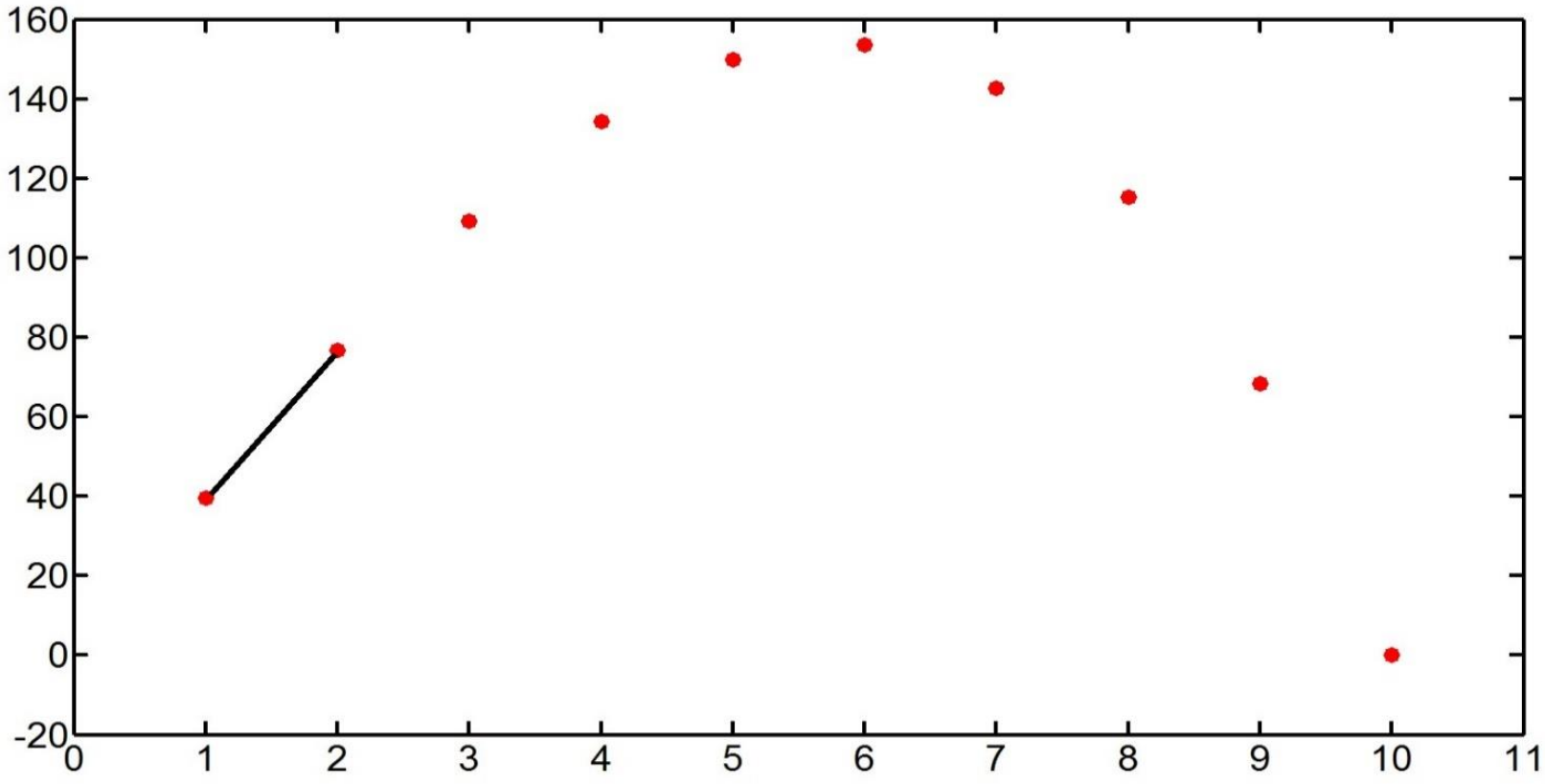
No spikes, splashes in this case. However, a drawback:

there are jumps, discontinuities of the first-order derivative

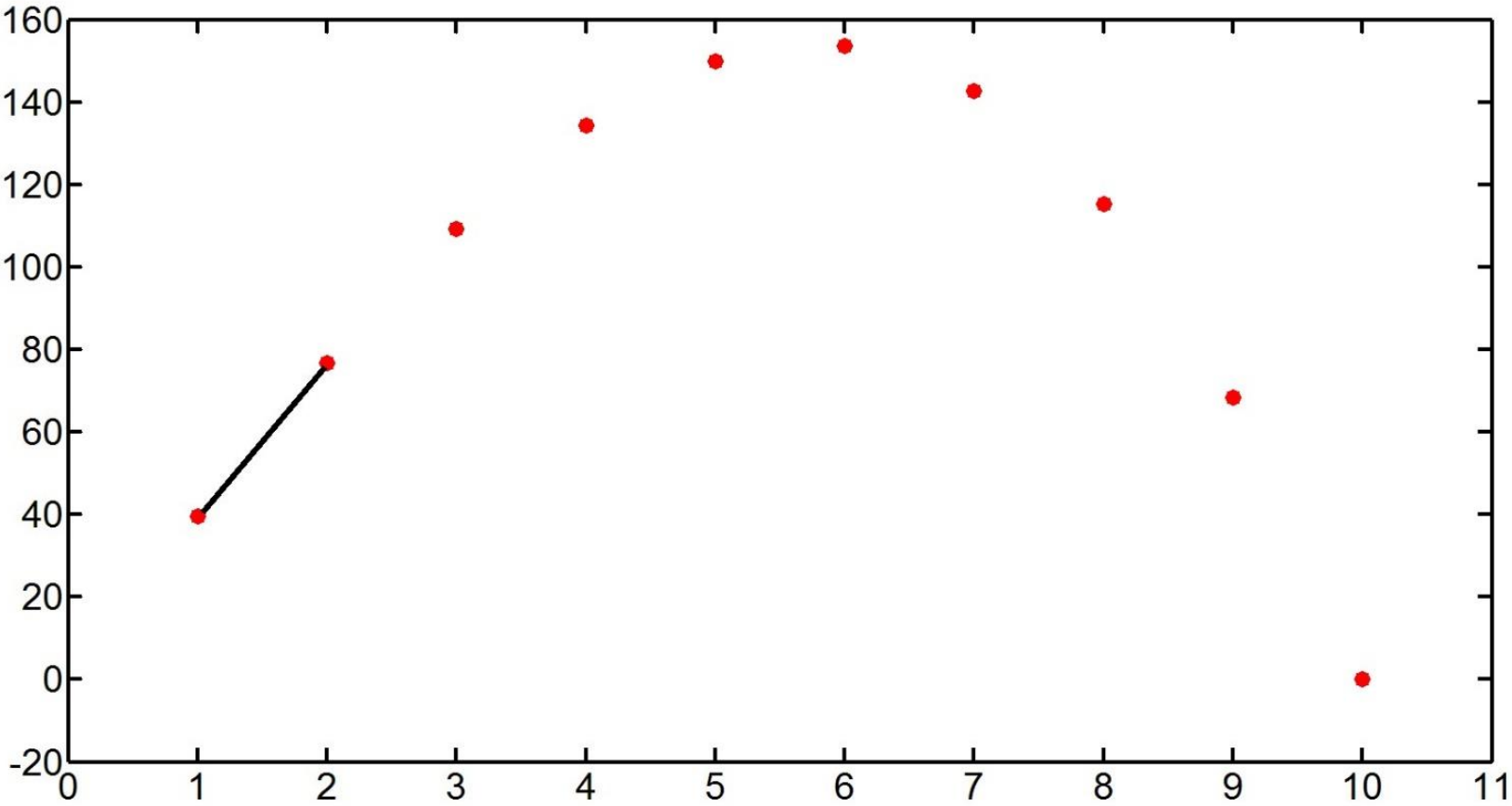
which is $(y_{i+1} - y_i)/h$ *when* $x_i < x < x_{i+1}$,

and $(y_{i+2} - y_{i+1})/h$ *when* $x_{i+1} < x < x_{i+2}$.

That is why, it was suggested to use splines for interpolation purposes



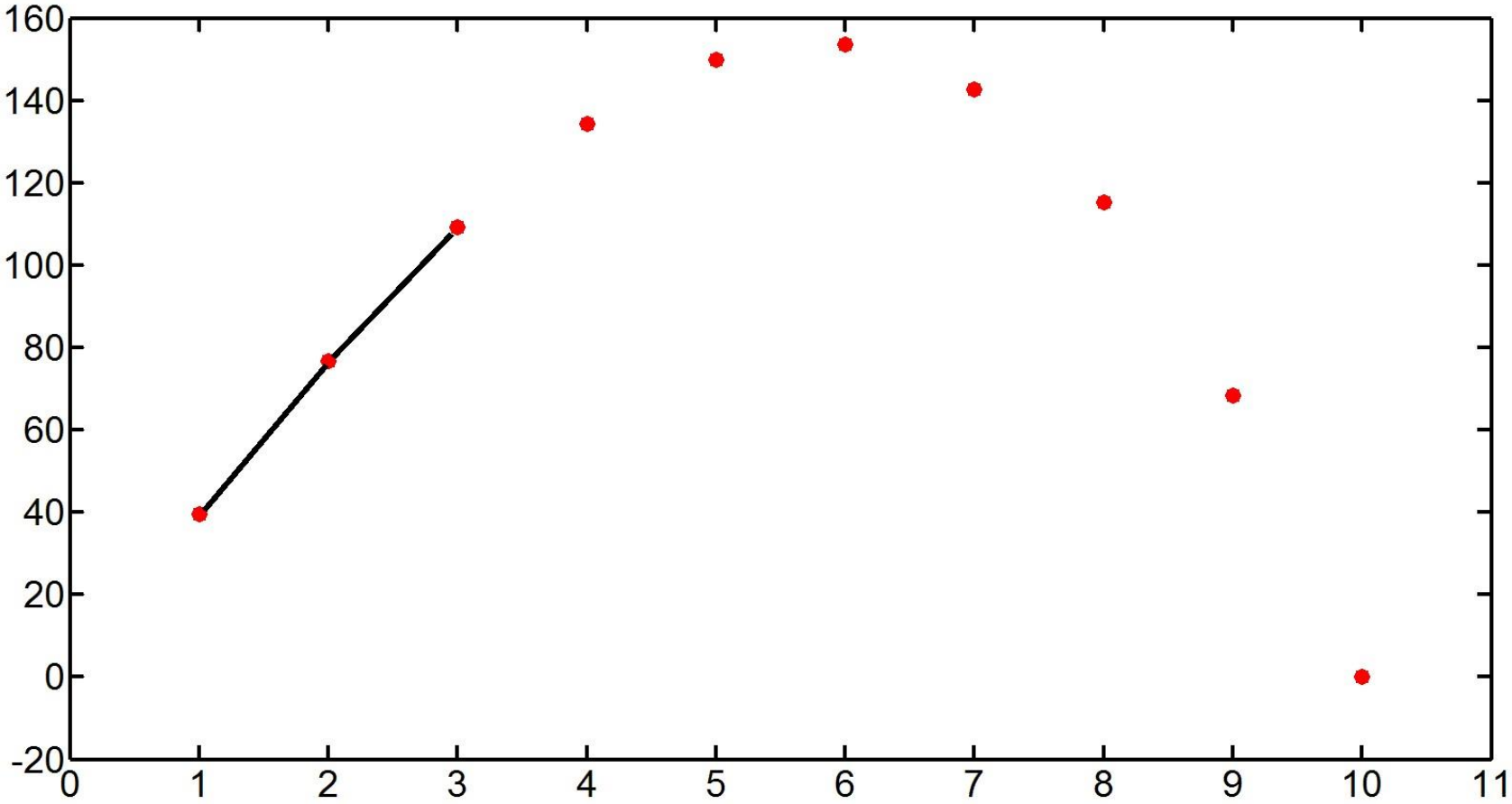
Spline $S(x)$ is a train of linked parabolas of degree 2 or 3, whose derivative dS/dx is continuous everywhere, including points x_i **(invented in 1946).**

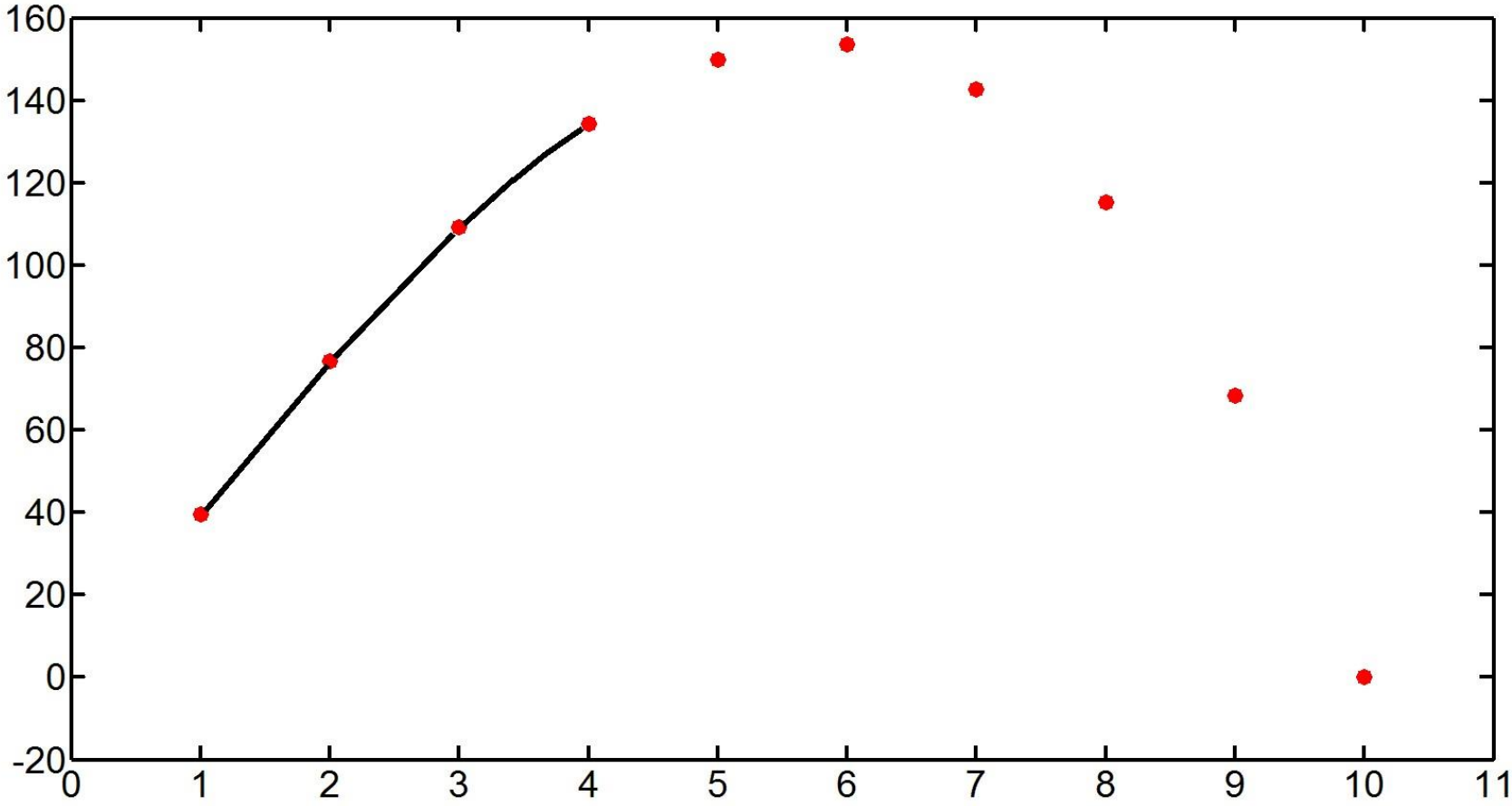


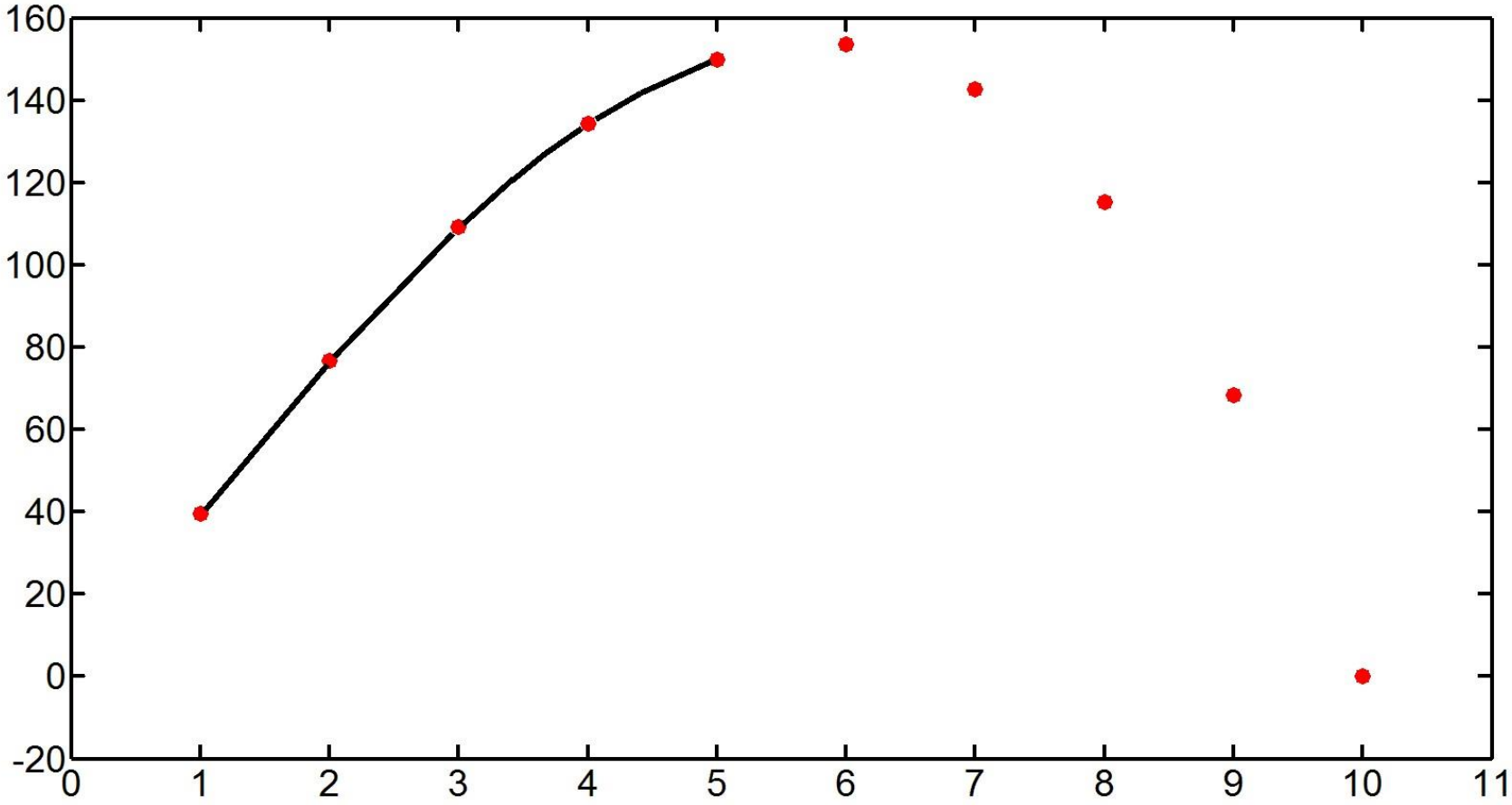
First, we consider quadratic splines

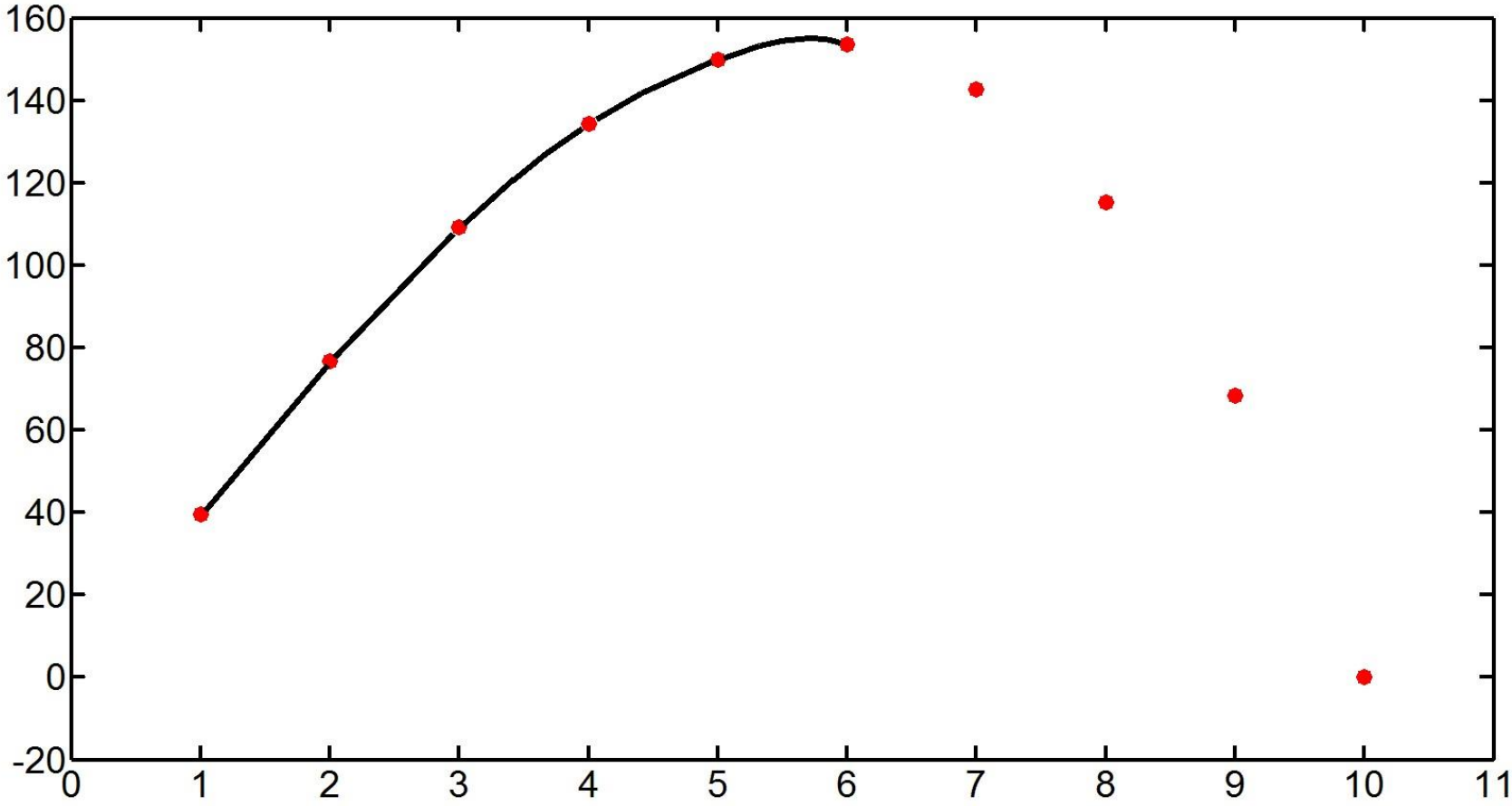
$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

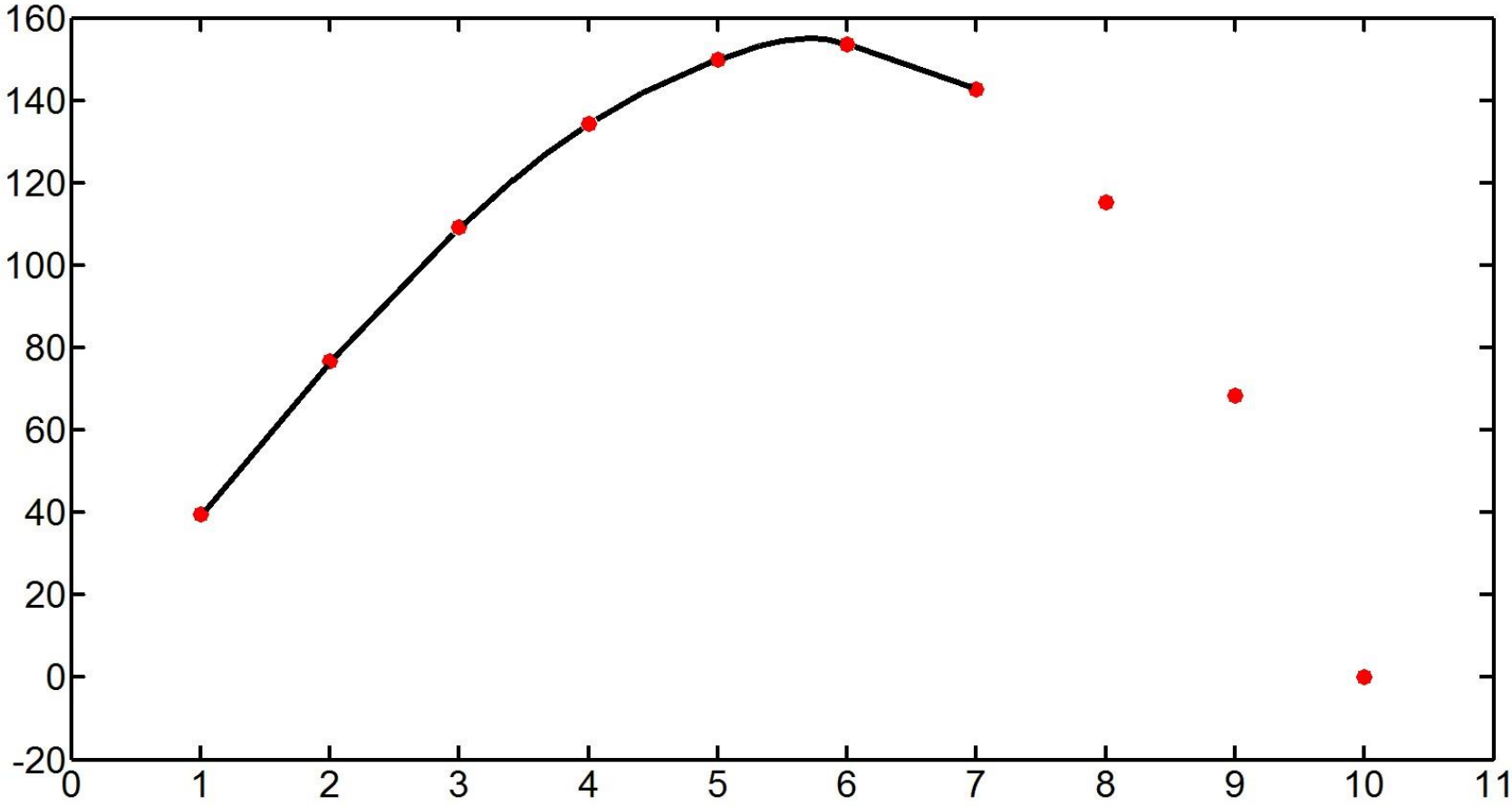
$$at \quad x_i \le x \le x_{i+1} \;, \qquad i = 0, 1, \ldots, n-1$$

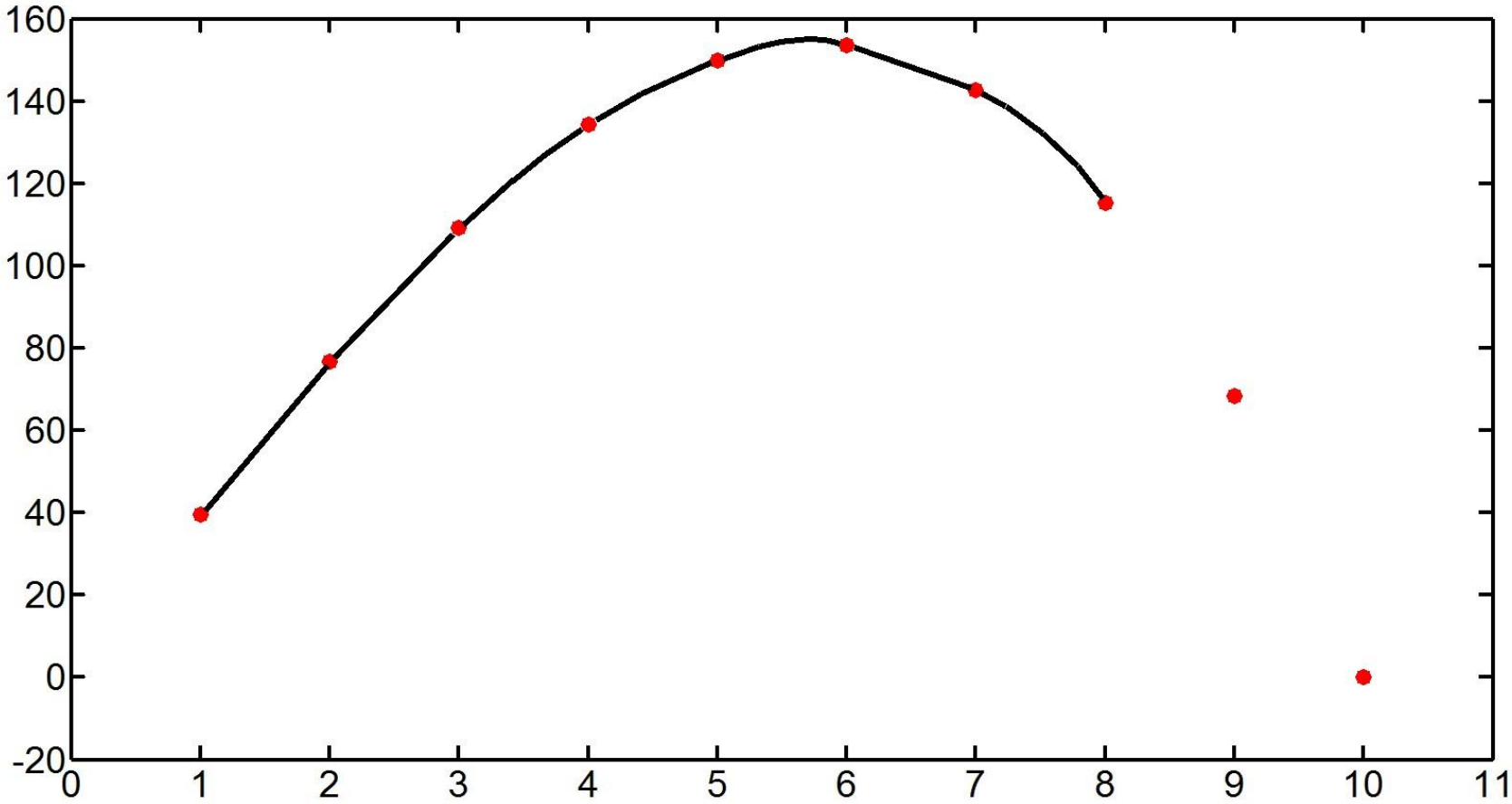

160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11

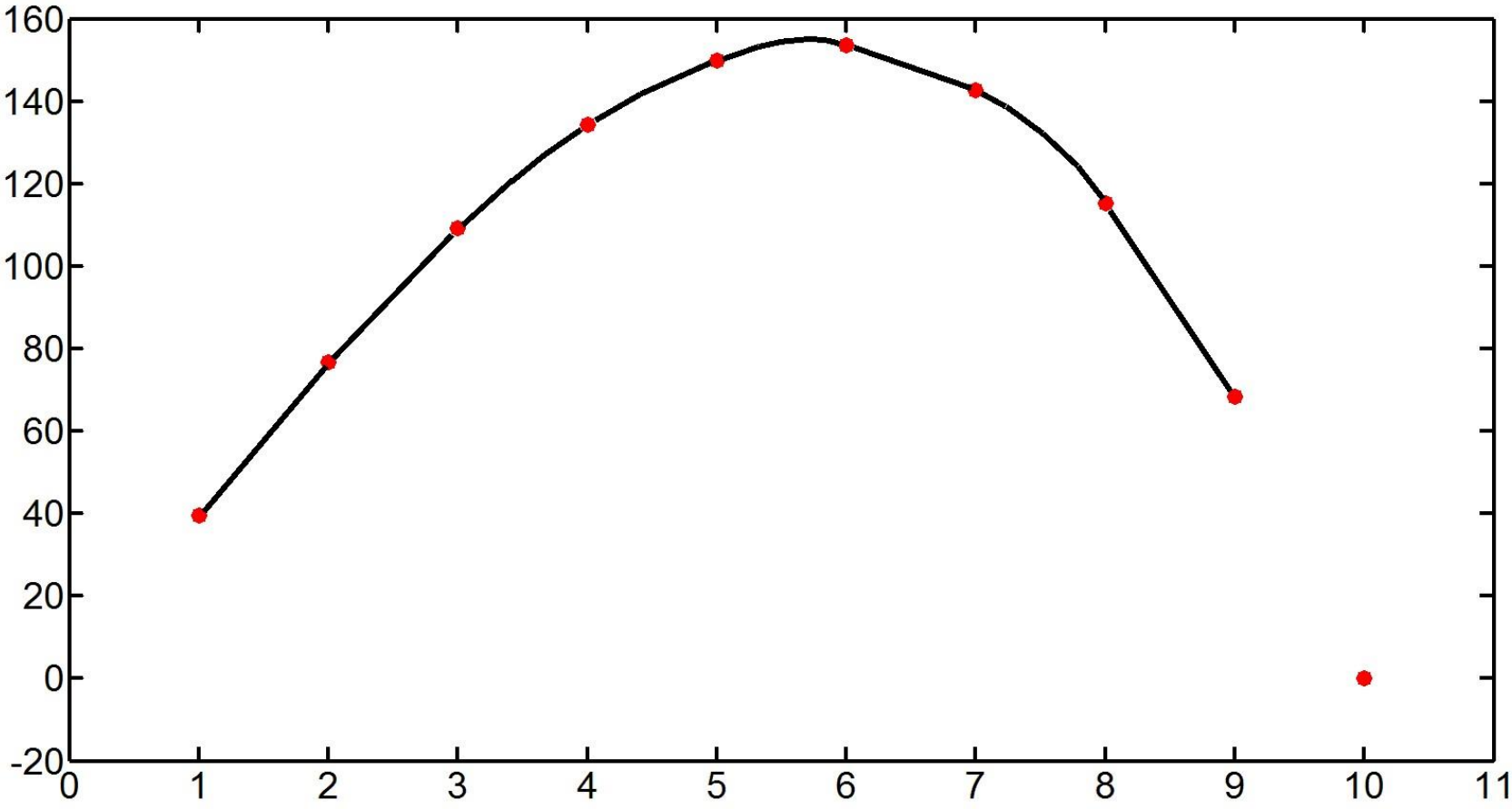

160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11

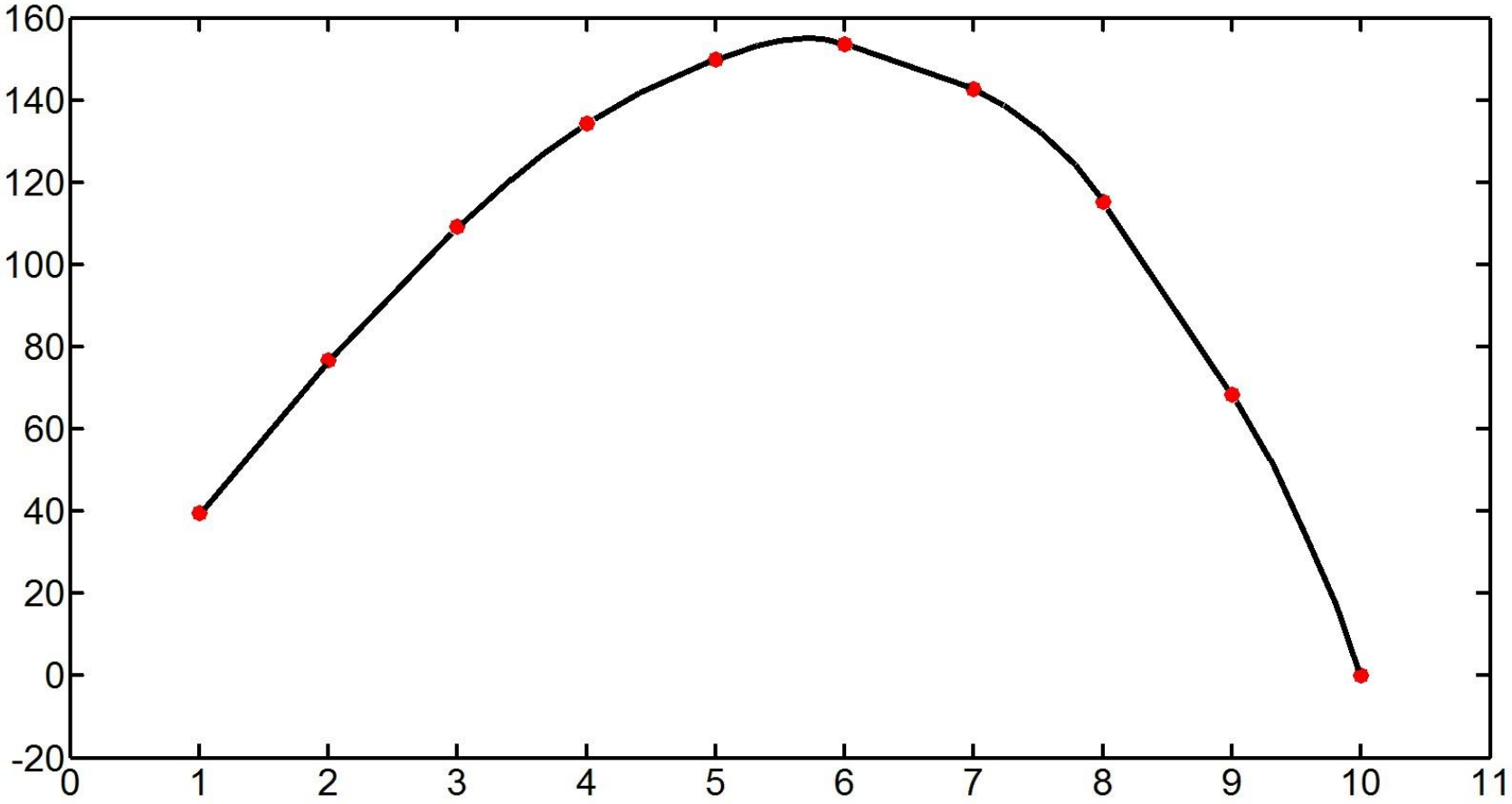

160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11


160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11


160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11


160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11


160
140
120
100
80
60
40
20
0
-20
0
1
2
3
4
5
6
7
8
9
10
11



$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

at $x_i \le x \le x_{i+1}$, $i = 0, 1, \ldots, n-1$

$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

at $x_i \le x \le x_{i+1}$, $i = 0, 1, \ldots, n-1$

Let us find coefficients a_i , b_i , c_i using 3 conditions:

1) The condition that $S(x)$ equals to given y_i at the left endpoint x_i :

$$S(x_i) = y_i \quad => \quad a_i = y_i \qquad i = 0, 1, \ldots, n-1$$

$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

at $x_i \le x \le x_{i+1}$, $i = 0, 1, \ldots, n-1$

Let us find coefficients a_i , b_i , c_i using 3 conditions:

1) The condition that $S(x)$ equals to given y_i at the left endpoint x_i :

$$S(x_i) = y_i \quad => \quad a_i = y_i \qquad i = 0, 1, \ldots, n-1$$

2)The condition that $S(x)$ equals to given value y_{i+1} at x_{i+1} :

$$S(x_{i+1}) = y_{i+1} \quad =>$$

$$=> \quad a_i + b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 = y_{i+1}$$

$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

at $x_i \le x \le x_{i+1}$, $i = 0, 1, \ldots, n-1$

Let us find coefficients a_i , b_i , c_i using 3 conditions:

1) The condition that $S(x)$ equals to given y_i at the left endpoint x_i :

$$S(x_i) = y_i \quad => \quad a_i = y_i \qquad i = 0, 1, \ldots, n-1$$

2)The condition that $S(x)$ equals to given value y_{i+1} at x_{i+1} :

$$S(x_{i+1}) = y_{i+1} \quad =>$$

$$=> \quad a_i + b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 = y_{i+1}$$

3) The condition that dS/dx is continuous at node x_{i+1} :

$$dS/dx = b_i + 2\,c_i \cdot (x - x_i) \qquad at \;\; x_i \le x \le x_{i+1}$$

$$b_i + 2\,c_i \cdot (x_{i+1} - x_i) = b_{i+1} + 2\,c_{i+1} \cdot (x_{i+1} - x_{i+1})$$

$$\begin{cases} b_i + 2\,c_i \cdot (x_{i+1} - x_i) = b_{i+1} & (1) \qquad i = 0, 1, \ldots, n-2 \\ b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 = y_{i+1} - y_i & (2) \end{cases}$$

see condition 2) above

$$i = 0, 1, \ldots, n-1$$

We get 2n-1 equations for 2n coefficients b_i, c_i

+ additional condition $c_0 = 0$

($S(x) = a_0 + b_0 \cdot (x - x_0) \qquad$ at $\; x_0 \le x \le x_1$)

In fact, coefficients b_i, c_i can be calculated <u>sequentially</u> for $i=1,2,\ldots,n$:

b_1 , c_1 ,

then b_2 *from (1),*

then c_2 *from (2),*

then b_3 *from (1), . . .*

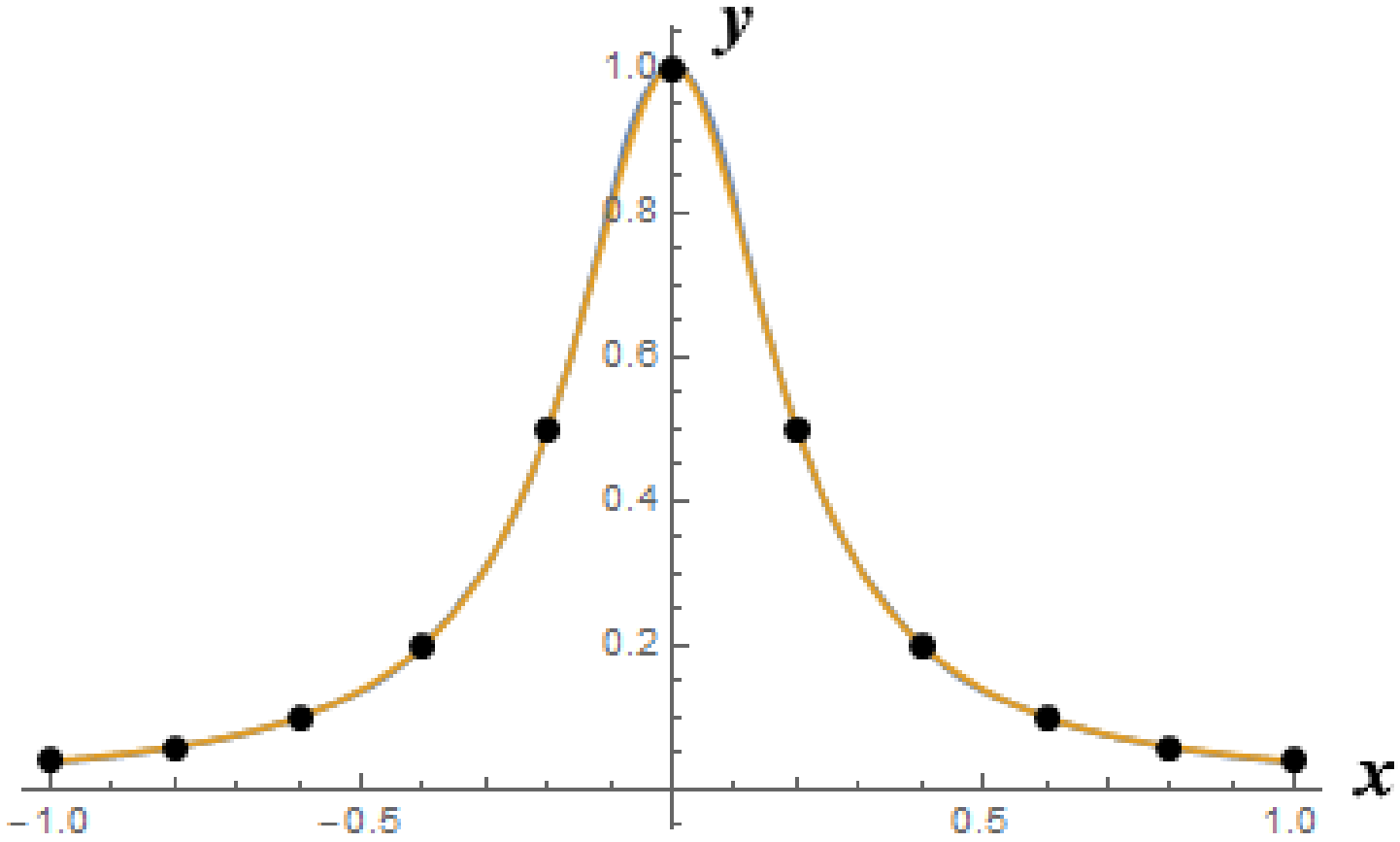
Theorem on the error of interpolation:

If $f'''(x)$ is continuous on $[x_0, x_n]$, then

$$|f(x)-S(x)| \le \max |f'''(x)| h^3/12,$$

where $h=\max |x_{i+1}-x_i|$.

Proof see in:
"Optimal Error Bounds for Quadratic Spline Interpolation"
Francois Dubeau
JOURNAL OF MATHEMATICAL ANALYSIS AND APPLICATIONS,
Vol. 198, pages 49-63, 1996



Example: Interpolation of the function $f(x)= 1/(1+25x^2)$ by a quadratic spline looks perfect (the curves $S(x)$ and $f(x)$ coincide) https://engcourses-uofa.ca/books/numericalanalysis in contrast to interpolation by a Lagrangian's polynomial.

We notice that the second-order derivative of

$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

is $\frac{d^2 S}{dx^2} = 2 c_i$

Therefore, it is constant at each interval

$$x_i \leq x \leq x_{i+1}$$

and usually jumps when x **moves from** $x_i \leq x \leq x_{i+1}$ **to next segment, since** c_i **are usually different in different segments.**

Using cubic splines

$$S_{cub}(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2 + d_i \cdot (x - x_i)^3$$

$$x_i \le x \le x_{i+1}\ , \qquad i = 0, 1, \ldots, n-1,$$

with coefficient d_i ***, we can provide the continuity of*** d^2S_{cub}/dx^2 ***at the nodes*** x_i **.**

Indeed, the derivative

$$d^2S_{cub}/dx^2 = 2c_i + 6d_i \cdot (x - x_i)$$

changes in $x_i \le x \le x_{i+1}$

and therefore its values at the ends of interval can be adjusted in a proper way.

We set 4 conditions:

1) $S_{cub}(x)$ is equal to y_i at left end of segment

$x_i \le x \le x_{i+1}$: $y_i = a_i$

2) $S_{cub}(x)$ is equal to y_{i+1} at right end of the segment

$$y_{i+1} = a_i + b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 + d_i \cdot (x_{i+1} - x_i)^3$$

3) **First-order derivative:**

$$dS_{cub}/dx = b_i + 2c_i \cdot (x - x_i) + 3d_i \cdot (x - x_i)^2$$

the condition of equal derivatives at the right endpoint:

$$b_i + 2c_i \cdot (x_{i+1} - x_i) + 3d_i \cdot (x_{i+1} - x_i)^2 = b_{i+1}$$

$i = 1, 2, \ldots, n-1$

4) Second-order derivative :

$$d^2S_{cub}/dx^2 = 2c_i + 6d_i \cdot (x-x_i)$$

the condition of equal second derivatives at the right endpoint:

$$2c_i + 6d_i \cdot (x_{i+1}-x_i) = 2c_{i+1} \qquad i=1,2,\ldots,n-1$$

We have got 2n+2(n-1) equations with respect to 4n coefficients of spline S_{cub} .

We add two conditions $d^2S_{cub}/dx^2 = 0$ at endpoints x_0, x_n :

$$c_0 = 0, \qquad 2c_{n-1} + 6d_{n-1} \cdot (x_n - x_{n-1}) = 0$$

Finally, we have 4n equations with respect to 4n coefficients.

```
x0= 1
x1= 2
x2= 3
x3= 5
      y0= 2
      y1= 2.9
      y2= 4.2
      y3= 6
plot(x0,y0,'o',x1,y1,'o',x2,y2,'o',x3,y3,'o')

d=splin( [x0 x1 x2 x3], [y0 y1 y2 y3] )
x= 1: 0.02 : 5
y=interp(x, [x0 x1 x2 x3], [y0 y1 y2 y3], d)
plot(x,y,'b','LineWidth',3)
```

Interpolation of functions of two independent variables

DOUBLE INTERPOLATION

In the preceding sections we have derived interpolation formulae to approximate a function of a single variable. For a function of two or more variables, the formulae become complicated but a simpler procedure is to interpolate with respect to the first variable keeping the others constant, then interpolate with respect to the second variable, and so on. The method is illustrated below for a function of two variables.

The following table gives the values of z for different values of x and y. Find z when $x = 2.5$ and $y = 1.5$.

	x				
y	0	1	2	3	4
0	0	1	4	9	16
1	2	3	6	11	18
2	6	7	10	15	22
3	12	13	16	21	28
4	18	19	22	27	34

We first interpolate with respect to x keeping y constant. For $x = 2.5$, we obtain the following table using *linear interpolation*.

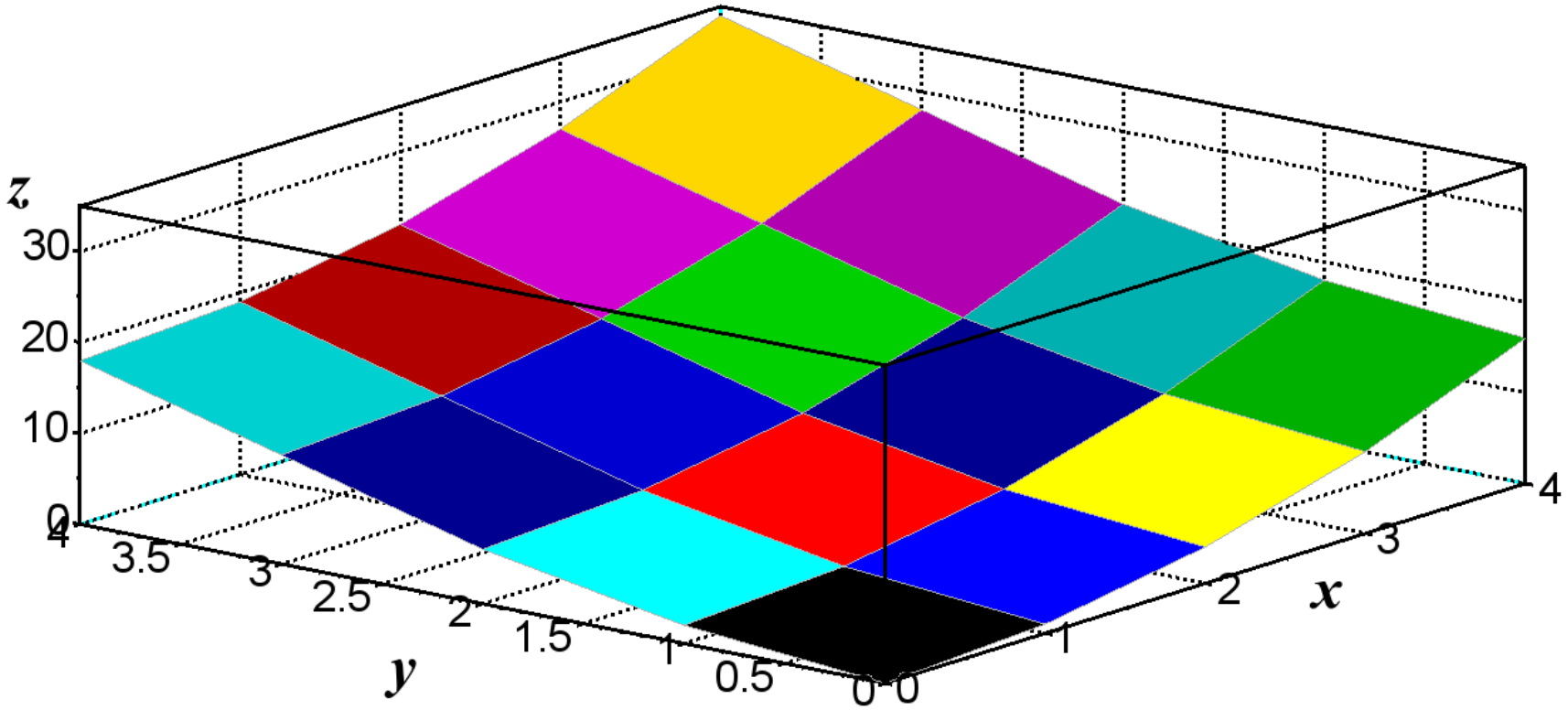
y	z
0	6.5
1	8.5
2	12.5
3	18.5
4	24.5

y	z
0	6.5
1	8.5
2	12.5
3	18.5
4	24.5

Now, we interpolate with respect to y using linear interpolation once again. For $y = 1/5$, we obtain

$$z = \frac{8.5 + 12.5}{2} = 10.5$$

so that $z(2.5, 1.5) = 10.5$. Actually, the tabulated function is $z = x^2 + y^2 + y$ and hence $z(2.5, 1.5) = 10.0$, so that the computed value has an error of 5%.



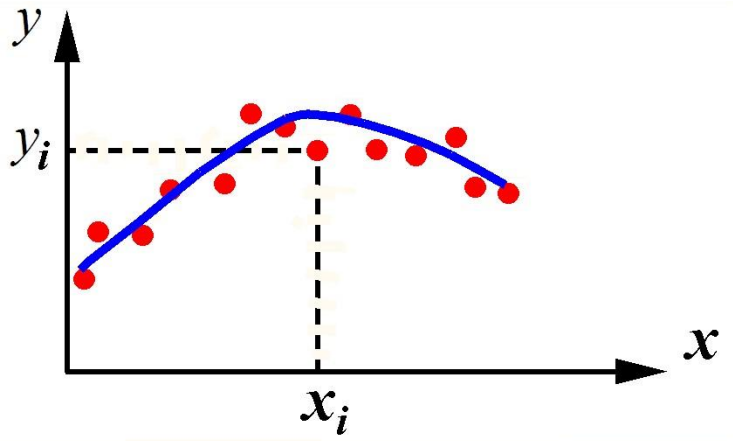
Scilab:

splin2d( )

interp2d( )

Chapter 7. Least-Squares approximation

(Least-Squares fitting procedure)



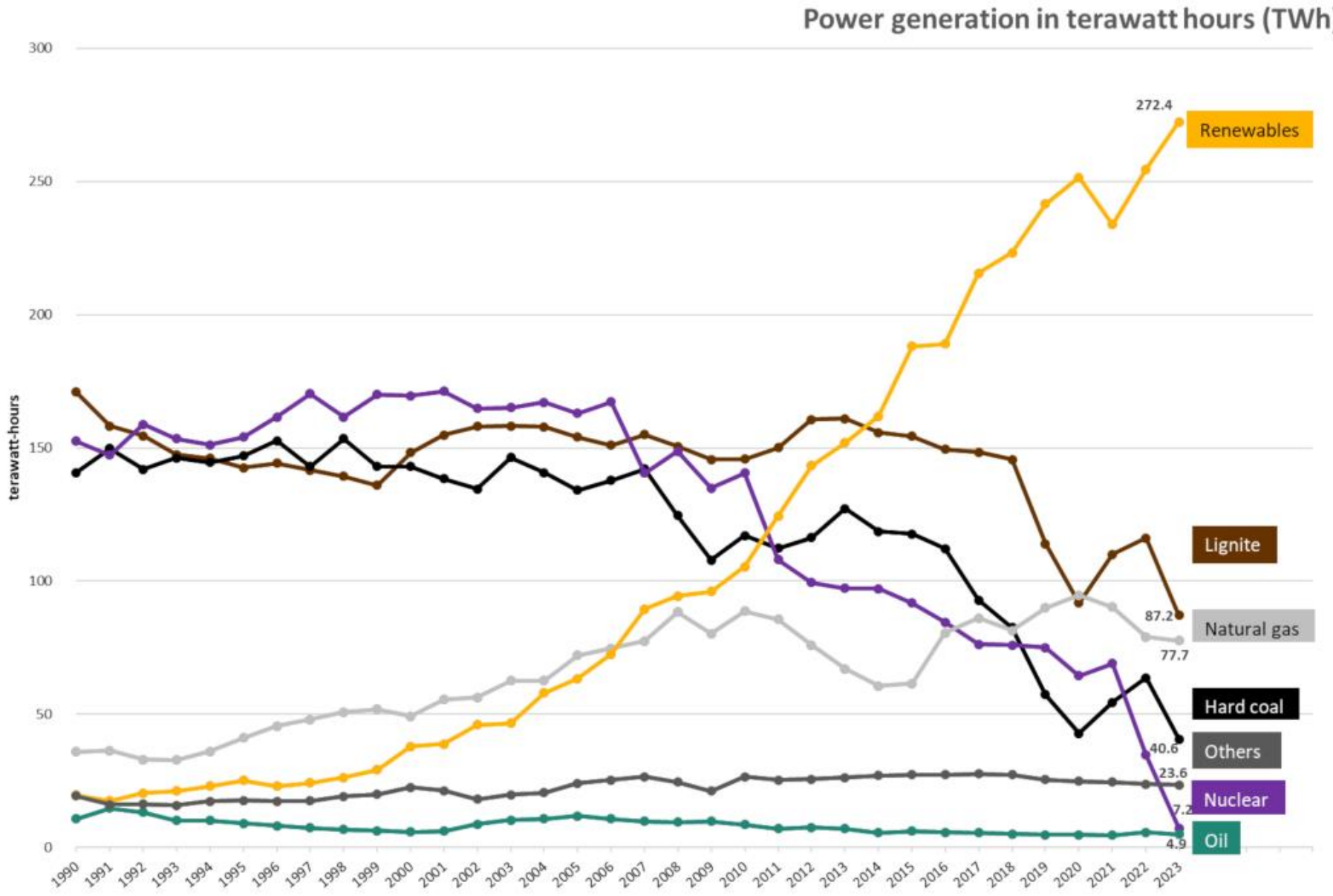
Suppose that given values x_i , y_i show fluctuations, deviations, from some smooth curve (for example, because of influence of random factors).

<u>Problem</u>: Find a function $p(x)$ that is smooth and well approximates x_i , y_i .

Gross power production in Germany 1990 - 2023, by source.

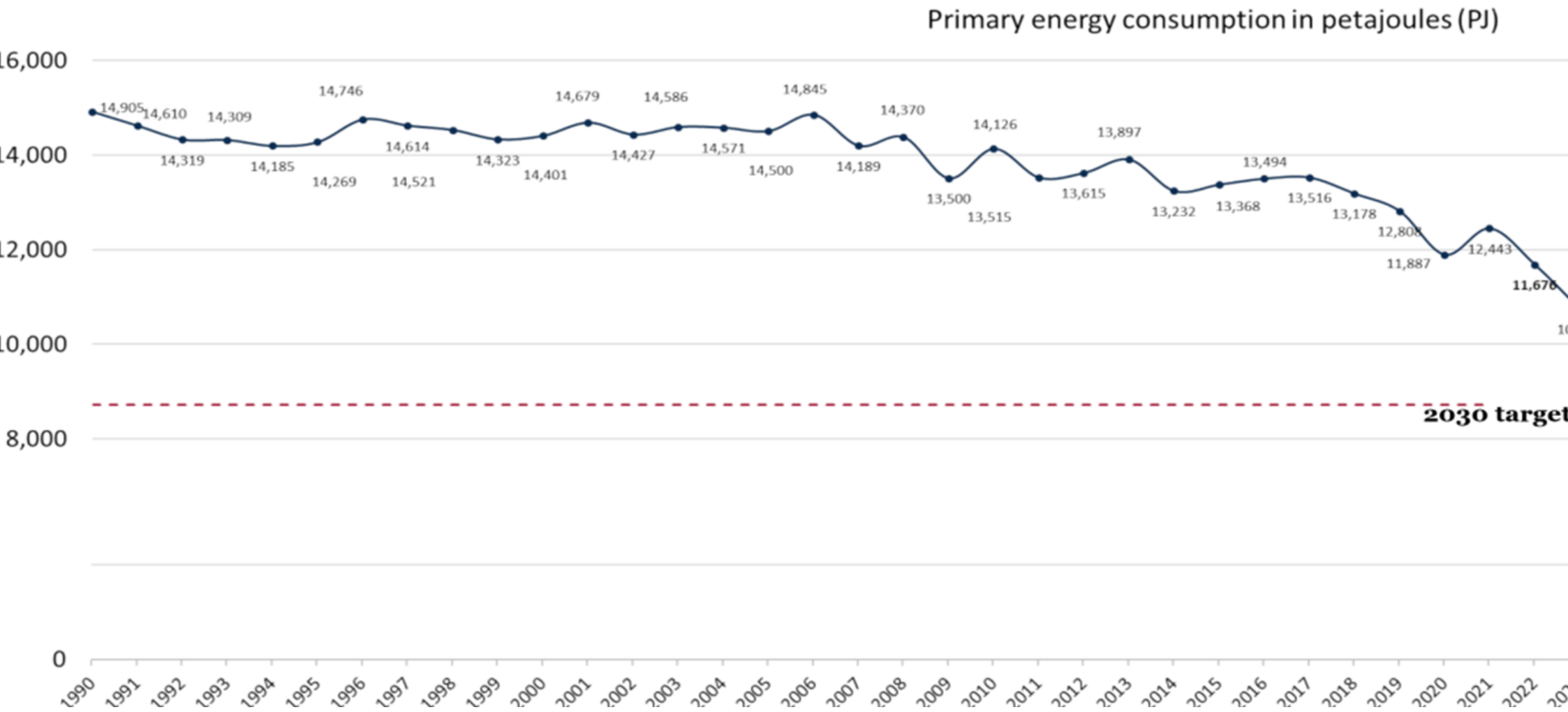
Data: AGEB 2024.

CLEAN ENERGY WIRE



Development of Germany's primary energy consumption 1990 - 2023.

Data: AG Energiebilanzen 2024.



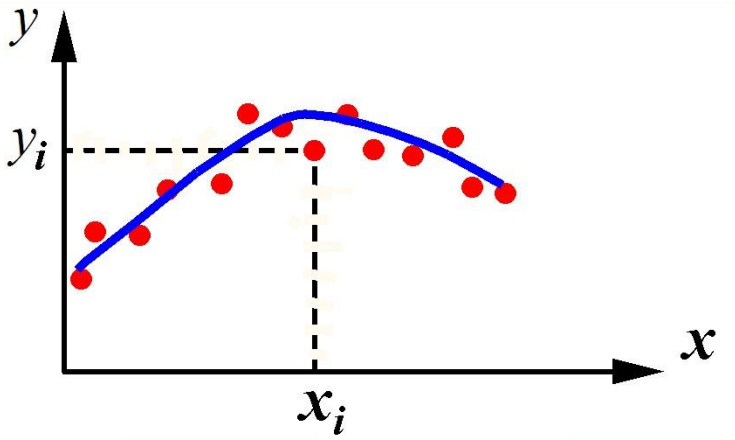
Primary energy is the energy found in nature that has not been subjected to any human engineered conversion process. wiki

Consider a polynomial

$p(x)=a_0 + a_1x + a_2x^2 + \ldots + a_mx^m, \quad m<n$

We can use either condition

$$\sum_{i=0}^{n} | p(x_i) - y_i | \rightarrow \min$$



or

$$\sum_{i=0}^{n} [p(x_i) - y_i]^2 / (n+1) \rightarrow \min$$

(title of method)

$$\sum_{i=0}^{n} [\, p(x_i) - y_i \,]^2 \quad \rightarrow \quad \min$$

$$\sum_{i=0}^{n} [a_0 + a_1 x_i + \ldots + a_k x_i^k + \ldots + a_m x_i^m - y_i \,]^2 \rightarrow \min$$

The sum involves n+1 square brackets

This is the condition for finding a_i

Let us denote the left-hand side by

$$J(a_0, a_1, \ldots, a_k, \ldots, a_m) = \sum_{i=0}^{n} [\, p(x_i) - y_i \,]^2$$

This is a function of $m+1$ variables a_j

Necessary condition of a minimum: $\partial J/\partial a_k = 0$

$$\partial J/\partial a_k = \sum_{i=0}^{n} 2[\, p(x_i) - y_i] \cdot \partial p(x_i)/\partial a_k = 0$$

$$\sum_{i=0}^{n} [\, p(x_i) - y_i] \cdot x_i^k = 0 \qquad k=0,1,\ldots m$$

$$\sum_{i=0}^{n} [a_0 + a_1 x_i + \ldots + a_m x_i^m] \cdot x_i^k = \sum y_i x_i^k$$

$$\sum_{i=0}^{n} [a_0 x_i^k + a_1 x_i^{k+1} + \ldots + a_m x_i^{k+m}] = \sum y_i x_i^k$$

and we get the system of $m+1$ **linear equations,** $k=0,1,\ldots m$**:**

$$a_0 \sum_{i=0}^{n} x_i^k + a_1 \sum_{i=0}^{n} x_i^{k+1} + \ldots + a_m \sum_{i=0}^{n} x_i^{k+m} = \sum_{i=0}^{n} y_i x_i^k \qquad (*)$$

Theorem If $m \le n$, then there exists a unique solution of the system of equations (*) with respect to coefficients a_k of the polynomial $p(x)$ (proof is omitted).

In particular, at m=n this polynomial is equivalent to Lagrange's polynomial (which passes through each node).

We used the necessary condition of minimum

$\partial J/\partial a_k = 0$.

The fact that obtained a_k **determine indeed a minimum, not maximum, is evident from the quadratic dependence of**

$$J(a_0 , a_1, + \ldots + a_m) = \sum_{i=0}^{n} [\, p(x_i) - y_i \,]^2 \quad \text{on} \quad a_k ,$$

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_m x^m$$

Scilab

[a,err]=datafit(J, xy, a0)

xy - matrix of given x_i , y_i

J - function that defines the approximating polynomial and the difference between its values and y_i

a - obtained coefficients of the approximating polynomial

a0 – initial approximation

err - obtained sum of squared differences

Scilab

```
function [fun]=J(a, xy)
fun=xy(2)-a(1)-a(2)*xy(1)-a(3)*xy(1)^2-a(4)*xy(1)^3
endfunction
x=[1.3   1.4   1.5     1.6   1.7     1.8];
y=[3.3   3.4   3.85   4.25   4.50    4.85];
xy=[x;y];
plot(x,y,'o','LineWidth',3);
//  initial approximation:
a0=[0;0;0;0]
// Solution:
[a,err]=datafit(J,xy,a0)
t=1.3:0.01:1.8;
poly=a(1)+a(2)*t+a(3)*t.^2+a(4)*t.^3;
plot(t,poly,'r');
xgrid
disp(t(41), poly(41))
```

Notice. When you choose the degree m , you should take into consideration the physics of the dependence $y_i(x_i)$ and the number of bends you want in fitted line.

Each increase in the degree m produces one more bend in the fitted line.

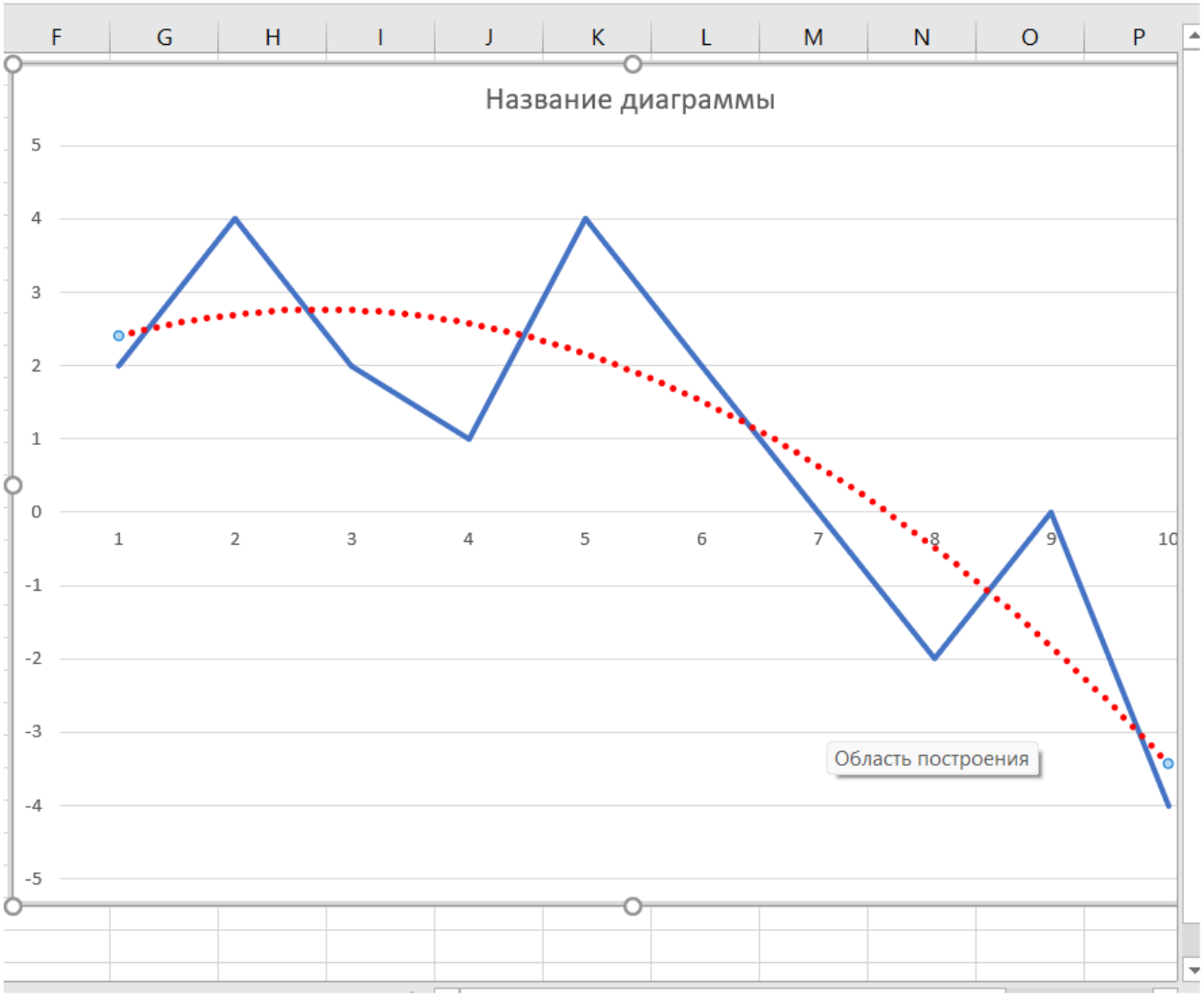
$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_m x^m$$

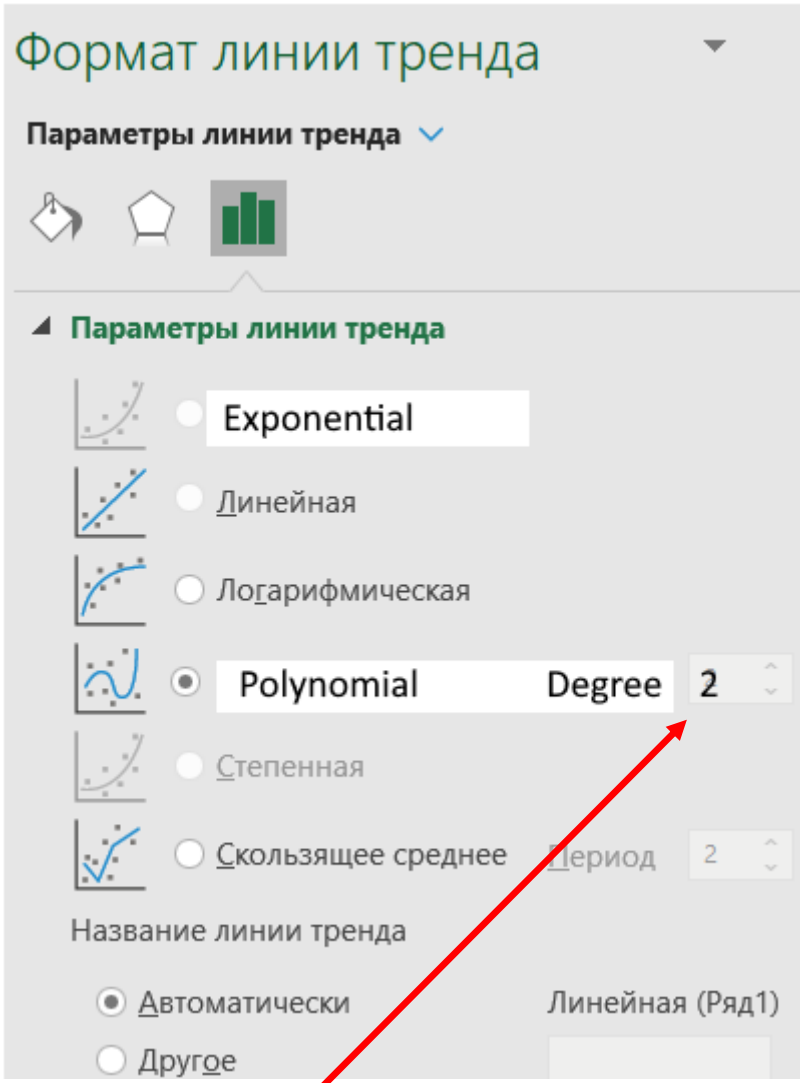
- -

Sometimes it is reasonable to use an exponential fitting curve ae^{bx} **or** ab^x **,**

or logarithm $a + \ln(bx)$

EXCEL





It is recommended to try several values of degree

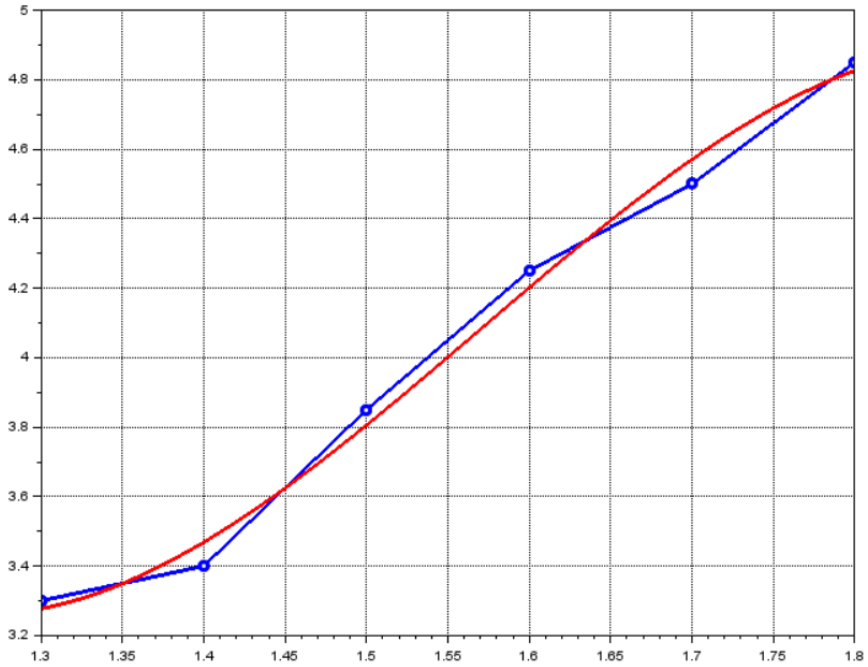
3 pages above, we considered the example of fitting the function

x_i= 1.3 1.4 1.5 1.6 1.7 1.8

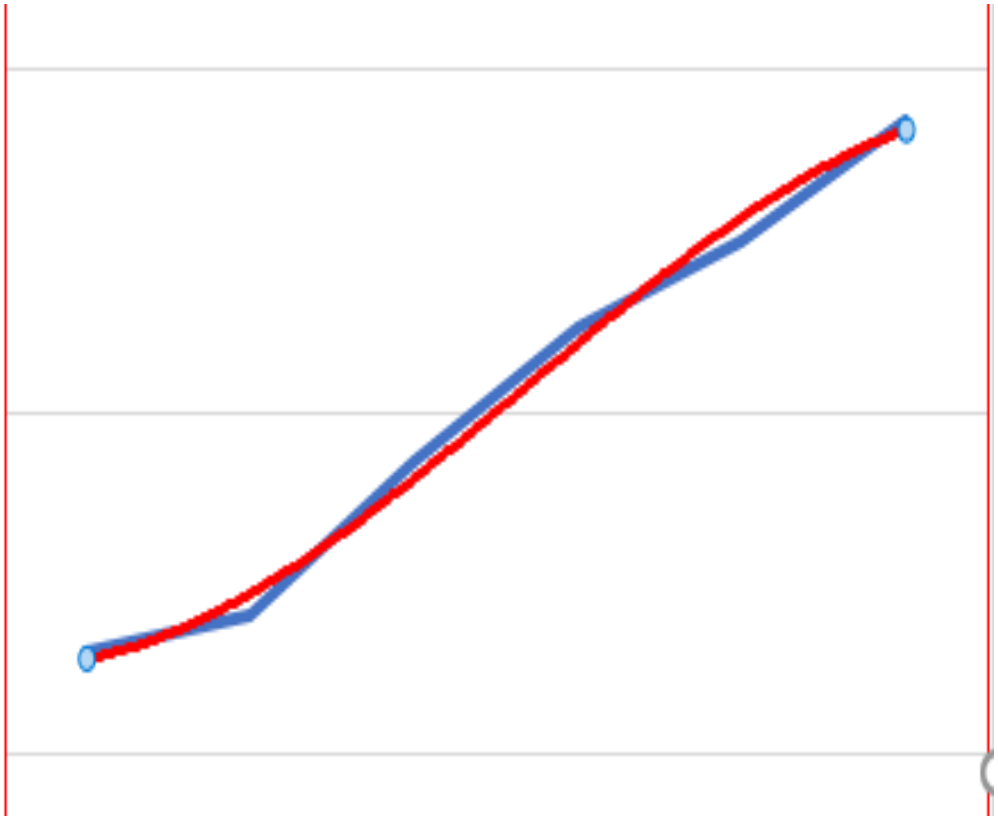
y_i= 3.3 3.4 3.85 4.25 4.50 4.85

with a polynomial of degree m=3

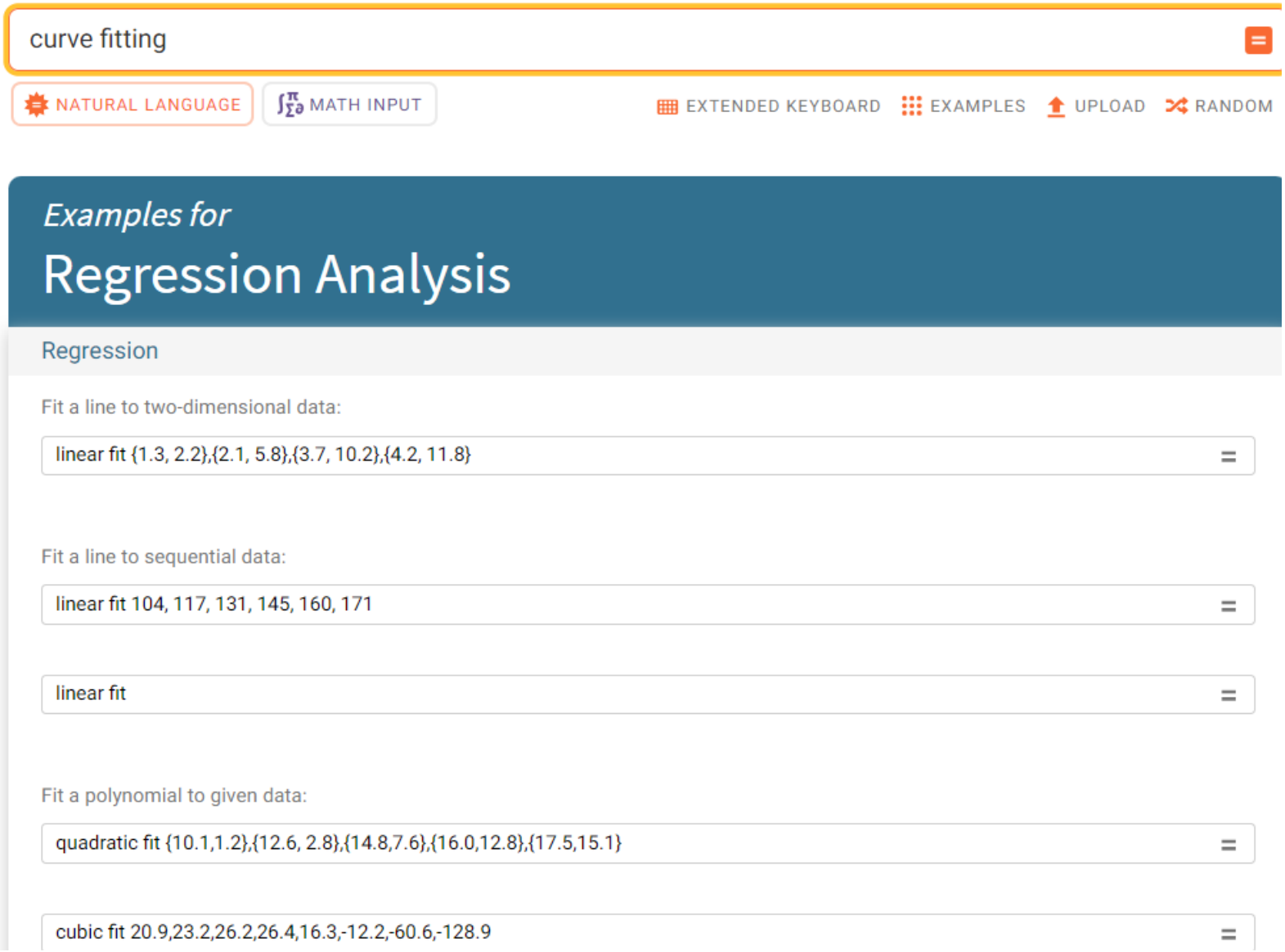
The result produced by Scilab was:



The result produced by EXCEL:



https://www.wolframalpha.com/input?i=curve+fitting



Fitting by a logarithm $a + \ln(bx)$

» data set of y values: {1.8, 2.4, 2.7, 3.1, 3.2, 3.4, 3

Compute

Assuming data set of y values | Use data set of {x,y} values

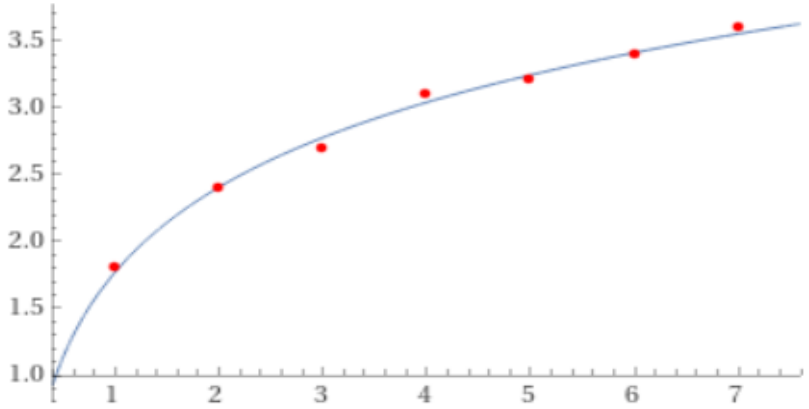
Input interpretation

fit	data	{1.8, 2.4, 2.7, 3.1, 3.2, 3.4, 3.6}
	model	logarithmic

Least-squares best fit

$0.914619 \log(6.93942\, x)$

Plot of the least-squares fit



Notice. Polynomials

$$p(x) = a_0 + a_1 x + a_2 x^2 + \ldots + a_m x^m \qquad (**)$$

where m>10 are difficult to use in practice, as determinant of the system (*) becomes very close to 0, and the system becomes ill-conditioned.

Then, for least squares approximation, one can use **Chebyshev polynomials** $T_k(x)$ instead of (**):

$$a_0 T_0 + a_1 T_1(x) + a_2 T_2(x) + \ldots + a_m T_m(x)$$

$$\sum_{i=0}^{n} [a_0 T_0 + a_1 T_1(x_i) + \ldots + a_k T_k(x_i) + \ldots + a_m T_m(x_i) - y_i]^2 \rightarrow$$

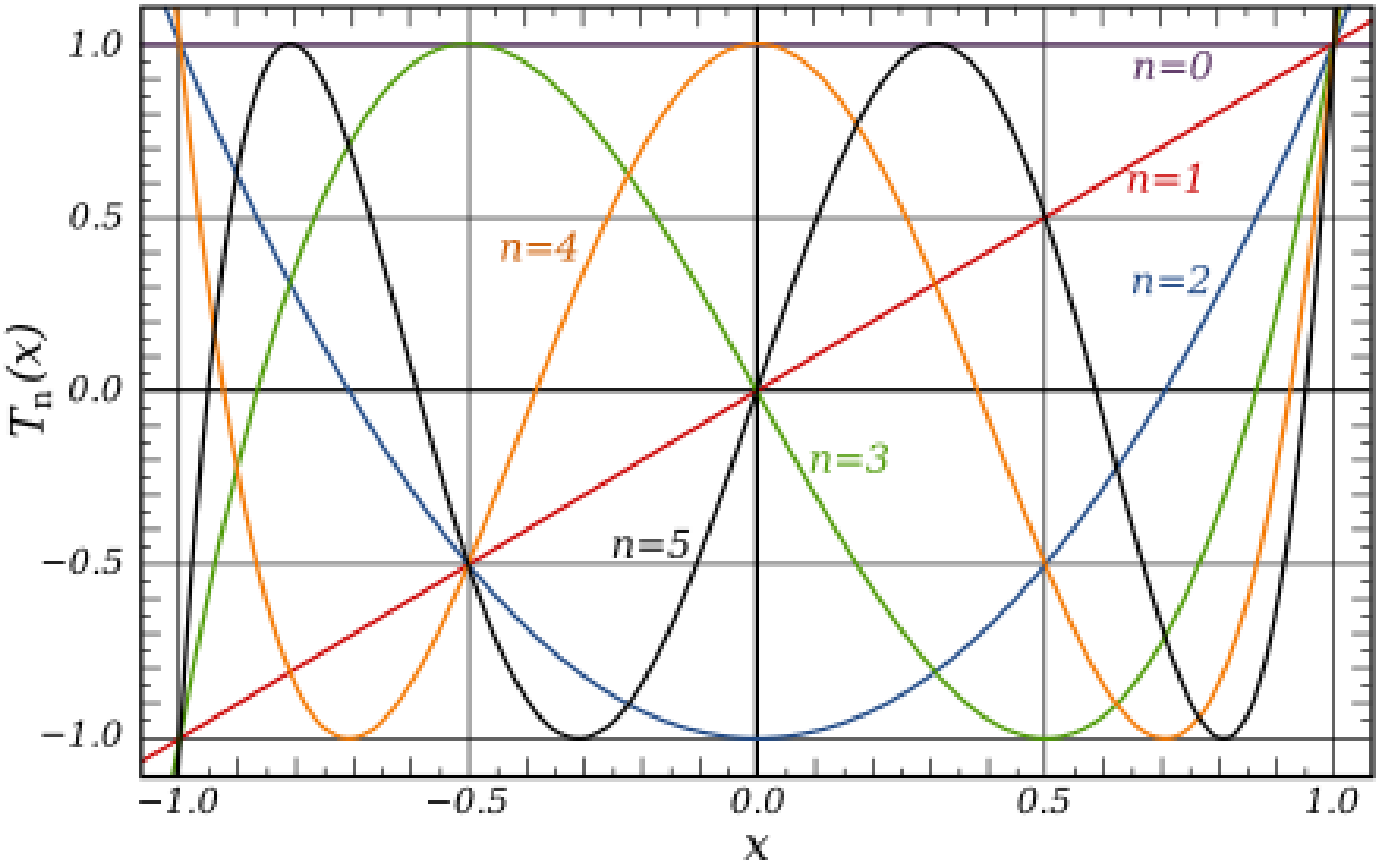
$$\rightarrow \min$$

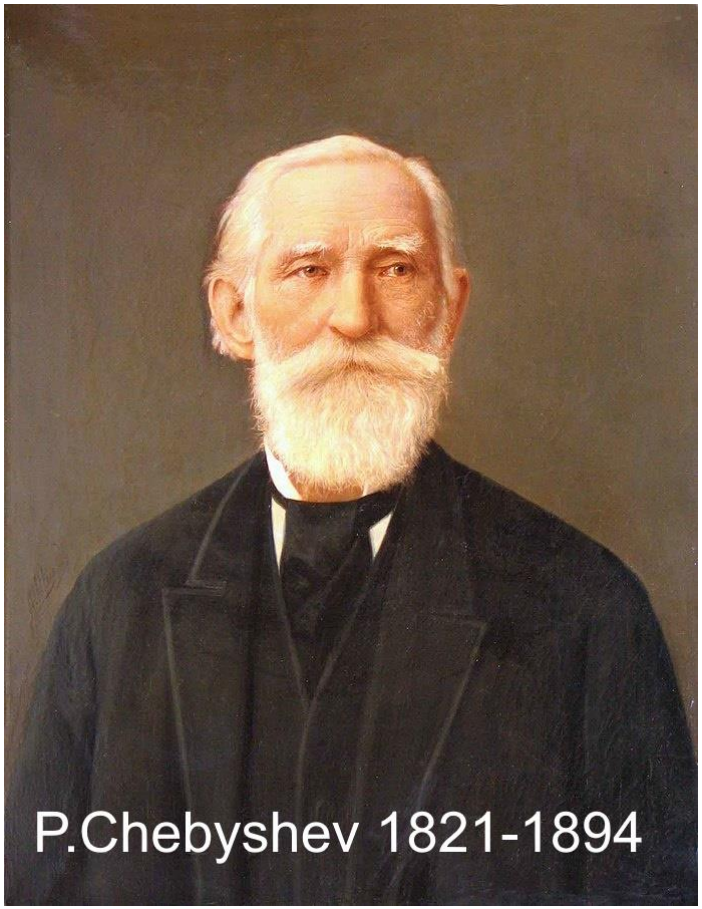
***Definition of* Chebyshev polynomials :**

$$T_0=1 \qquad T_1(x)=x \qquad T_2(x)=2x^2-1$$

$$T_k(x)=2xT_{k-1}(x)-T_{k-2}(x), \qquad k=3,4,\ldots$$

$$-1 \le x \le 1$$





For a different interval, one can use the substitution

$$t=(2x-x_0-x_n)/(x_n-x_0)$$

Least Squares for a function of 2 variables

Let function $z=f(x,y)$ be given by a table

(x_0, y_0, z_0), (x_1, y_1, z_1), . . . , (x_n, y_n, z_n)

In the following example, a linear function

$p(x,y) = a_0 + a_1 x + a_2 y$

is used for the least-squares approximation of z .

We should choose a_k to minimize the sum:

$$J = \sum_{i=0}^{n} (z_i - a_0 - a_1 x_i - a_2 y_i)^2$$

$$\frac{\partial J}{\partial a_0} = -2\sum (z_i - a_0 - a_1 x_i - a_2 y_i) = 0\,,$$

$$\frac{\partial J}{\partial a_1} = -2\sum x_i\,(z_i - a_0 - a_1 x_i - a_2 y_i) = 0\,,$$

and

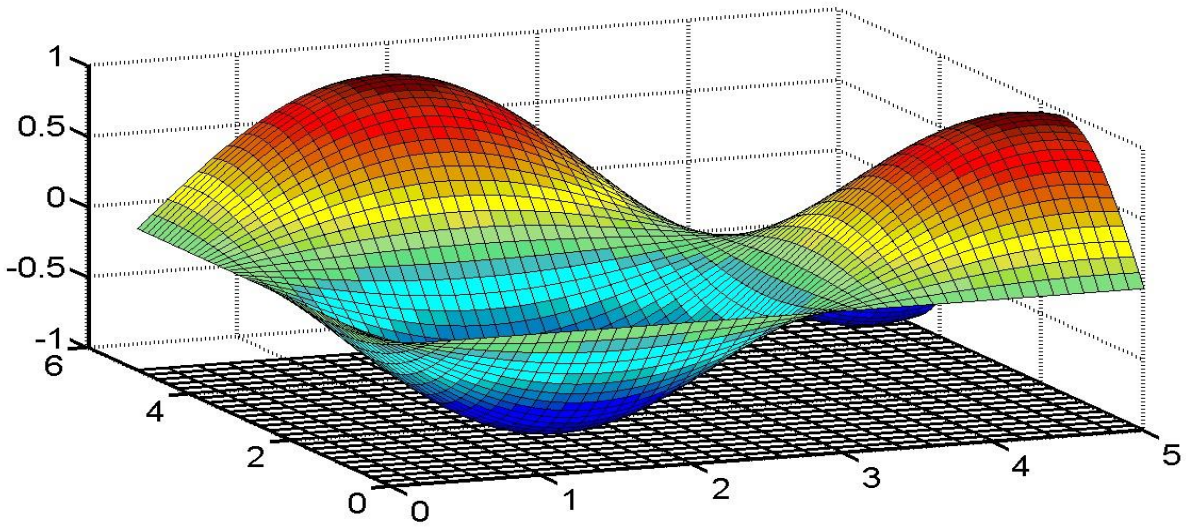
$$\frac{\partial J}{\partial a_2} = -2\sum y_i\,(z_i - a_0 - a_1 x_i - a_2 y_i) = 0\,.$$

These equations simplify to

$$\left.\begin{aligned} (n+1)a_0 + a_1\Sigma x_i + a_2\Sigma y_i &= \Sigma z_i \\ a_0\Sigma x_i + a_1\Sigma x_i^2 + a_2\Sigma x_i y_i &= \Sigma z_i x_i \\ a_0\Sigma y_i + a_1\Sigma y_i x_i + a_2\Sigma y_i^2 &= \Sigma z_i y_i \end{aligned}\right\}$$

from which a_0, a_1 and a_2 can be determined.

Chapter 8. Method of coordinate descent for finding a minimum of a multivariable function



Let we have a function of n variables

$$F(x_1, x_2, \ldots, x_n) \qquad a_i < x_i < b_i$$

and we seek coordinates of a point at which the function attains a minimum.

(If you need to find a maximum, then consider the function $-F$ and seek a minimum.)

The most primitive way is to introduce a lot of nodes in the domain of definition, then calculate F at the nodes and use computer command "if . . . less . . . then" for identifying the node at which minimum is attained.

This may be very time-consuming. For example, if n=7 and each segment $[a_i, b_i]$ contains 101 node, then total number of nodes is $101^7 > 10^{14}$

If calculation of *F* at a single node requires, for example, 50 arithmetic operations, then total number of necessary operations is more than 5×10^{15}.

Intel Core i7-7700K Processor performs roughly 38 billion = $38 \cdot 10^9$ floating point operations per second. It will take $5 \times 10^{15} / 38 \cdot 10^9 = 131600$ seconds ≈ 36.5 hours to do this job.

Therefore, there is need in more efficient methods of finding minimum.

Method of coordinate descent:

an idea is to vary coordinates by turns for the search of a minimum.

Let us choose some initial point

$$x^{(0)} = (x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)})$$

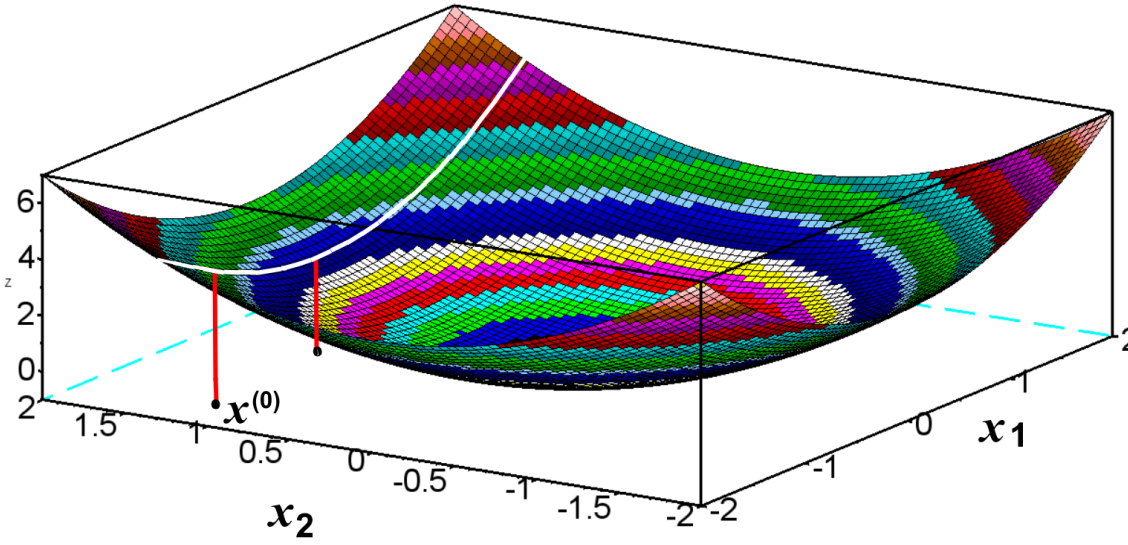
and vary the first coordinate, keeping others:

$$(x_1, x_2^{(0)}, \ldots, x_n^{(0)})$$

$$a_1 \le x_1 \le b_1$$

By calculation of F at nodes on this segment, one can find a point of minimum

$$(x_1^{(1)}, x_2^{(0)}, \ldots, x_n^{(0)}).$$



After that, we fix all coordinates except for

x_2 **and search a minimum under variation of** x_2 **:**

$(x_1^{(1)}, x_2, \ldots, x_n^{(0)})$

$(x_1^{(1)}, x_2^{(1)}, \ldots, x_n^{(0)})$

. . . . . .

$(x_1^{(1)}, x_2^{(1)}, \ldots, x_n^{(1)}) = x^{(1)}$

$x^{(2)}$

$x^{(3)}$

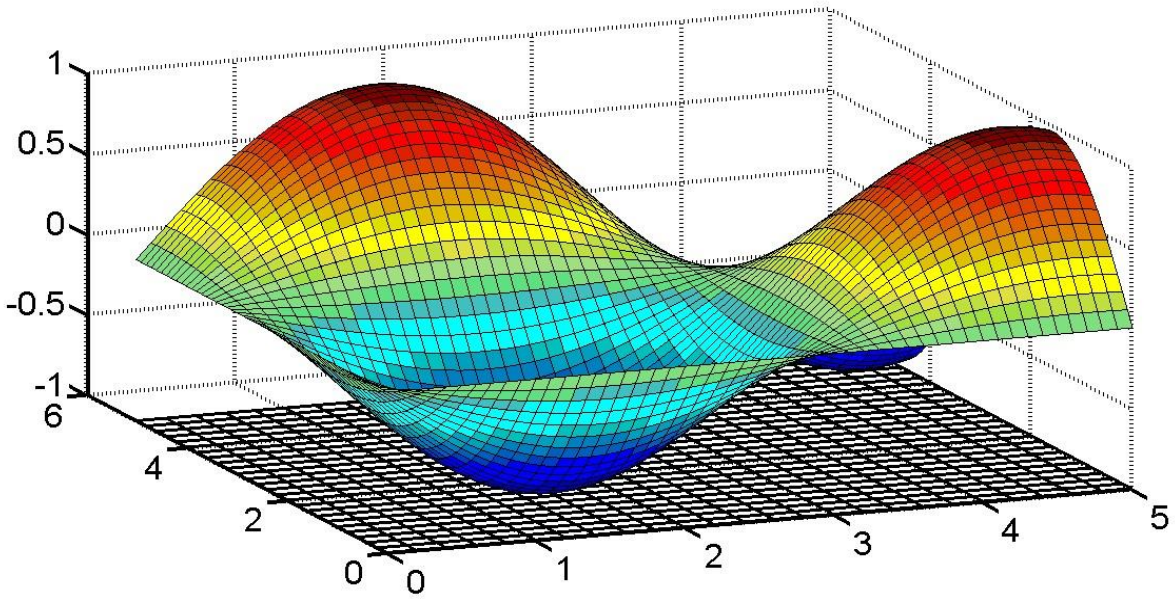
$x^{(k)}$

At each step, $F(x^{(k)}) \leq F(x^{(k-1)})$

Method of gradient descent

for finding a minimum of a multivariable function

https://en.wikipedia.org/wiki/Gradient_descent



Again, we have a function of n variables

$$F(x_1, x_2, \ldots, x_n)$$

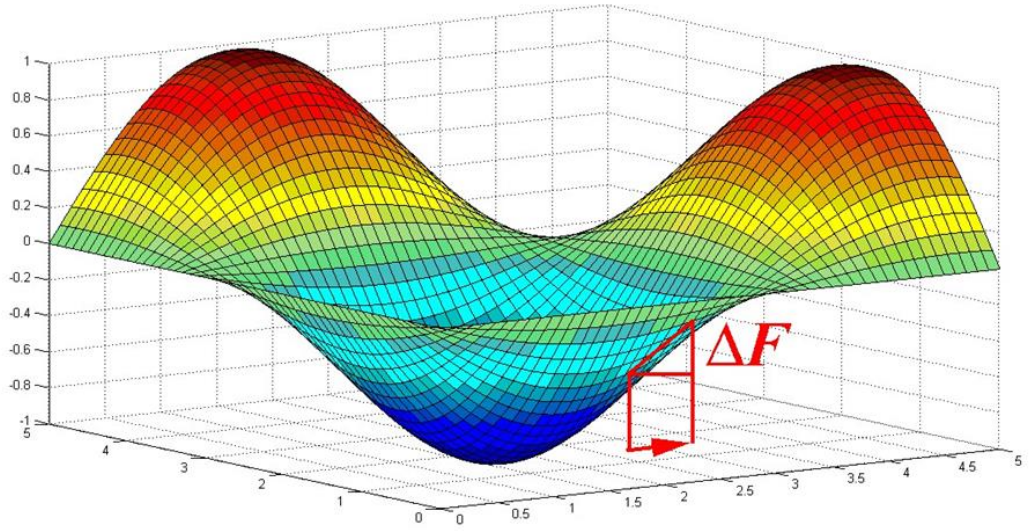
and we seek coordinates of a point at which the function attains a local minimum.

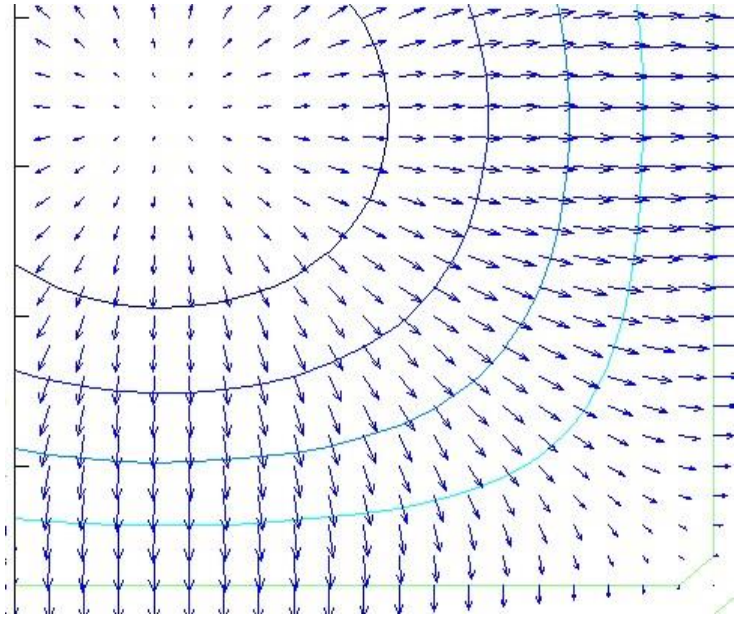
Let us choose an initial point: $x^{(0)} = (x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)})$,

calculate partial derivatives, and compose the gradient

$$\mathbf{grad}\ F(x^{(0)}) = \sum_{i=1}^{n} \bar{e}_i\, \partial F(x^{(0)})/\partial x_i$$

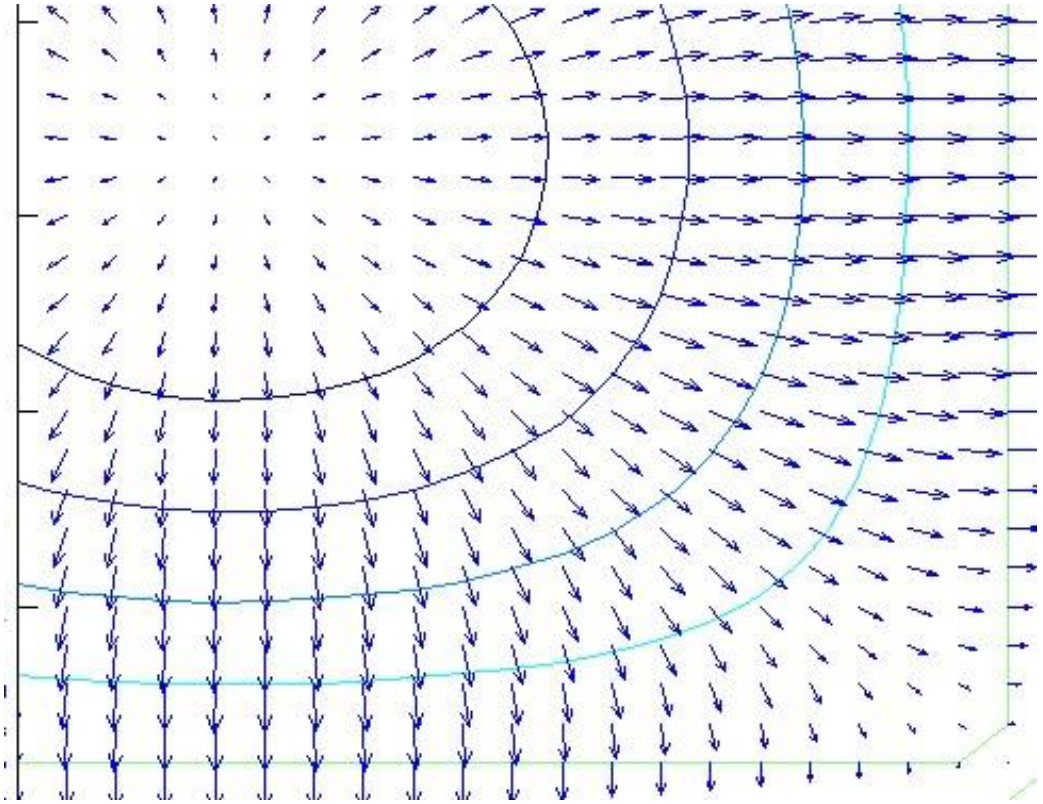
As known, vector $\mathbf{grad}\ F(x^{(0)})$ indicates the direction of most rapid rise of the function $F(x)$ at point $x^{(0)}$ in the plane (x,y).



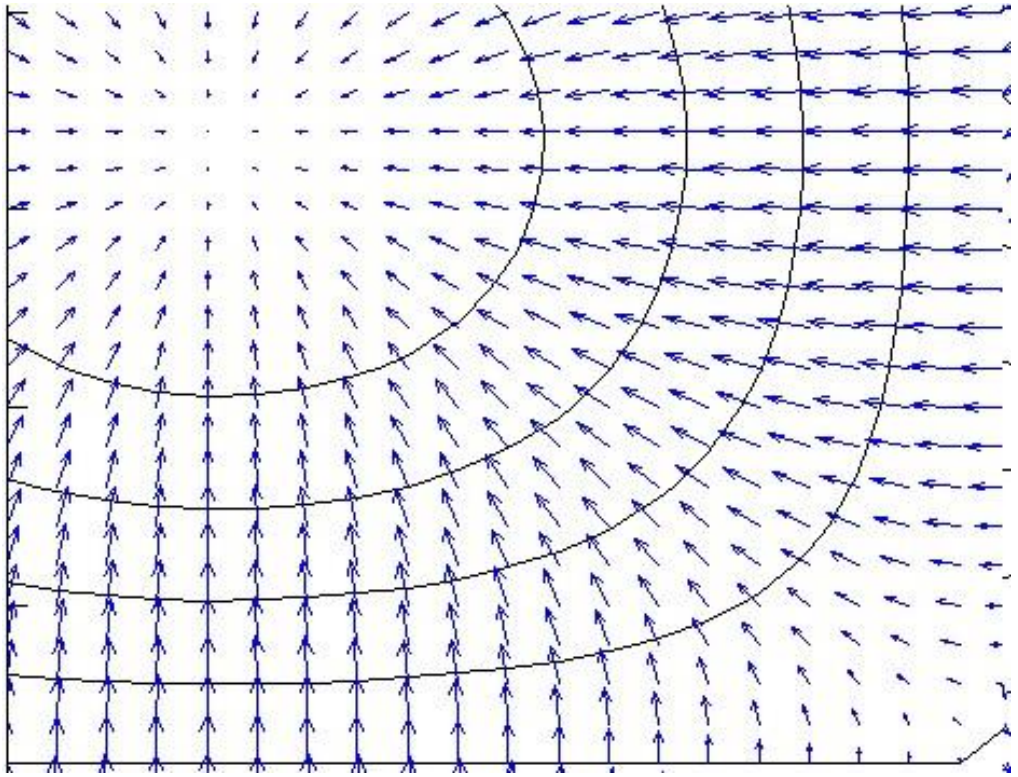


The gradient grad F is normal to level lines

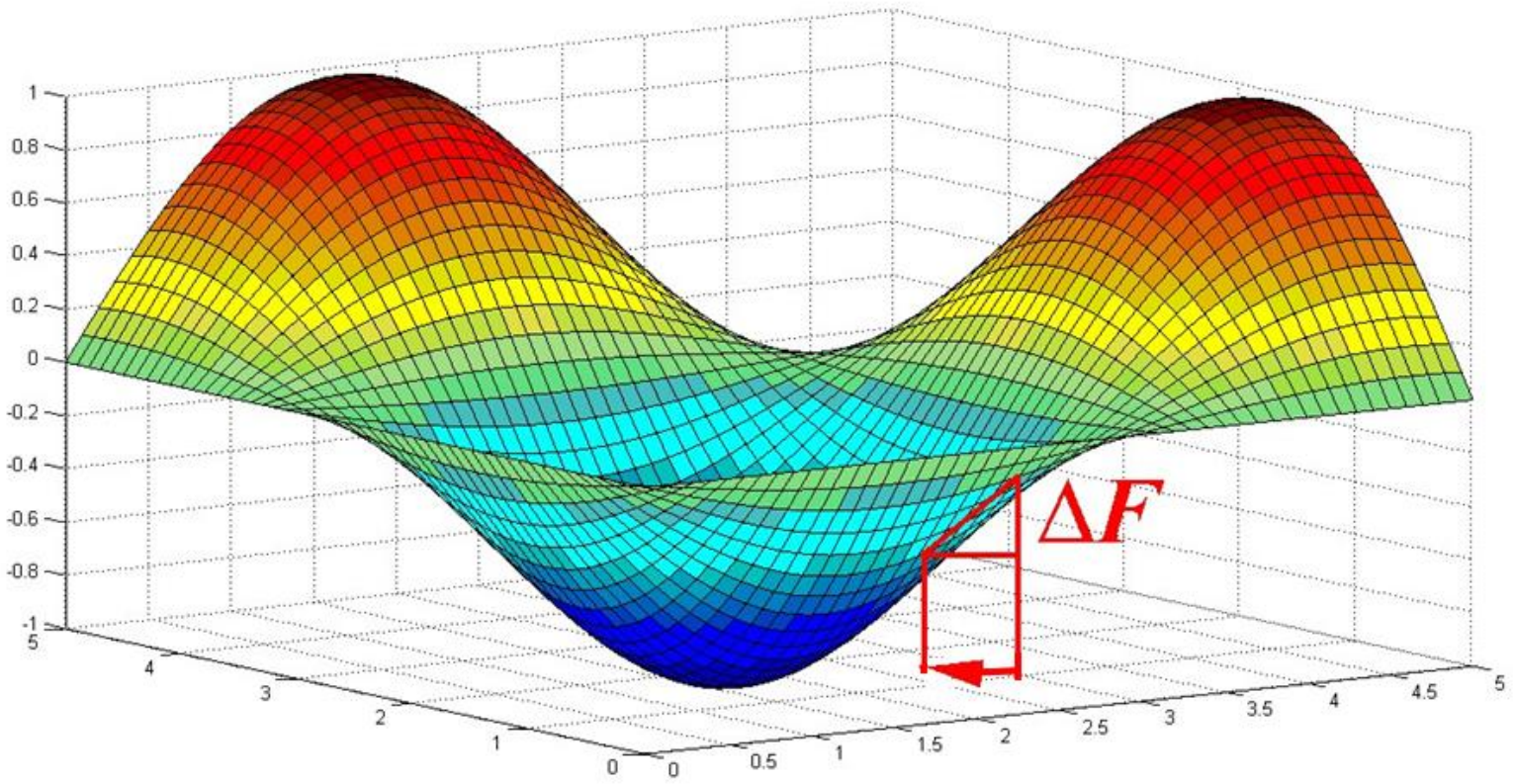
grad F



-grad F

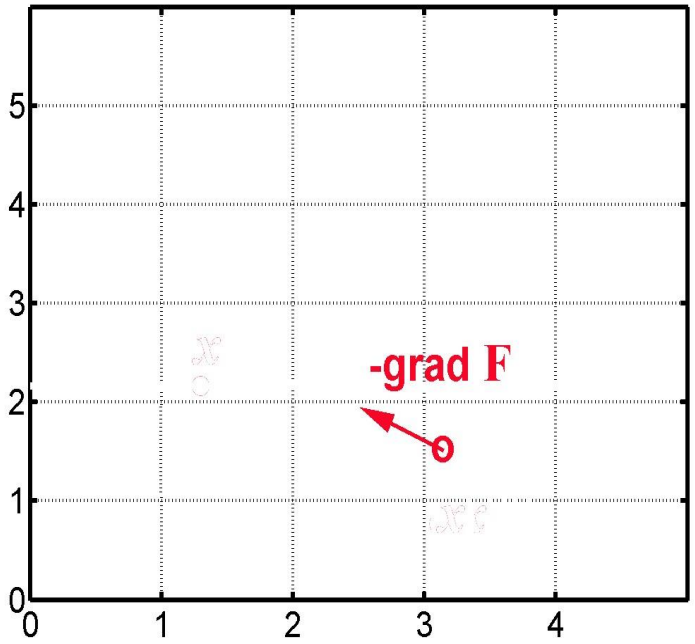


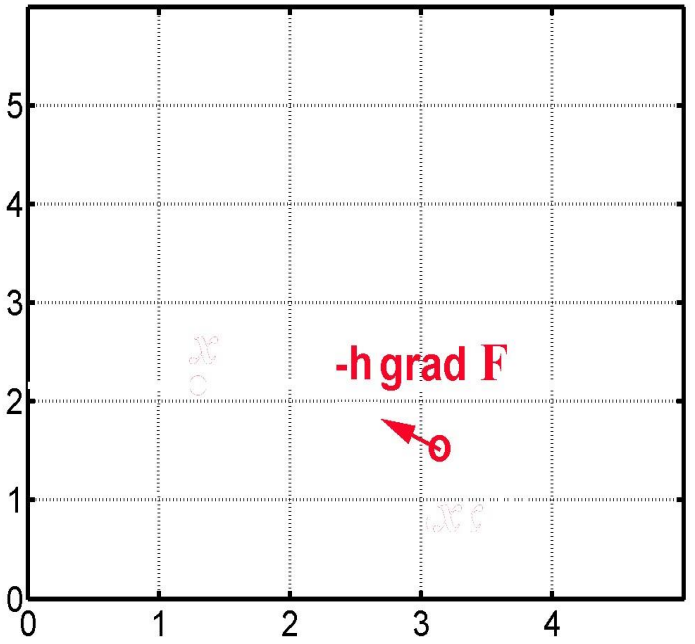
Antigradient $-\mathbf{grad}\ \mathrm{F}(x^{(0)})$ **indicates the direction of fastest <u>decrease</u> (descent) of the function** $\mathrm{F}(x)$ **at this point**



If we make a step of length $\left|\mathrm{grad}\,F(x^{(0)})\right|$ in the direction of antigradient, then we arrive at a point where gradient changes, therefore, the direction of fastest decrease changes.

It is reasonable to consider a shorter vector $-h\cdot\mathrm{grad}F(x^{(0)})$, where h is a small parameter, in order to correct the direction of descent:

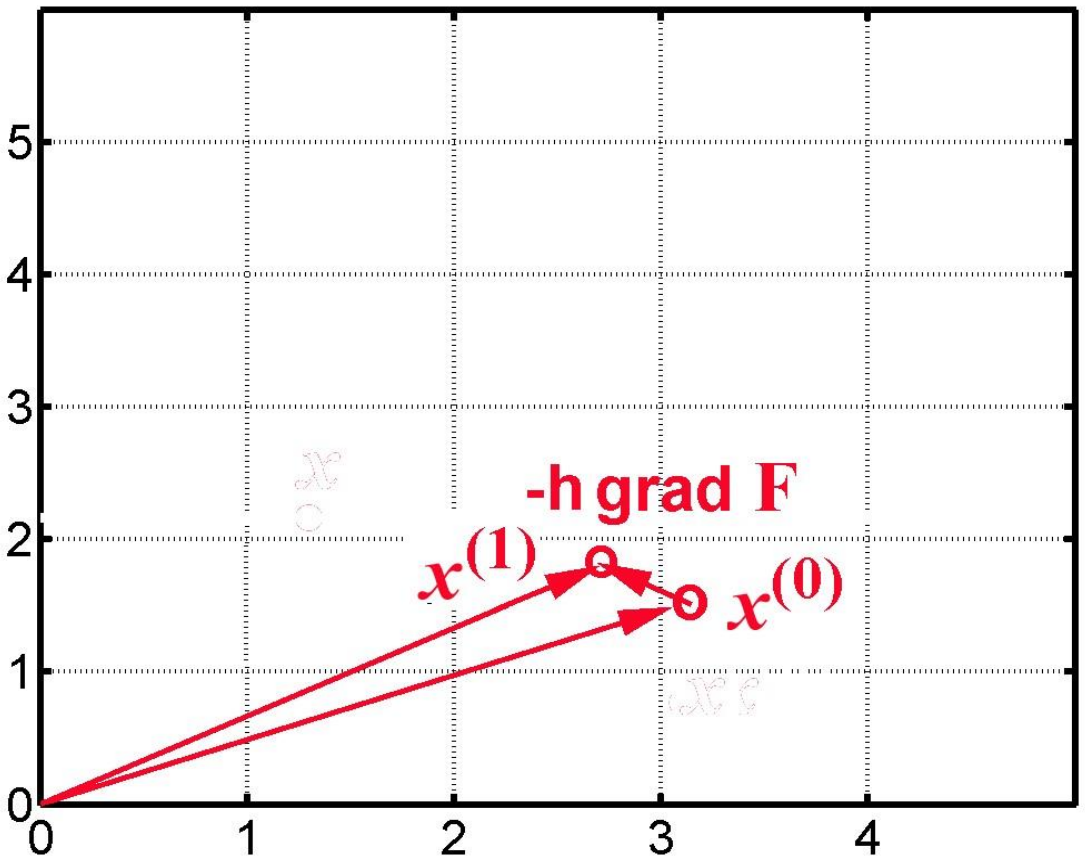




$x^{(1)} = x^{(0)} - h \cdot \mathrm{grad}\, F(x^{(0)})$ **- in vector form.**

Cartesian components:

$$x_i^{(1)} = x_i^{(0)} - h \cdot \partial F(x^{(0)}) / \partial x_i$$



Similarly, we perform further steps, which lead to a minimum of F(*x*):

$$x_i^{(k+1)} = x_i^{(k)} - h \cdot \partial F(x^{(k)}) / \partial x_i$$

$$x_i^{(k+1)} = x_i^{(k)} - h \cdot \partial F(x^{(k)})/\partial x_i$$

Notice:

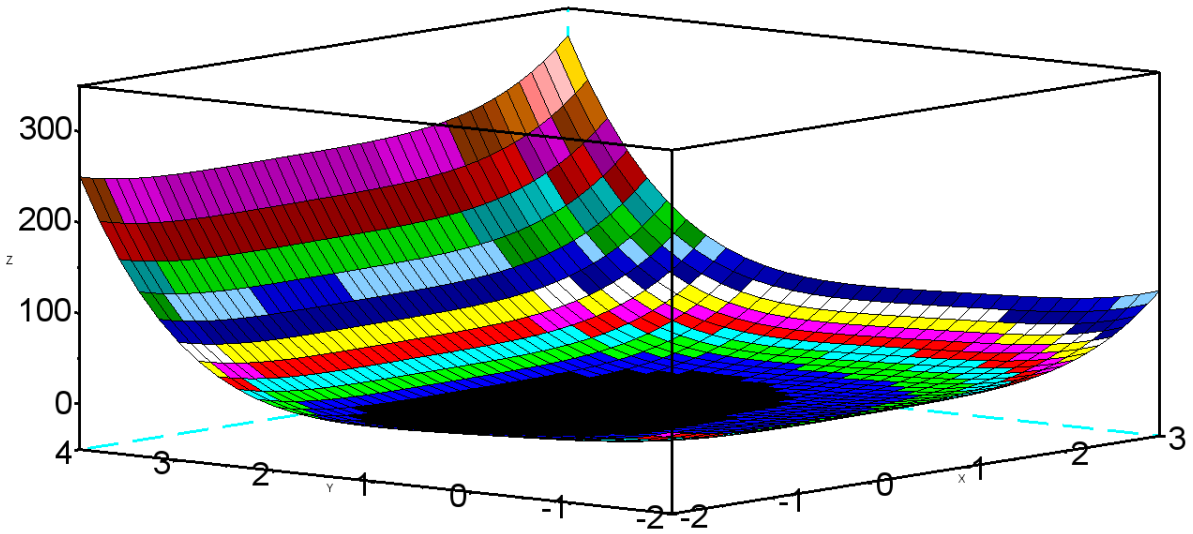
As soon as we approach a minimum, the derivatives $\partial F(x^{(k)})/\partial x_i$ decrease; therefore, the difference $x^{(k+1)} - x^{(k)}$ decreases as well.

Example 1. Find a minimum

z=F(x,y)= x^4 + y^4 - 5y +x -2<x<3, -2<y<4

Scilab :

```
for i=1:41
for j=1:31
    x(i,j)=0.125*(i-1)-2
    y(i,j)=0.2*(j-1)-2 end
end
F=x.^4+y.^4 -5*y+x
surf(x,y,F)
```



```
h=0.01
xx=2.5
yy=3.5
plot(4,5)
for k=1:300
dFdx=4*xx^3 +1
dFdy= 4*yy^3-5
xx=xx-h*dFdx
yy=yy-h*dFdy
plot(xx, yy,'or','LineWidth',2)
end
disp('xx=',xx,'yy=',yy,'k=',k)
```

answer: xx= -0.6299189
yy= 1.0772173

In Scilab code, it makes sense to set another condition to stop computations:

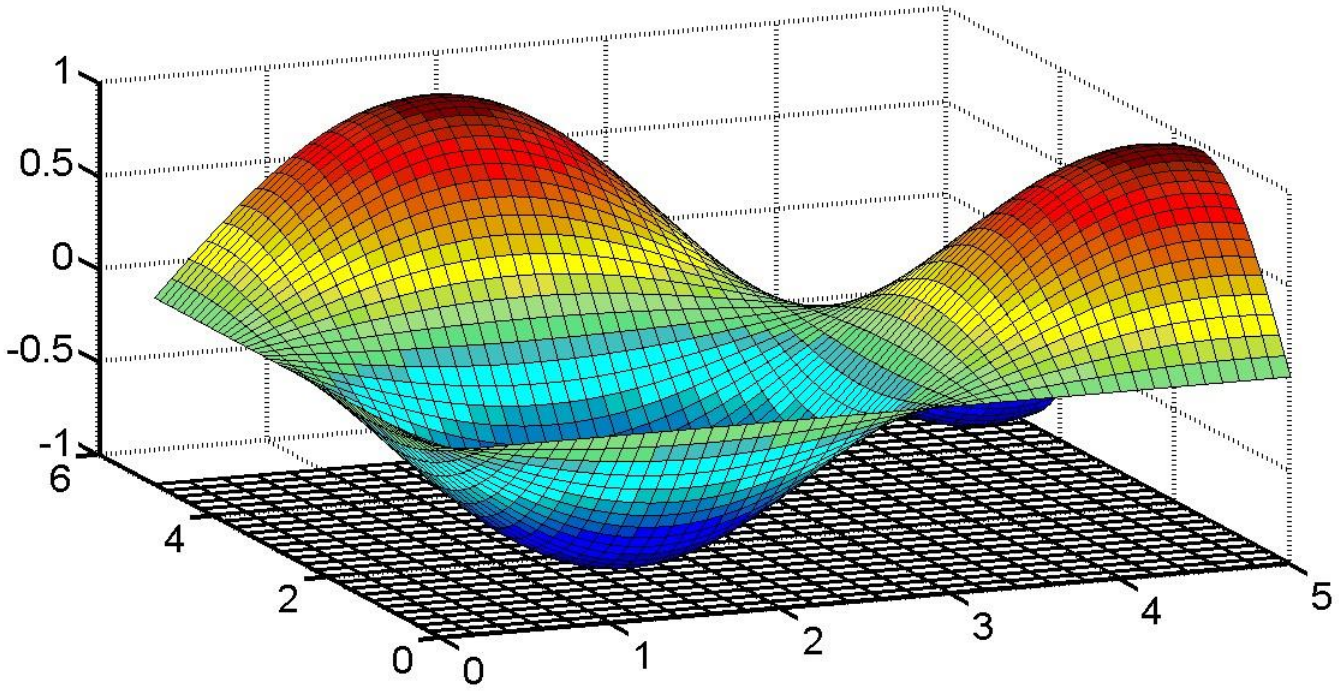
$$\left| x_i^{(k+1)} - x_i^{(k)} \right| < \varepsilon \qquad \text{for all} \quad i$$

Or $$\left| F(x^{(k+1)}) - F(x^{(k)}) \right| < \varepsilon$$

(instead of setting total number of steps k=1:300**)**

Remark 1.

Possible non-uniqueness of local minima:



Remark 2.

Often, in the method of gradient descent, one cannot find an analytical expression for partial derivatives of F.

Then some numerical methods for finding the derivatives can be used, see Chapter 11.

Method of most rapid gradient descent for finding a minimum of a function

$$z = \mathbf{F}(x_1, x_2, \ldots, x_n)$$

Recall Method of gradient descent:

Choose some $x^{(0)} = (x_1^{(0)}, x_2^{(0)}, \ldots, x_n^{(0)})$

Calculate

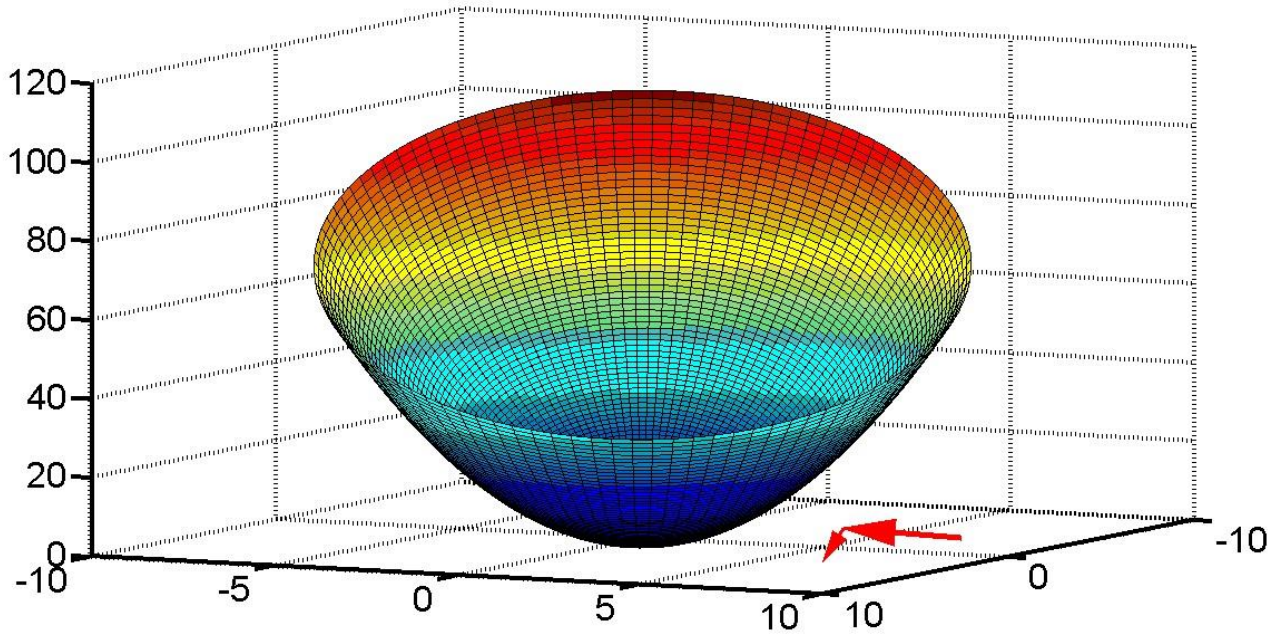
$$\mathbf{grad}\, F(x^{(0)}) = \sum_i \bar{e}_i \, \partial F(x^{(0)})/\partial x_i$$

Sequential approximations:

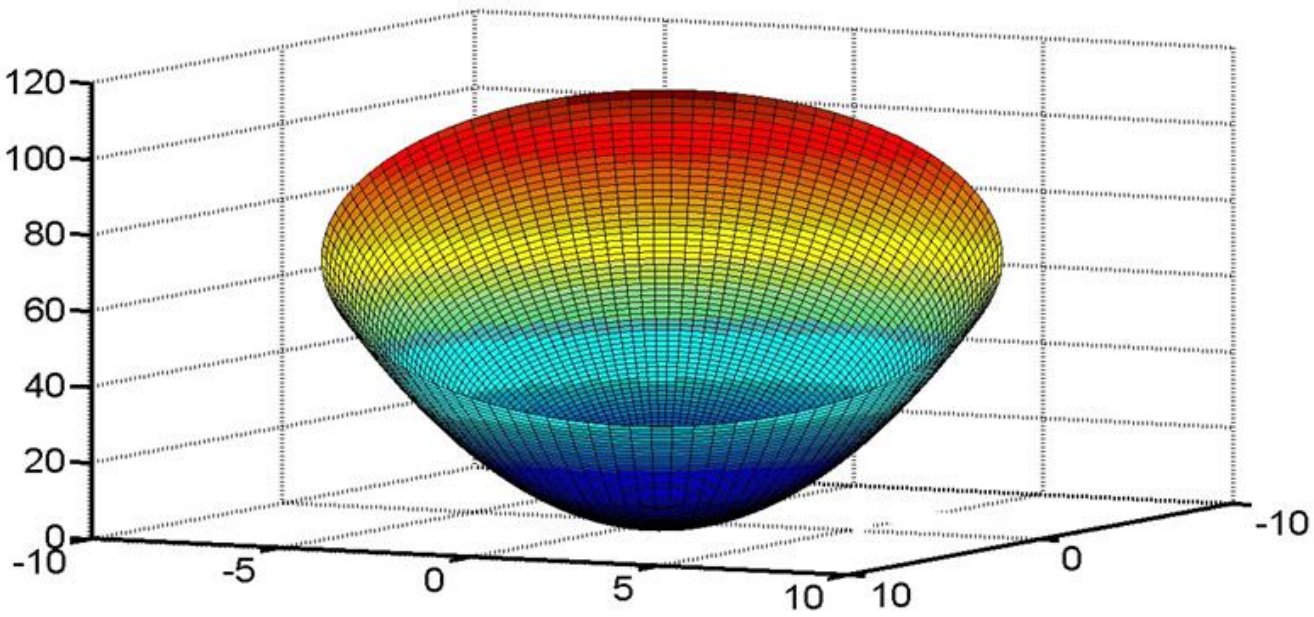
$$x_i^{(k+1)} = x_i^{(k)} - h \cdot \partial F(x^{(k)})/\partial x_i$$

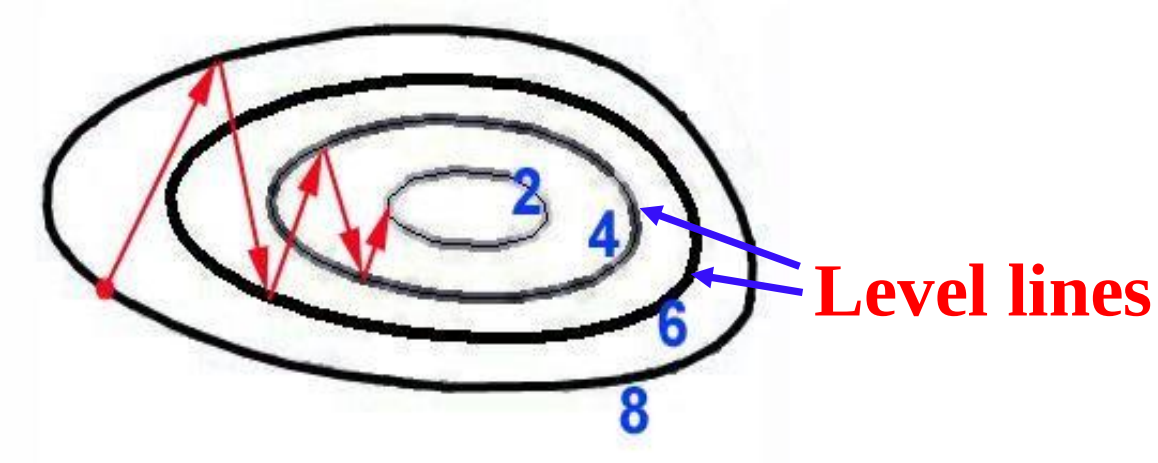
The parameter h governs the length of vector.

If h is too small, then sequence $X^{(k)}$ approaches a solution *slow*, as it takes too many steps and a lot of computing time.



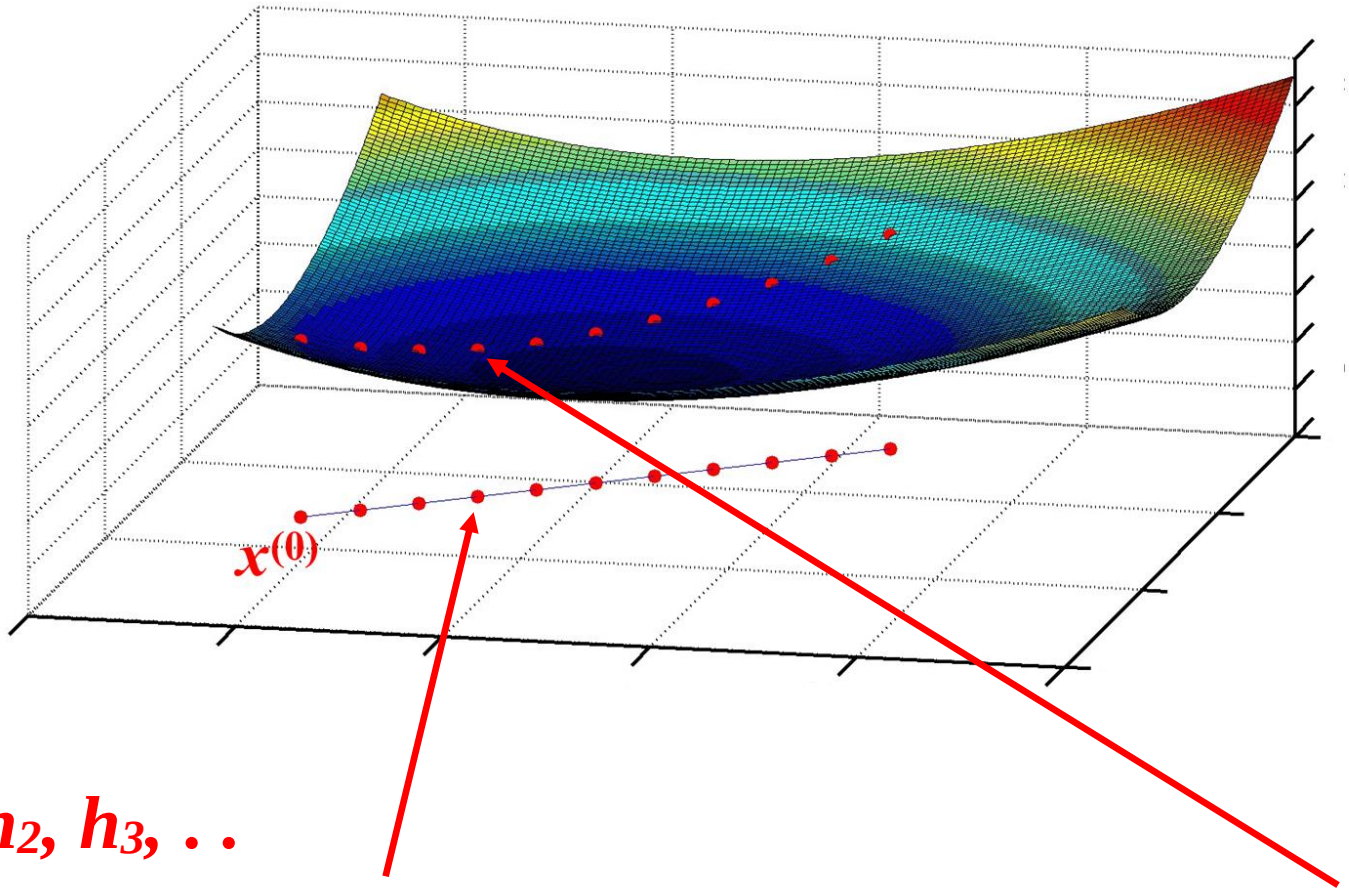
If h is too large, then sequence $x^{(k)}$ may converge slow as well, as it may pass by a minimum:





(or the sequence may happen to diverge if h is very large).

What is the optimal value of h ?

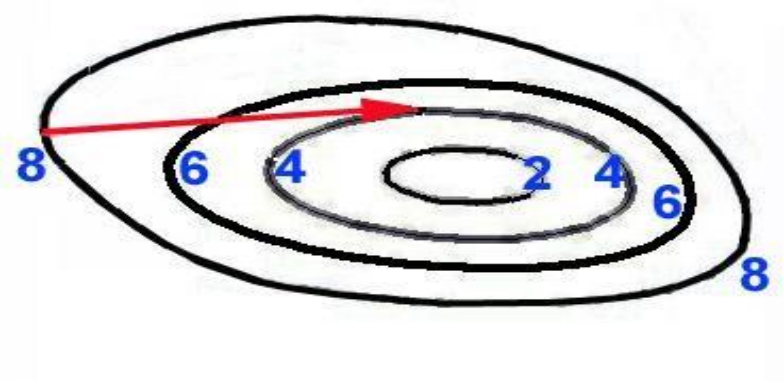


Consider various $h_1, h_2, h_3, \ldots$

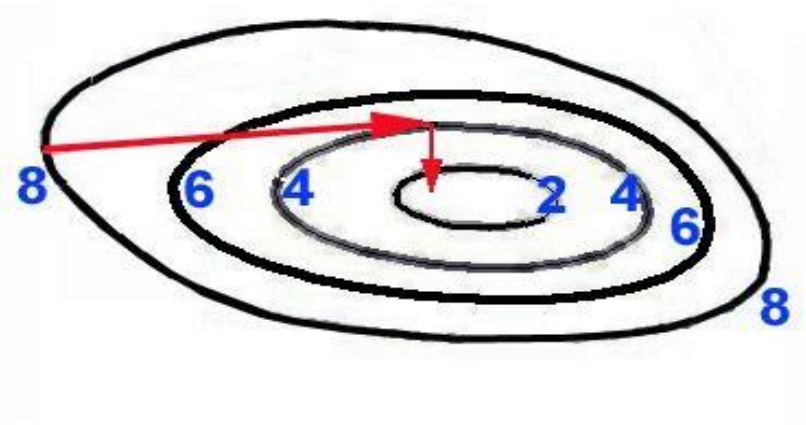
Suppose we have found such h_{opt}, that $\mathbf{F}$ attains a minimum $\mathbf{F}_{min}$

in the direction of antigradient. This can be achieved with a method of 1D optimization, for example method of golden section, see the end of this chapter.

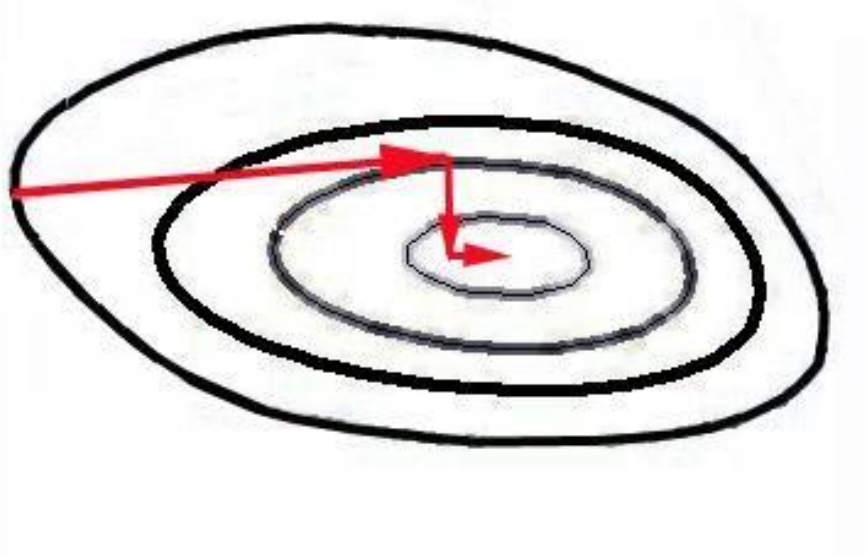
As soon as we found such optimum h_{opt}, that ensures a minimum of F, we move from $x_i^{(0)}$ to $x_i^{(1)} = x_i^{(0)} - h_{opt} \cdot \partial F(x^{(0)})/\partial x_i$



At the point of minimum $x^{(1)}$, the direction, in which the step was made, is tangent to a level line.
Antigradient calculated at this point will be normal to the level line:



Therefore next step from $x^{(1)}$ to $x^{(2)}$ will be made in the normal (orthogonal) direction, as antigradient is normal to the level line that passes through point $x^{(1)}$.

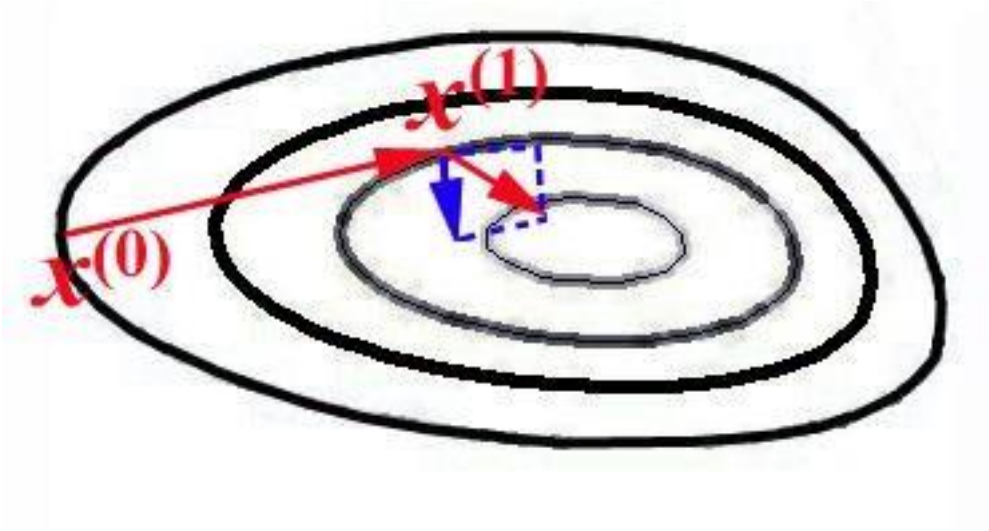


In the same way, at next steps, we seek $h^{(k)}_{opt}$ and move on:

$$x_i^{(k+1)} = x_i^{(k)} - h^{(k)}_{opt} \cdot \partial F(x^{(k)})/\partial x_i$$

Practice showed that method of <u>fastest</u> gradient descent is more efficient (from the viewpoint of necessary computing time) than method of gradient descent with a constant h .

Conjugate Gradient Method



Most rapid gradient descent:

$$x^{(1)}=x^{(0)}- h_{\text{opt}}\cdot \text{grad}F(x^{(0)}) \ , \qquad x^{(2)}=x^{(1)}- h_{\text{opt}}\cdot \text{grad}F(x^{(1)})$$

Conjugate gradient method:

$$x^{(2)}=x^{(1)}- h_{\text{opt}}\left[\text{grad}F(x^{(1)}) + \beta^{(1)}\cdot \text{grad}F(x^{(0)})\right]$$

correction of the direction of step

Scilab built-in command: fminsearch

```
function y=HIT(x)
  y = x(2)^2 + (1-x(1))^2;
endfunction
[x, fval] = fminsearch ( HIT, [1 -1] )
```

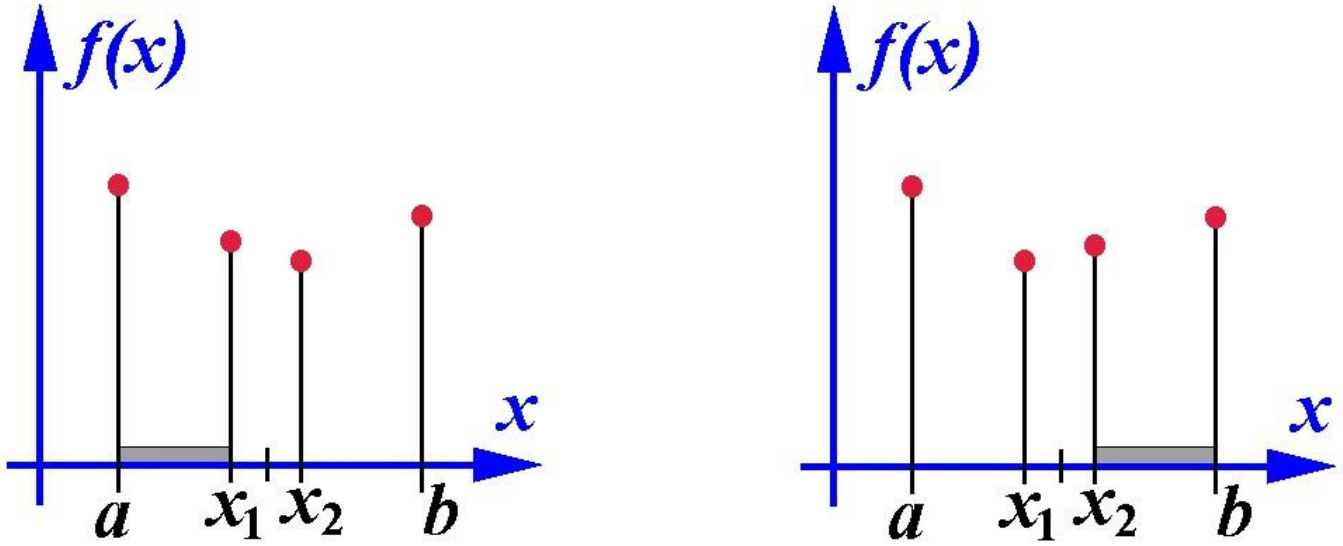
Computes the unconstrained minimum of given function HIT with the Nelder-Mead algorithm.

Same command "fminsearch" in Matlab. In addition, "gradient"

```
[FX,FY] = gradient(F)
```

returns the *x* and *y* components of the two-dimensional numerical gradient of matrix `F`.

Search of a minimum in case of a function of single independent variable. A bisection method.



Let $f(x)$ be given on $[a,b]$. We divide the segment into two equal parts and consider two points x_1 and x_2 located symmetrically about the midpoint

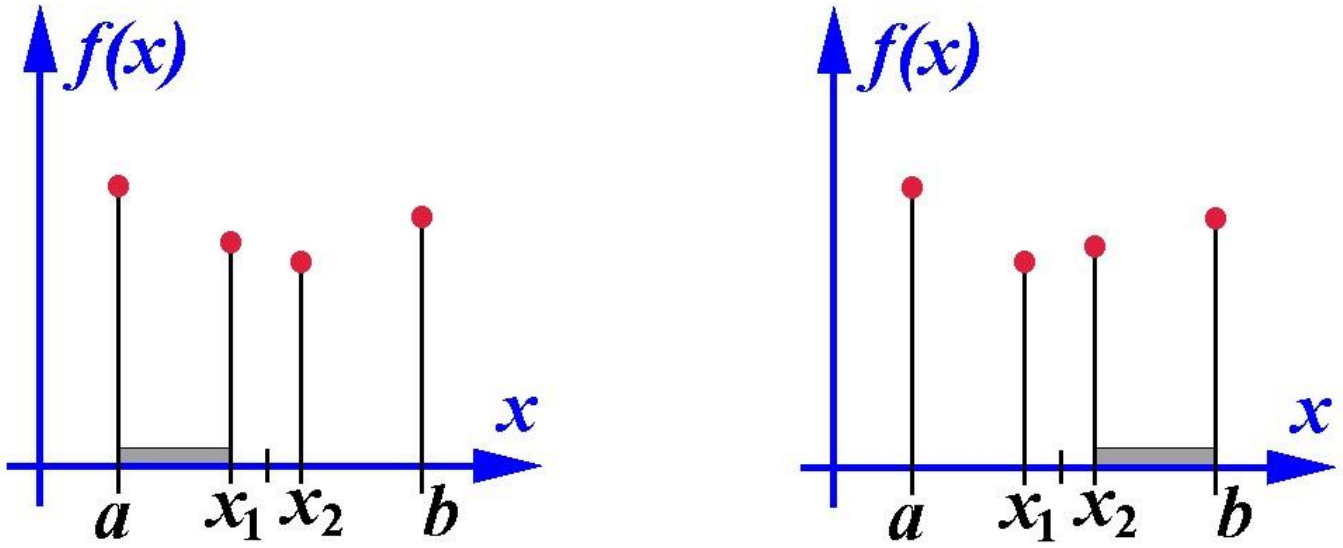
$$x_1 = 0.5\,(a+b) - \delta\,, \qquad x_2 = 0.5\,(a+b) + \delta\,,$$

where δ is small.

Let us calculate $f(x_1)$ and $f(x_2)$.

If $f(x_1) > f(x_2)$, then we drop $[a, x_1]$ and retain $[x_1, b]$

If $f(x_1) < f(x_2)$, then we drop $[x_2, b]$ and retain $[a, x_2]$

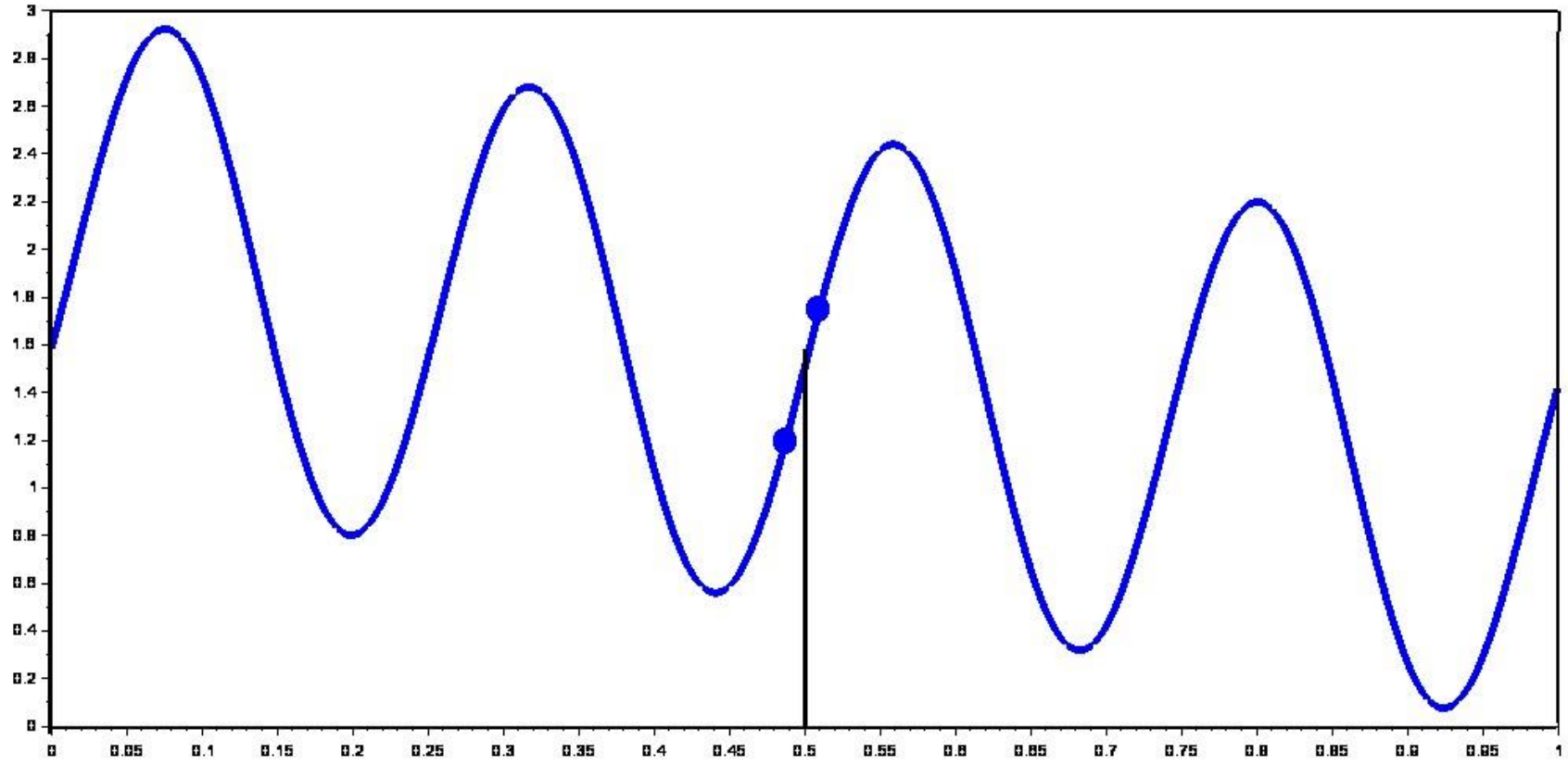


For the obtained segment $[a, x_2]$ or $[x_1, b]$ the procedure of halving the segment keeps on until the length of segment becomes small enough.

```
x=0:0.0002:1
y=x.*exp(x)+50*x.*sin(2*x).^2 -30*x
      plot(x,y)
      xgrid
 x1=0
 x2=1
 delta=0.0000005
 i=1
 while x2-x1> .000002
     x3=0.5*(x1+x2)-delta  ;
     x4=0.5*(x1+x2)+delta ;
 i=i+1 ;
 y3= x3*(exp(x3)+50*sin(2*x3)^2 -30)  ;
 y4= x4*(exp(x4)+50*sin(2*x4)^2 -30) ;
 if y3<y4 then        x2=x4   ;
 else x1=x3  ;
```

```
    end
disp( y3,i)
end
xmin=0.5*(x3+x4)
ymin= xmin*(exp(xmin)+50*sin(2*xmin)^2 -30)
disp("ymin=",ymin,"xmin=", xmin)
```

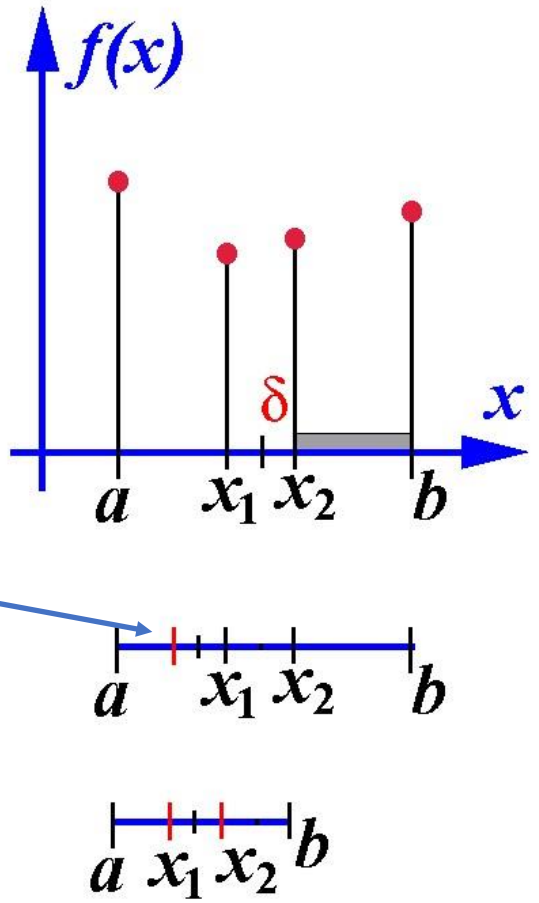
It is recommended to plot a graph of the function $f(x)$ in order to avoid a loss of a minimum.


3
2.8
2.8
2.4
2.2
2
1.8
1.8
1.4
1.2
1
0.8
0.8
0.4
0.2
0
0
0.05
0.1
0.15
0.2
0.25
0.3
0.35
0.4
0.45
0.5
0.55
0.8
0.85
0.7
0.75
0.8
0.85
0.9
0.95
1

A golden section method.

In the bysection method, at each step, one needs to calculate $f(x)$ at two new points x_1, x_2 .

In a golden section method, at each step, one calculates $f(x)$ only at one new point. This can be performed by choosing



$$\delta = (\sqrt{5}/2 - 1)*(b-a)$$

In such a case, after dropping $[x_2, b]$, on the retained segment $[a, x_2]$ we already know $f(x_1)$, and x_1 is well located with respect to the middle of the retained segment $[a, x_2]$. Therefore, we only need to calculate f at the symmetric point indicated by a bar |

On the new segment $[a, x_2]$, all proportions between locations of points are the same as ones on $[a, b]$:

$$(b-a)/(x_2-a) = (x_2-a)/(x_1-a) .$$

As a consequence, further reduction of the segment can be performed in the same way.

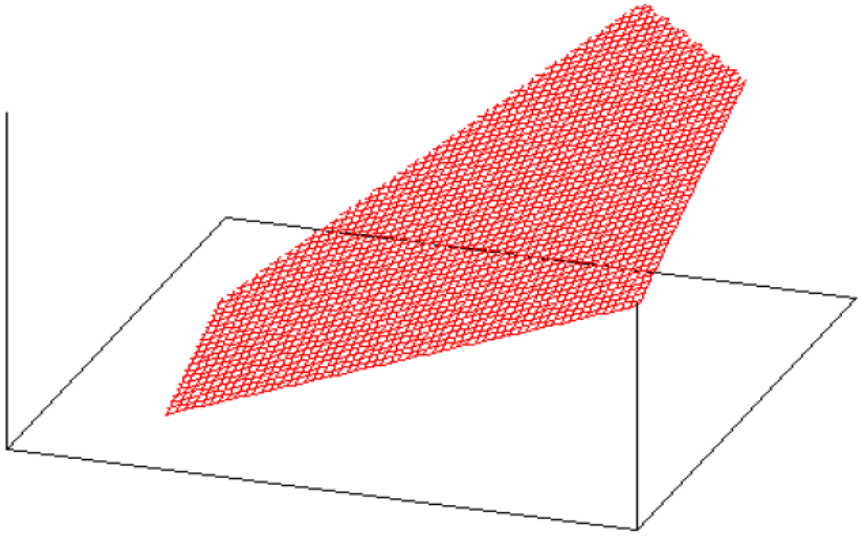
In the method of golden section, we need two times smaller amount of computations of $f(x)$; meanwhile the length of segment is reduced at each step by the smaller factor

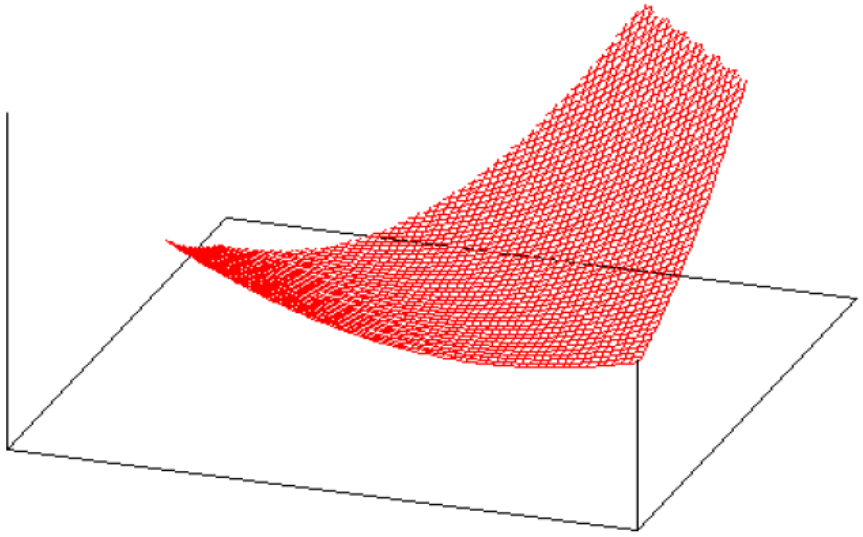
1.618033988… .

(not by factor 2). Therefore, the eventual gain of computational time is:

$$2 / 1.618033988$$

Chapter 9. Finding a minimum or maximum of a function under constraints for variables





9-1. Solving optimization problems under constraints using graphs.

Example. Find a maximum of the linear function

$$F(x,y) = x+3y$$

under constraints:

$$x - y \leq 1$$

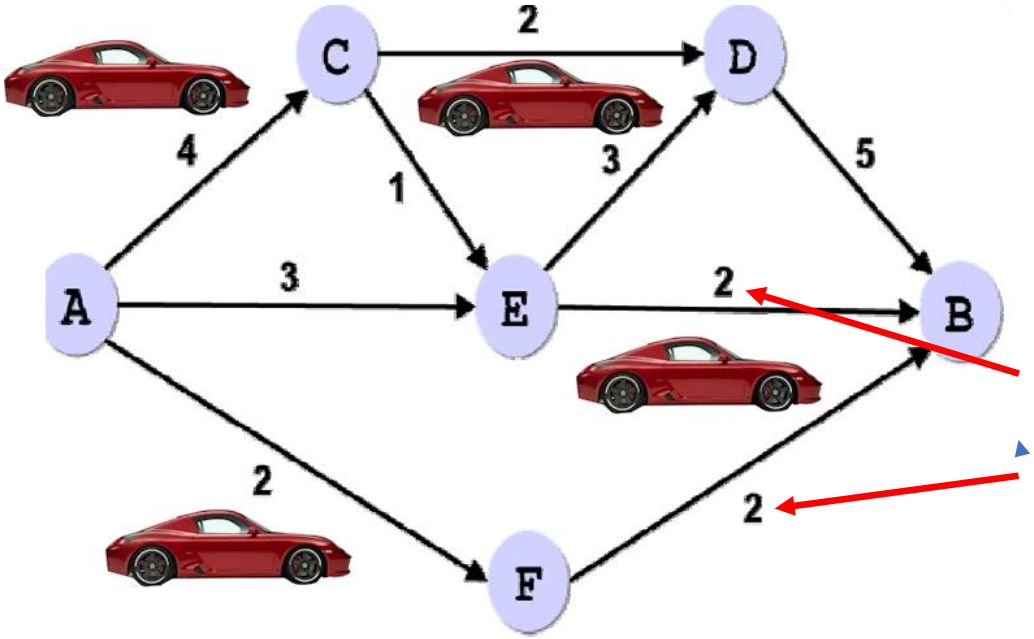
$$2x + y \leq 2$$

$$x - y \geq 0$$

$$x \geq 0$$

$$y \geq 0$$

9-2. Problem of Maximum Traffic



1) There are roads between cities A, B, C, D, E, F,

2) there are upper limits c_{ij} on the number of cars which can pass along each road per hour.

(the limits c_{ij} are given in thousands of cars).

Problem: find the maximum number of cars that can arrive in town B per hour.

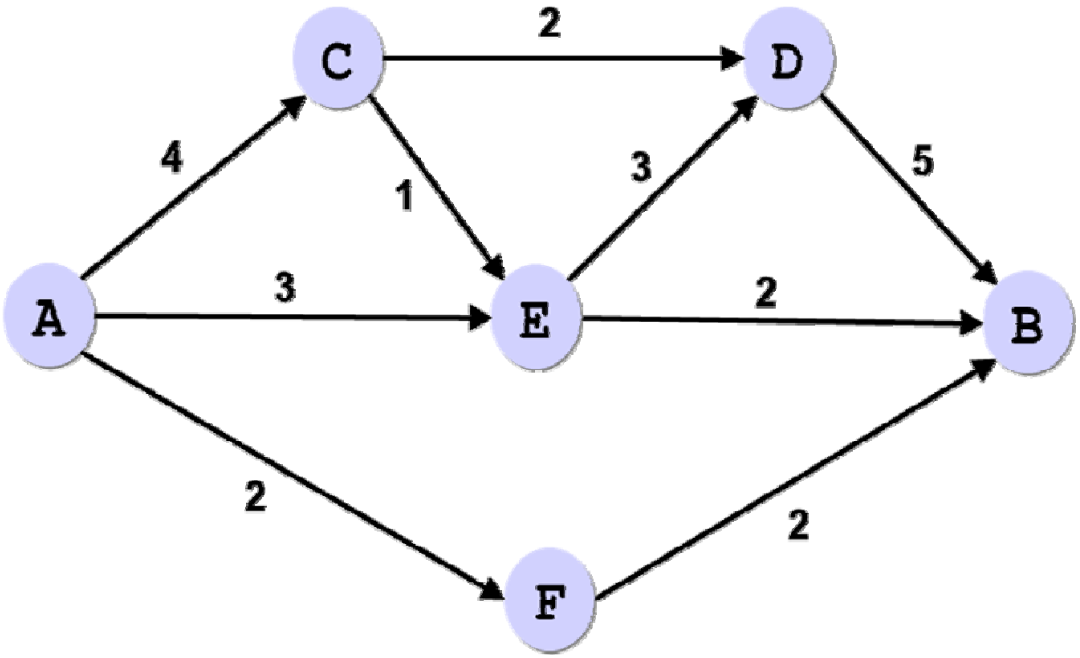
Unknown quantities are the optimum numbers of cars x_{ij} passing along each road per hour. **Here we have CONSTRAINTS c_{ij} .**

Conditions:

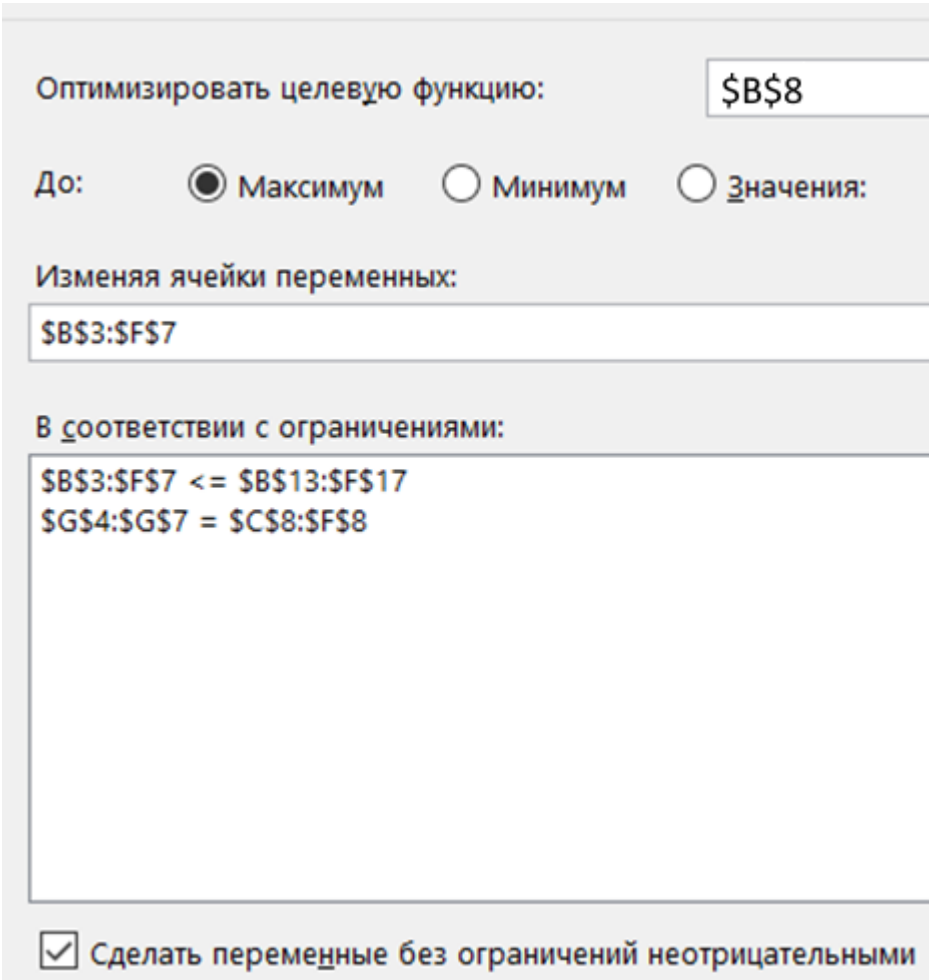
1) the number of cars that enter and depart from the city must be the same.

2) traffic limits: $0 \leq X_{ij} \leq c_{ij}$

3) target function: ΣX_{iB} → max



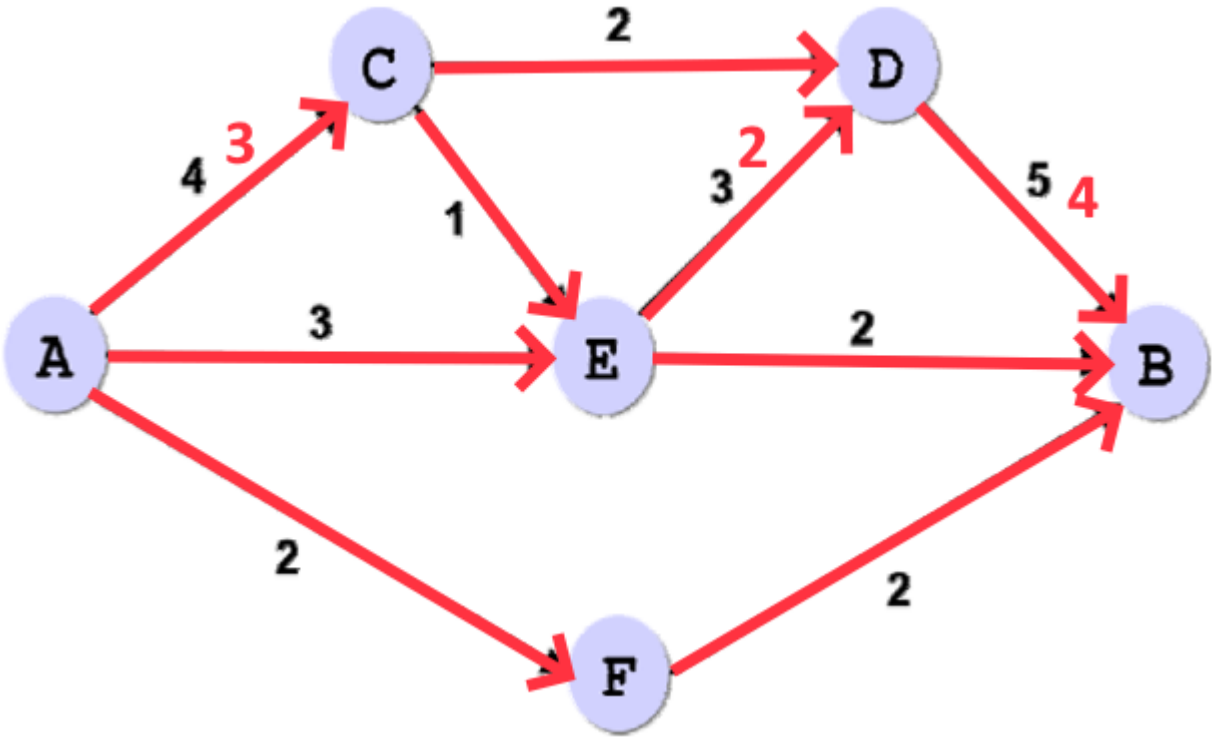
	A	B	C	D	E	F	G
1		unknown optimum traffic xij					
2		city B	city C	city D	city E	city F	Total from city
3	cityA	1	1	1	1	1	5
4	cityC	1	1	1	1	1	5
5	cityD	1	1	1	1	1	5
6	cityE	1	1	1	1	1	5
7	cityF	1	1	1	1	1	5
8	Total to city	5	5	5	5	5	
9							
10							
11		maximum traffic					
12		to B	to C	to D	to E	to F	
13	from A	0	4	0	3	2	
14	from C	0	0	2	1	0	
15	from D	5	0	0	0	0	
16	from E	2	0	3	0	0	
17	from F	2	0	0	0	0	
18							



In the option “Solve” we set the conditions 1) – 3):

Solution:

	A	B	C	D	E	F	G
1		unknown optimum traffic xij					
2		city B	city C	city D	city E	city F	Total from city
3	cityA	0	3	0	3	2	8
4	cityC	0	0	2	1	0	3
5	cityD	4	0	0	0	0	4
6	cityE	2	0	2	0	0	4
7	cityF	2	0	0	0	0	2
8	Total to city	8	3	4	4	2	
9							


A
C
D
E
B
F
4
3
2
1
3
2
3
2
5
4
2
2

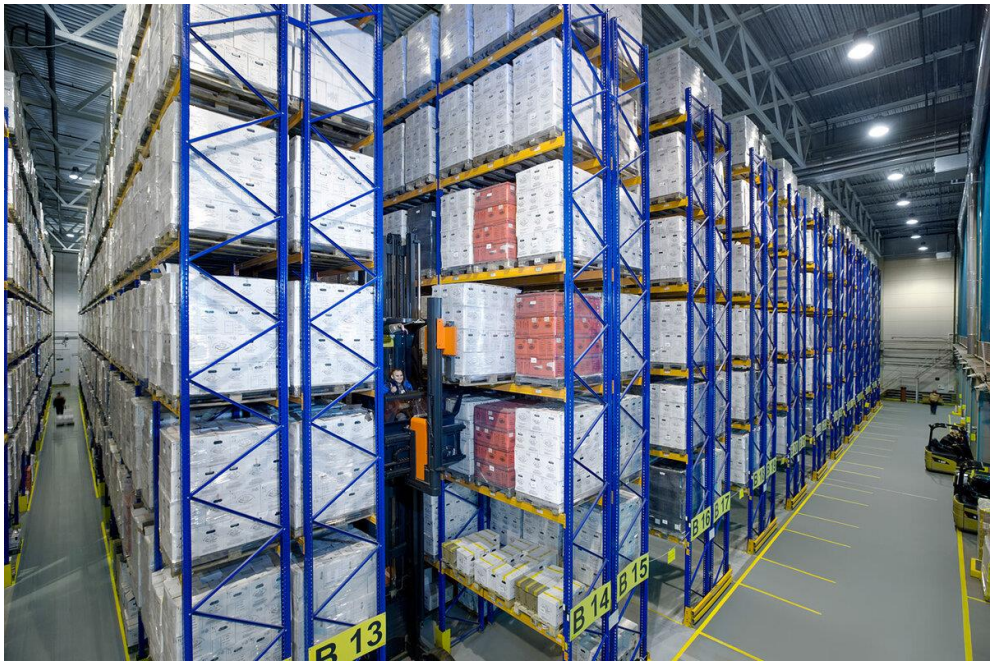
9-3. Minimization of the delivery cost in case of a multi-variable function under constraints.

Example.

Shops / Available goods (e.g., bottles of juice)	Shop 1 needs b_1=110	Shop 2 needs b_2=350	Shop 3 needs b_3=140
Warehouse 1: a_1= 180	c_{11}	c_{12}	c_{13}
Warehouse 2: a_2= 300	c_{21}	c_{22}	c_{23}
Warehouse 3: a_3= 120	c_{31}	c_{32}	c_{33}

c_{ij} – delivery cost for 1 piece of goods from Warehouse i to Shop j

Shop



Warehouse/Storage

Shops Available goods	Shop 1 needs b_1=110	Shop 2 needs b_2=350	Shop 3 needs b_3=140
Warehouse 1: a_1= 180	2 $	5 $	2 $
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: a_3= 120	3 $	6 $	8 $

c_{ij} – delivery cost for 1 piece of goods from Warehouse i to Shop j

x_{ij} – number of goods to be delivered from Warehouse i to Shop j - ?

Suppose that total need $\Sigma_i a_i = \Sigma_j b_j$ total storage

Full cost:

$$F = \sum_{i=1}^{m}\sum_{j=1}^{n} c_{ij}x_{ij} \to \min$$

Each shop's need must be satisfied

$$\begin{cases} \sum_{i=1}^{m} x_{ij} = b_j, \ j = \overline{1,n} \\ \sum_{j=1}^{n} x_{ij} = a_i, \ i = \overline{1,m} \\ x_{ij} \geq 0 \end{cases}$$

All available Warehouse's goods must be sent out

1st step of solving the problem:

Search of a first approximation to solution x_{ij} by

(*) Method of “North-West corner” , or

(*) Method of the least element.

2nd step: refinement of x_{ij} using a tool of Excel

(*) Method of “North-West corner”

	Shop 1 b_1=110	Shop 2 b_2=350	Shop 3 b_3=140
Warehouse 1: a1= 180	c_{11} x_{11}	c_{12} x_{12}	c_{13} x_{13}
Warehouse 2: a2= 300	c_{21} x_{21}	c_{22} x_{22}	c_{23} x_{23}
Warehouse 3: a3= 120	c_{31} x_{31}	c_{32} x_{32}	c_{33} x_{33}

	Shop 1 initial need b_1=110 current 0	Shop 2 b_2=350	Shop 3 b_3=140
Warehouse 1: initial a_1= 180 current 70	2 $ 110	5 $	2 $
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: a_3= 120	3 $	6 $	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350 current 280	Shop 3 b_3=140
Warehouse 1: initial a_1= 180 current 0	2 $ 110	5 $ 70	2 $
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: a_3= 120	3 $	6 $	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350 current 0	Shop 3 initial need b_3=140
Warehouse 1: initial a_1= 180 current 0	2 $ 110	5 $ 70	2 $
Warehouse 2: initial a_2= 300 current 20	7 $	7 $ 280	13 $
Warehouse 3: a_3= 120	3 $	6 $	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350 current 0	Shop 3 initial need b_3=140 current 120
Warehouse 1: initial a_1= 180 current 0	2 $ 110	5 $ 70	2 $
Warehouse 2: initial a_2= 300 current 0	7 $	7 $ 280	13 $ 20
Warehouse 3: a_3= 120	3 $	6 $	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350 current 0	Shop 3 initial need b_3=140 current 0
Warehouse 1: initial a_1= 180 current 0	2 $ 110	5 $ 70	2 $
Warehouse 2: initial a_2= 300 current 0	7 $	7 $ 280	13 $ 20
Warehouse 3: initial a_3= 120 current 0	3 $	6 $	8 $ 120

Total cost x_{ij} : F= 110×2+70×5+280×7+20×13+120×8 = 3750 $

(*) Method of the least element:

We choose step-by-step the shop with minimum c_{32} and maximum b_j

	Shop 1 initial need b_1=110	Shop 2 b_2=350	Shop 3 b_3=140
Warehouse 1: **a1= 180**	2 $	5 $	2 $
Warehouse 2: **a2= 300**	7 $	7 $	13 $
Warehouse 3: **a3= 120**	3 $	6 $	8 $

	Shop 1 initial need b_1=110	Shop 2 initial need b_2=350	Shop 3 initial need b_3=140 current 0
Warehouse 1: initial a_1= 180 current 40	2 $	5 $	2 $ 140
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: a_3= 120	3 $	6 $	8 $

	Shop 1 initial need b_1=110 current 70	Shop 2 initial need b_2=350	Shop 3 initial need b_3=140 current 0
Warehouse 1: initial a_1= 180 current 0	2 $ 40	5 $	2 $ 140
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: a_3= 120	3 $	6 $	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350	Shop 3 initial need b_3=140 current 0
Warehouse 1: initial a_1= 180 current 0	2 $ 40	5 $	2 $ 140
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: initial a_3= 120 current 50	3 $ 70	6 $	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350 current 300	Shop 3 initial need b_3=140 current 0
Warehouse 1: initial a_1= 180 current 0	2 $ 40	5 $	2 $ 140
Warehouse 2: a_2= 300	7 $	7 $	13 $
Warehouse 3: initial a_3= 120 current 0	3 $ 70	6 $ 50	8 $

	Shop 1 initial need b_1=110 current 0	Shop 2 initial need b_2=350 current 0	Shop 3 initial need b_3=140 current 0
Warehouse 1: initial a_1= 180 current 0	2 $ 40	5 $	2 $ 140
Warehouse 2: initial a_2= 300 current 0	7 $	7 $ 300	13 $
Warehouse 3: initial a_3= 120 current 0	3 $ 70	6 $ 50	8 $

Total cost X_{ij} : F= 40×2+70×3+300×7+50×6+140×2 = 2970 $

Solving the same problem with EXCEL

First, we insert given c_{ij} a_i b_j

	A	B	C	D	E
1		110	350	140	
2	180	2	5	2	
3	300	7	7	13	
4	120	3	6	8	
5					

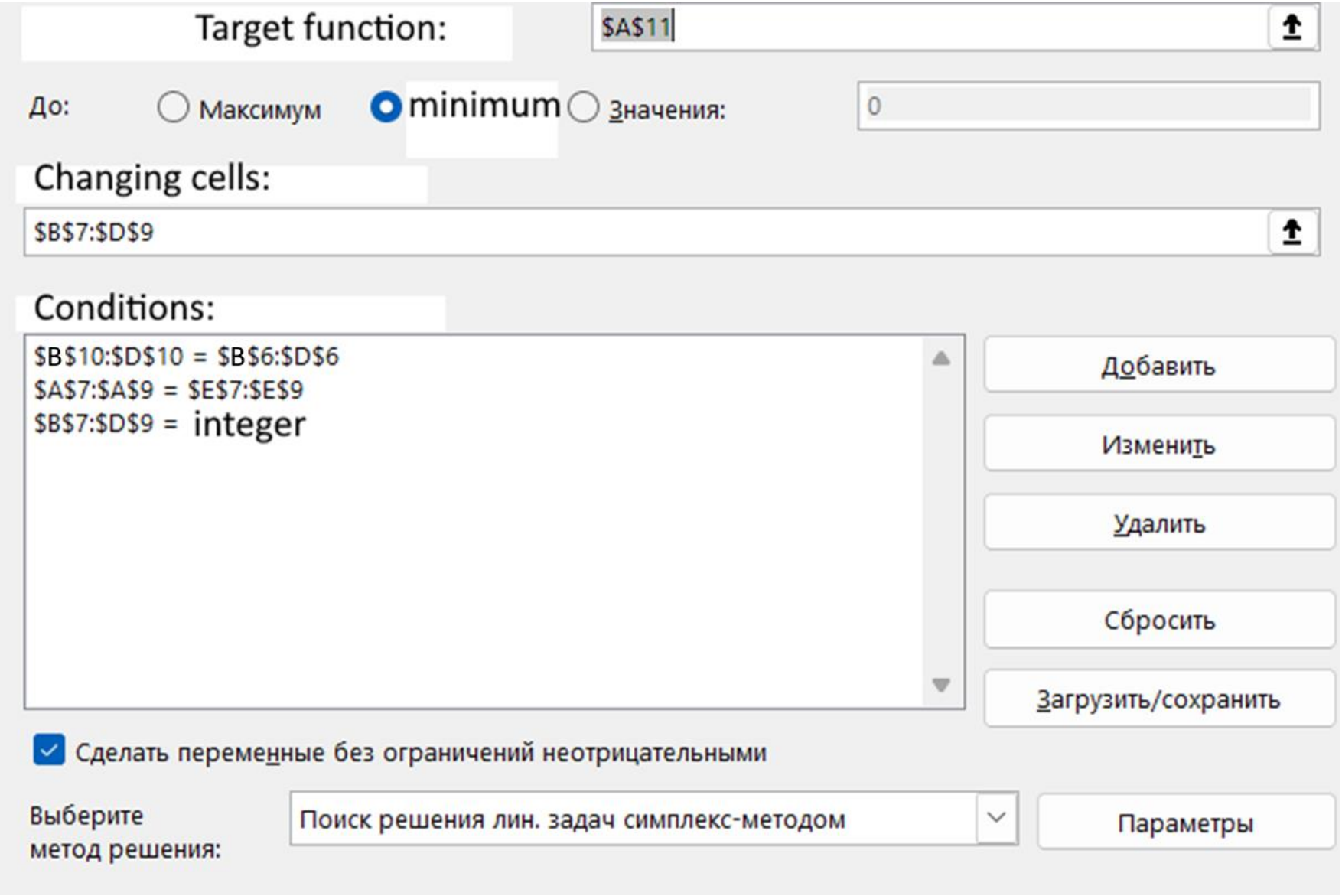
initial values for x_{ij}

6		110	350	140	
7	180	50	50	50	=sum
8	300	50	50	50	=sum
9	120	50	50	50	=sum
10		=sum	=sum	=sum	

11 =sumproducts(B2:D4;B7:D9)

?

Search of solution:



Target function: A11
Target: minimum
Changing B7 : D9

Conditions:

B10 : D10 = B6 : D6 need of each store
A7 : A9 = E7 : E9 amount at warehouse

B7 : D9 = integer
B7 : D9 >=0

Solution:

	110	350	140	
180	0	40	140	180
300	0	300	0	300
120	110	10	0	120
	110	350	140	
2970				

Выберите метод решения: Поиск решения лин. задач симплекс-методом Параметры

We supposed above that $\Sigma_i a_i = \Sigma_j b_j$ (*)

If not, then we can add extra an Shop or Warehouse to meet condition (*) :

Example: needs are less than reserves

Shops / Warehouses	Shop 1 b_1=110	Shop 2 b_2=350	Shop 3 b_3=40
Warehouse 1: a_1=180	2	5	2
Warehouse 2: a_2= 300	7	7	13
Warehouse 3: a_3= 120	3	6	8

Shops / Warehouses	Shop 1 b_1=110	Shop 2 b_2=350	Shop 3 b_3=40	Fictituous shop b_3=100
Warehouse 1: a_1=180	2	5	2	0
Warehouse 2: a_2= 300	7	7	13	0
Warehouse 3: a_3= 120	3	6	8	0

Contribution of the last column to total cost of delivery will be zero.

Therefore, the obtained solution will be optimized from the viewpoint of Shops 1 – 3. $X_{i,\ fictituous}$ **must remain at warehouses.**

Example: needs are larger than reserves

Shops / Warehouses	Shop 1 b_1=110	Shop 2 b_2=350	Shop 3 b_3=240
Warehouse 1: a_1=180	2	5	2
Warehouse 2: a_2= 300	7	7	13
Warehouse 3: a_3= 120	3	6	8

	Shop 1 b_1=110	Shop 2 b_2=350	Shop 3 b_3=240
Warehouse 1: a_1=180	2	5	2
Warehouse 2: a_2= 300	7	7	13
Warehouse 3: a_3= 120	3	6	8
Fictituous warehouse a_4= 100	0	0	0

Contribution of the last row/line to total cost of delivery will be zero.

Therefore, the obtained solution will be optimized from the viewpoint of Shops 1 – 3.

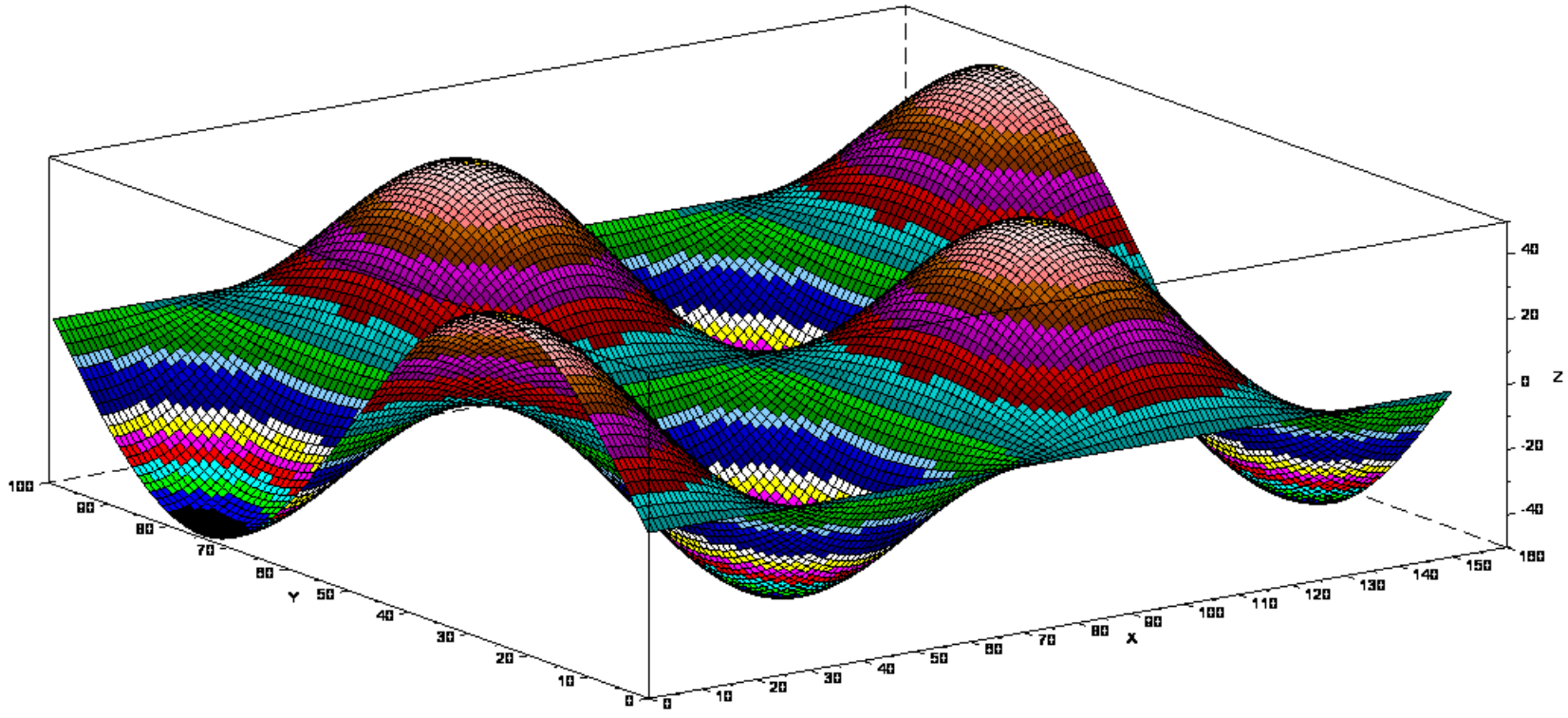
Last line will show deficit/undersupplies x_{4j}

9-4. Solving nonlinear optimization problems under constraints using penalty method

penalty = 罰款

In Section 9-3, we assumed that the cost of delivery is proportional to the number of goods delivered. Actually, this dependence is nonlinear.

In the case of nonlinear functions with constraints, the problem of finding a minimum becomes more difficult.



We can use gradient methods considered in Chapter 8

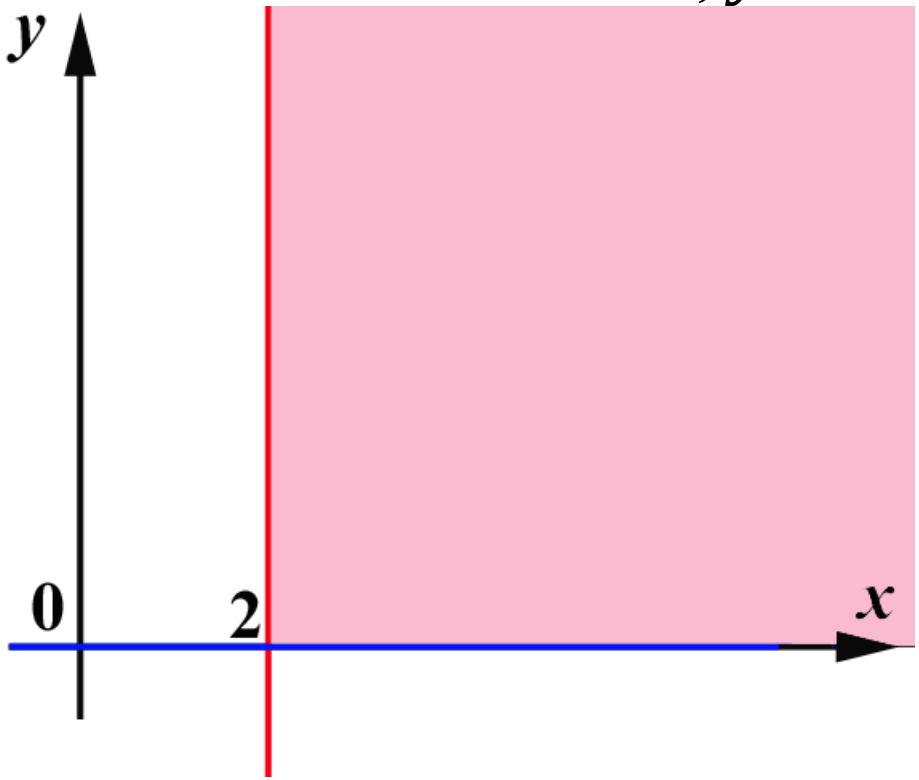
$$x^{(k+1)} = x^{(k)} - h \cdot \text{grad } F(x^{(k)})$$

but we must modify them to retain $x^{(k+1)}$ <u>inside</u> the given domain.

The idea of the method of fines is to introduce an extra function $P(x, y)$ that is nearly zero in the given domain, except for a vicinity of the boundary where it <u>increases abruptly</u>.

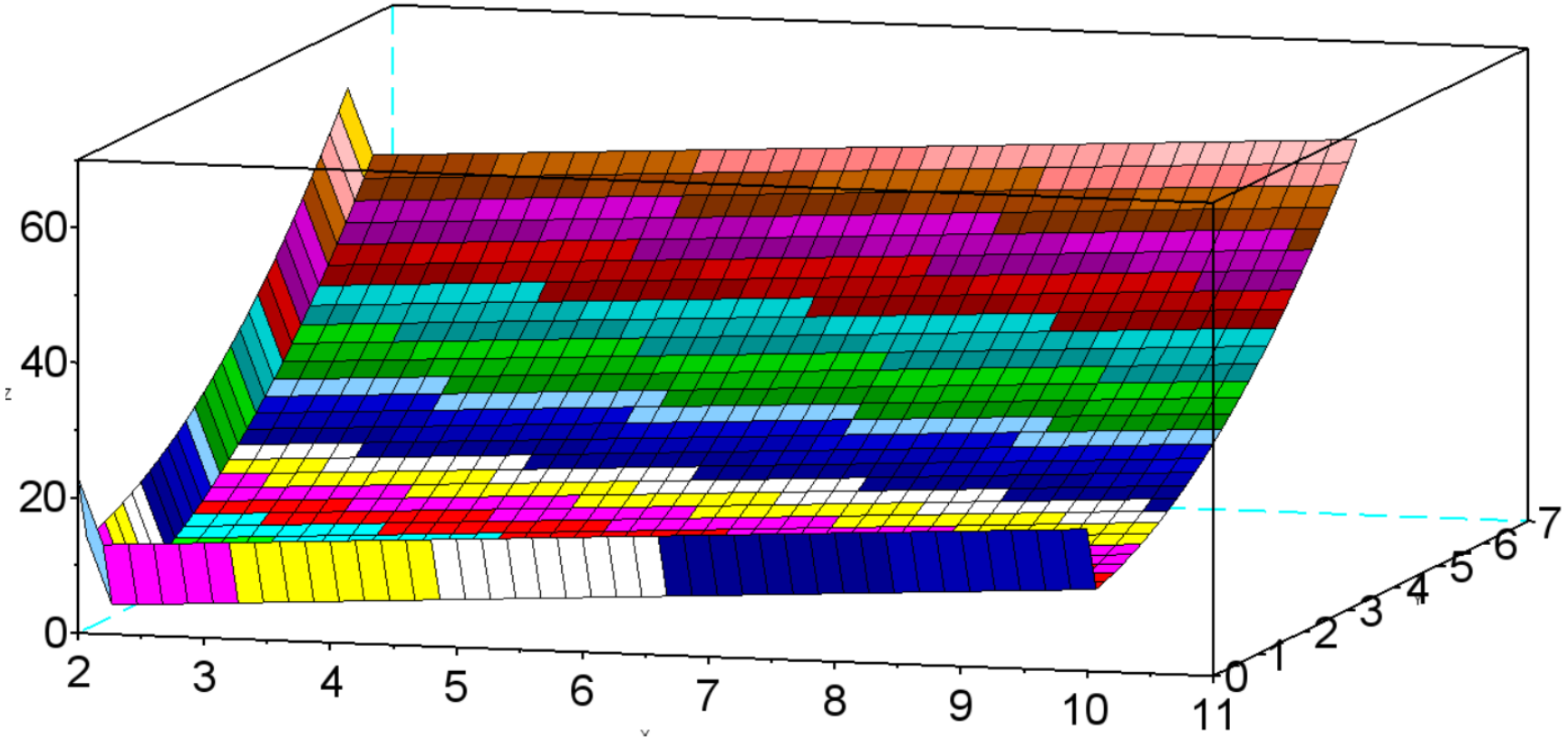
This prevents $x^{(k+1)}$ from crossing the boundary.

Example. Find a minimum of the function $f(x, y)= x+(y+1)^2$ under constraints $x \geq 2, y \geq 0$.



Let us choose a penalty function : $P(x, y)= 0.0001 \times [\ 1/(x-2) +1/y\ ]$ and add it to $f(x, y)$:

$$F(x, y)= x+(y+1)^2 + 0.0001 \times [\ 1/(x-2) +1/y\ ]$$


60
40
20
0
2
3
4
5
6
7
8
9
10
11
0
1
2
3
4
5
6
7

```
clear
for i=1:41
for j=1:31
x(i,j)=0.2*(i-1)+2+0.00001 ;
y(i,j)=0.2*(j-1)+0.00001 ;
end
end
F=x+(y+1).^2 + 0.0001*(1./(x-2) +1./y) ;
//surf(x,y,F)
// Using Gradient descent:
h=0.006
xx=5 ;
yy=5 ;
for k=1:500
```

```
dFdx= 1 -0.0001/(xx-2)^2 ;
dFdy= 2*(yy+1) - 0.0001/yy^2 ;
xx=xx-h*dFdx
yy=yy-h*dFdy
disp(k,xx,yy)
plot(xx,yy,'o')
end
```

General formulation:

If a minimum of a function $f(x_1, x_2, \ldots, x_n)$ **is to be found under** m **constraints**

$$g_i(x_1, x_2, \ldots, x_n) \geq 0 , \quad i=1, 2, \ldots, m$$

then it makes sense to consider the function

$$F(x_1, x_2, \ldots, x_n) = f(x_1, x_2, \ldots, x_n) + P(x_1, x_2, \ldots, x_n)$$

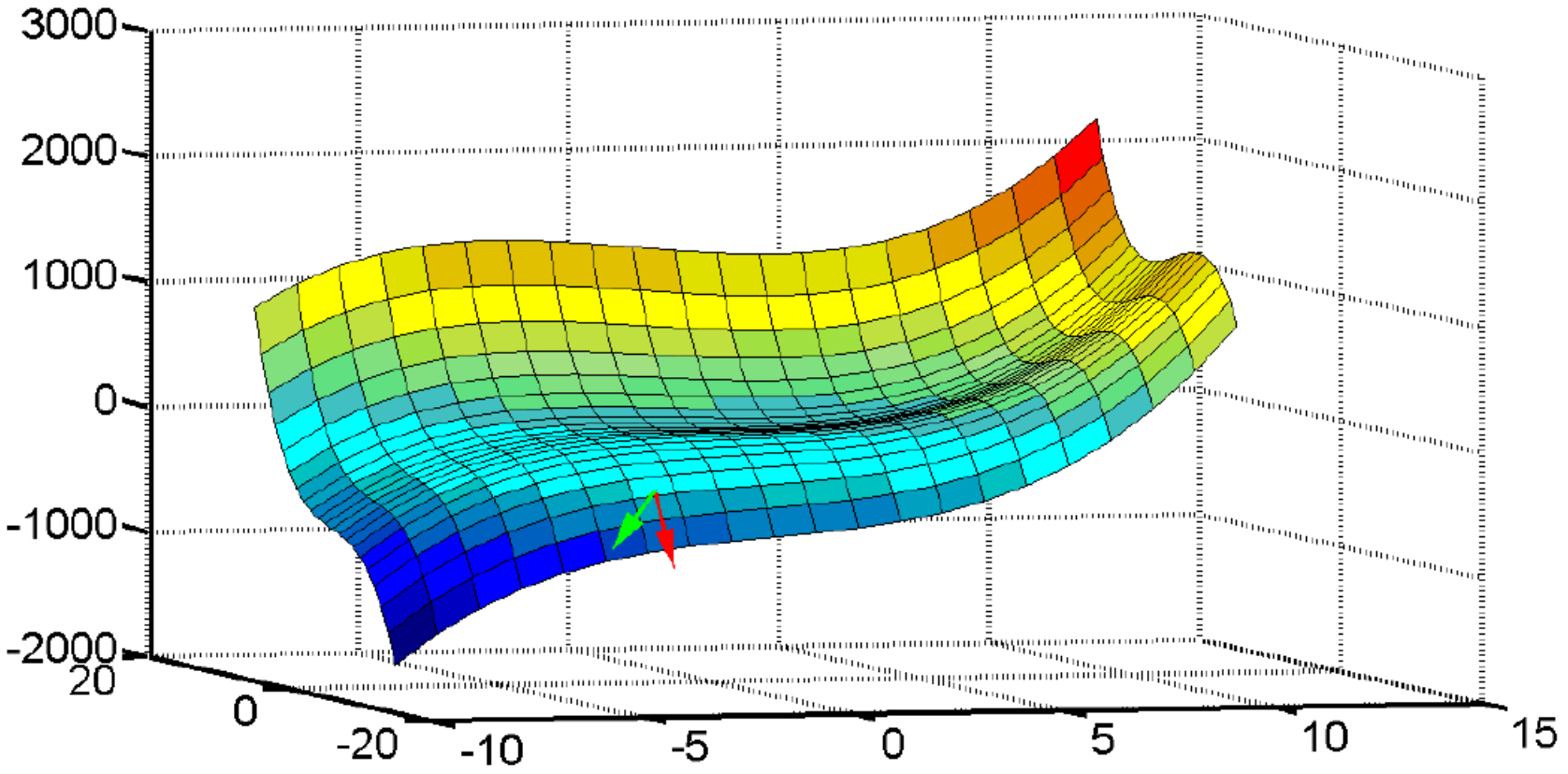
where penalty P **can be chosen as follows:**

$$P(x_1, x_2, \ldots, x_n) = r \sum_{i=1}^{m} [\, 1/g_i(x_1, x_2, \ldots, x_n) \,]$$

Method of admissible directions of gradient descent

Idea: a direction of next step of gradient descent

$x^{(k+1)} = x^{(k)} - h \cdot \mathrm{grad}\, F(x^{(k)})$ **is not allowed if the point** $x^{(k+1)}$ **gets outside of the domain specified by constraints.**



EXCEL

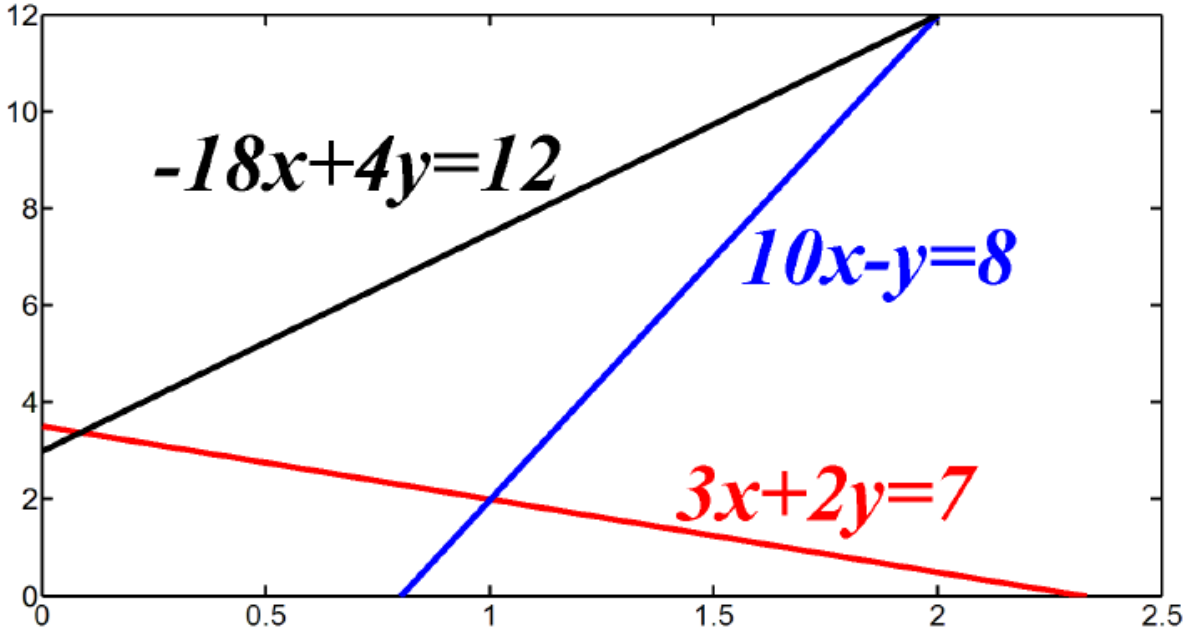
Solution of an optimization problem can be obtained with the extension "Solver".

Example. Find a minimum of function

$$f(x,y) = (x-3)^2 + (y-4)^2$$

under constraints

$$3x + 2y \geq 7$$
$$10x - y \leq 8$$
$$-18x + 4y \leq 12, \qquad x \geq 0, \qquad y \geq 0$$


-18x+4y=12
10x-y=8
3x+2y=7
12
10
8
6
4
2
0
0
0.5
1
1.5
2
2.5

	A	B	C
1	1	4	=(A1-3)^2+(B1-4)^2
2			
3			
4	=3*A1+2*B1		
5	=10*A1 - B1		
6	=-18*A1+ 4*B1		

Matlab

One of the available in Matlab commands for solving optimization problems under constraints is fmincon

Example. Find a maximum of the function

$f(x)=x_1 \cdot x_2 \cdot x_3$

under linear constraints

$0 \le x_1+2x_2+2x_3 \le 72$

Initial point:

x= [10; 10; 10]

First, create file myfun.m which determines the target function:

```
function f = myfun(x)
f = -x(1)*x(2)*x(3);
endfunction
```

Then we rewrite constraints in the form " ≤ "

$-x_1-2x_2-2x_3 \leq 0$

$x_1+2x_2+2x_3 \leq 72$

In the matrix form this can be written as Ax ≤ b

```
>> A = [-1 -2 -2; 1 2 2];
>> b = [0; 72];
```

```
>> x0 = [10; 10; 10];
>> [x,fval] = fmincon('myfun',x0,A,b)
```

Answer:

x = 24.0000 , 12.0000 , 12.0000

fval =-3.4560e+003

P.S. If constraints are nonlinear, then there is need to use more available options in fmincon

Chapter 10. Minimization of functionals

A variable I is called a functional , which depends on a function $y(x)$ given on a segment $a \le x \le b$, if each function $y(x)$ determines a value of I.

Examples:

$$I\,[\,y(x)] = \int_a^b y(x)dx$$

$$I\,[\,y(x)] = \int_a^b [\,x^2 + y^3(x)\,]\,dx$$

$$I\,[\,y(x)] = \int_a^b [\,x^2 + y^3(x) - y'(x)\,]\,dx$$

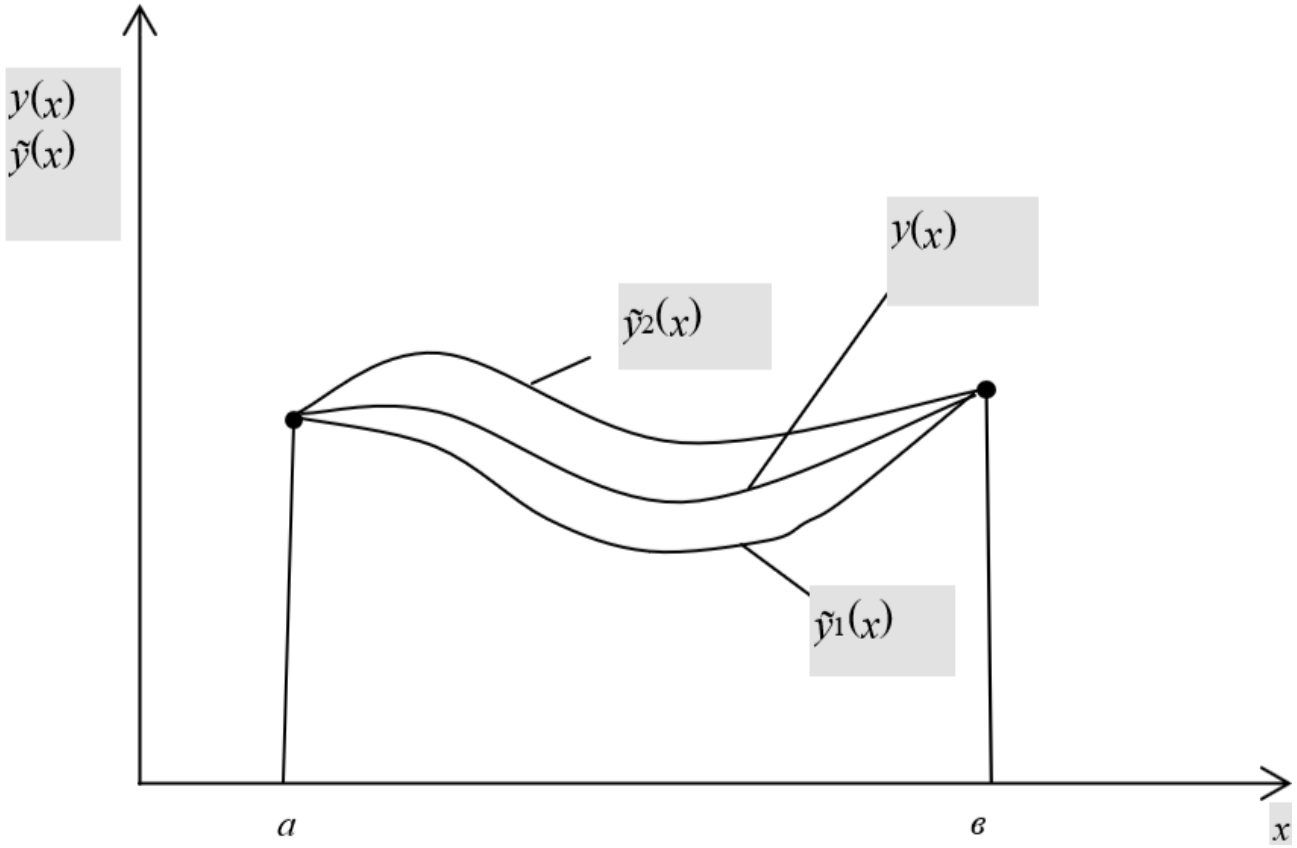
The problem of minimization for a functional

$$I[y(x)] = \int_a^b F(x, y(x), y'(x))\,dx$$

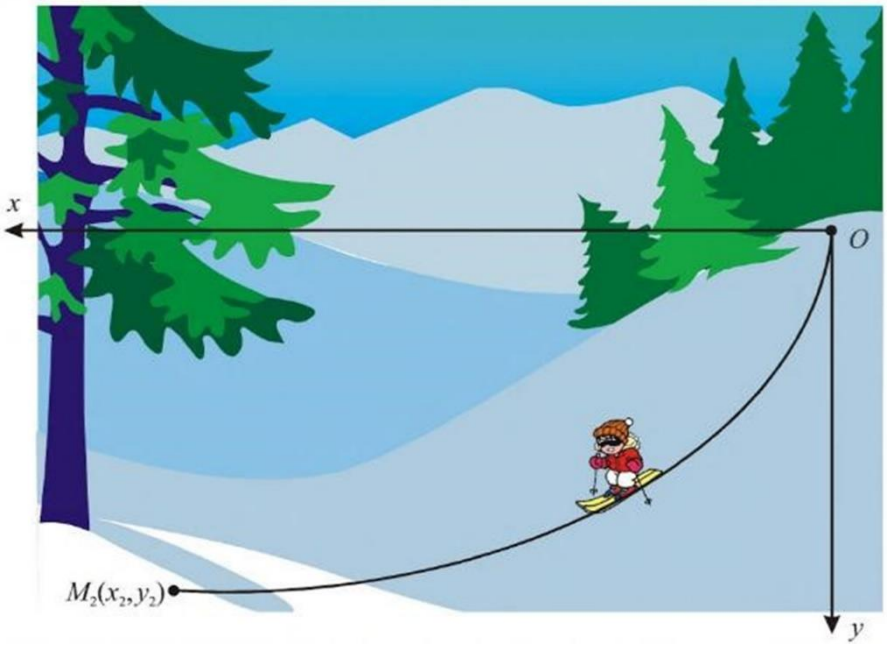
often involves conditions for $y(x)$ **at endpoints of the segment:**

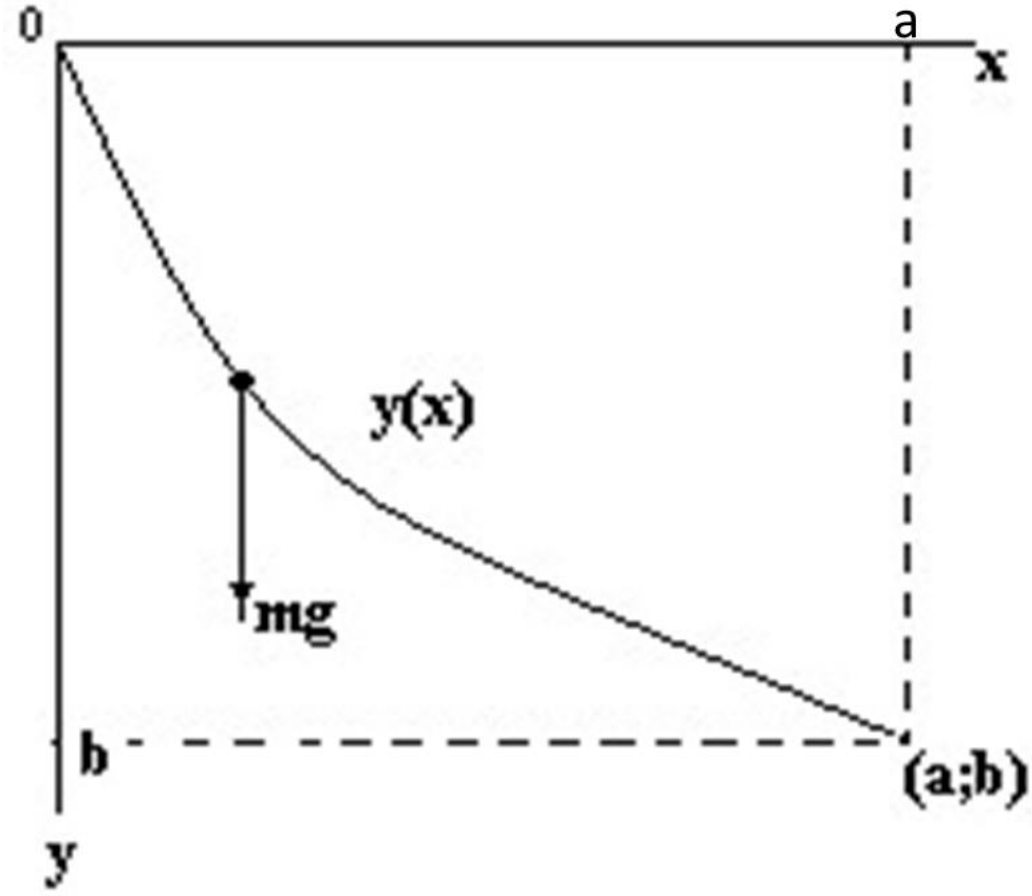
$y(a)=y_1, \quad y(b)=y_2$

(fixed endpoints).



Example 1. **Find the shape of a curve that begins at point x=0, y=0 and ends at x=a, y=b using the condition of minimum time a ball rolls down along the curve under the influence of gravity (in the absence of friction).**





$mV^2/2=mgy \quad \rightarrow \quad V^2/2=gy$

$$ds = V\,dt \qquad V=\sqrt{2gy} \qquad y(0)=0 \qquad y(a)=b$$

$$dt = \frac{ds}{V} = \frac{\sqrt{1+y'^2}\,dx}{\sqrt{2gy}}$$

$$t = \int_0^a \frac{\sqrt{1+y'^2}}{\sqrt{2gy}}\,dx$$

Johann Bernoulli, 1667 - 1748

The problem formulated in Example 1 was set by Bernoully in 1696

Example 2. Let us solve numerically a simpler problem

$$I = \int_0^1 [\, 12xy + (y')^2]\,dx$$

$y(0)=0 \qquad y(1)=1$

For an approximate solution, we will test 3 different polynomials.

1) **If we choose $y=x$, then easy calculations show $I_1 = 4+1=5$**

2) **Let us consider $y=ax^2+bx$ and select a, b so as to minimize the integral I:**

$$I = \int_0^1 [\, 12\,(ax^3+bx^2\,) + (2ax+b\,)^2\,]\,dx =$$

$$= 3a + 4b + \int (4a^2x^2+4abx+b^2\,)\,dx$$

$$= 3a + 4b + 4a^2/3+2ab+b^2$$

where $a+b=1$, $b=1-a$. $$= 3a + 4(1-a) + 4a^2/3+2a(1-a)+(1-a)^2$$

$$= 3a + 4(1-a) + 4a^2/3 + 2a(1-a) + (1-a)^2$$

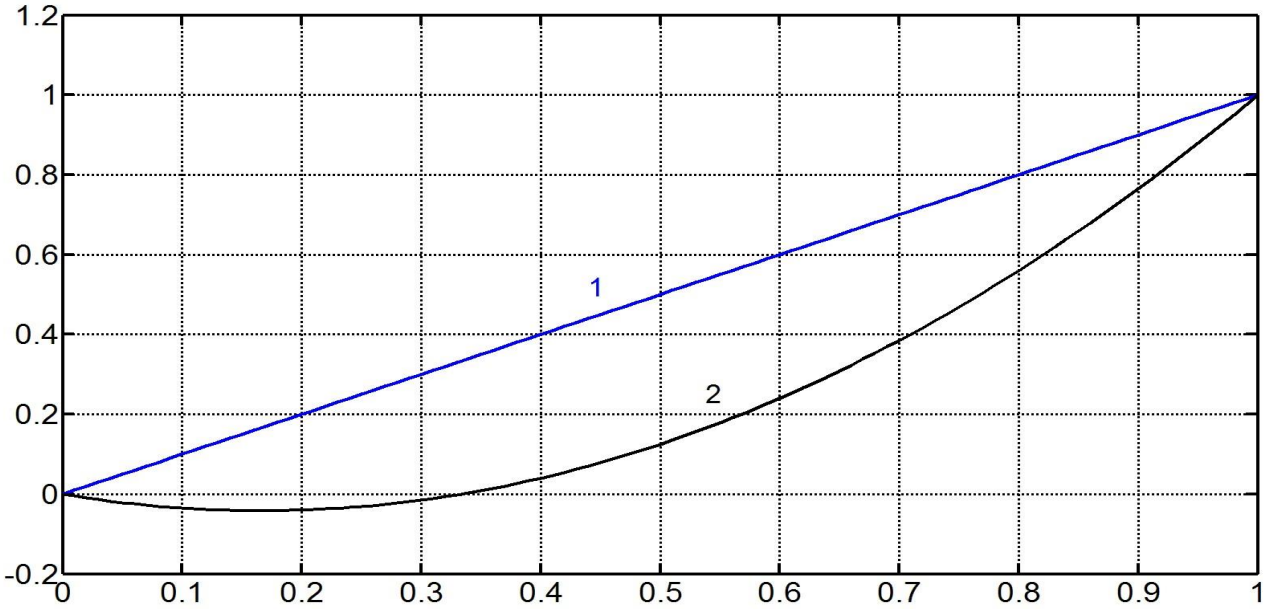
The equation $dI/da = 0$ **:**

$$3 - 4 + 8a/3 + 2 - 4a + 2(1-a)(-1) = 0\,, \quad -1 + 8a/3 - 2a = 0$$

and we find $a = 1.5$, $b = -0.5$.

Therefore, a minimum is realized at the function $y = 1.5x^2 - 0.5x$,

and this minimum is $I_2 = 3a + 4b + 4a^2/3 + 2ab + b^2 = 4.25$

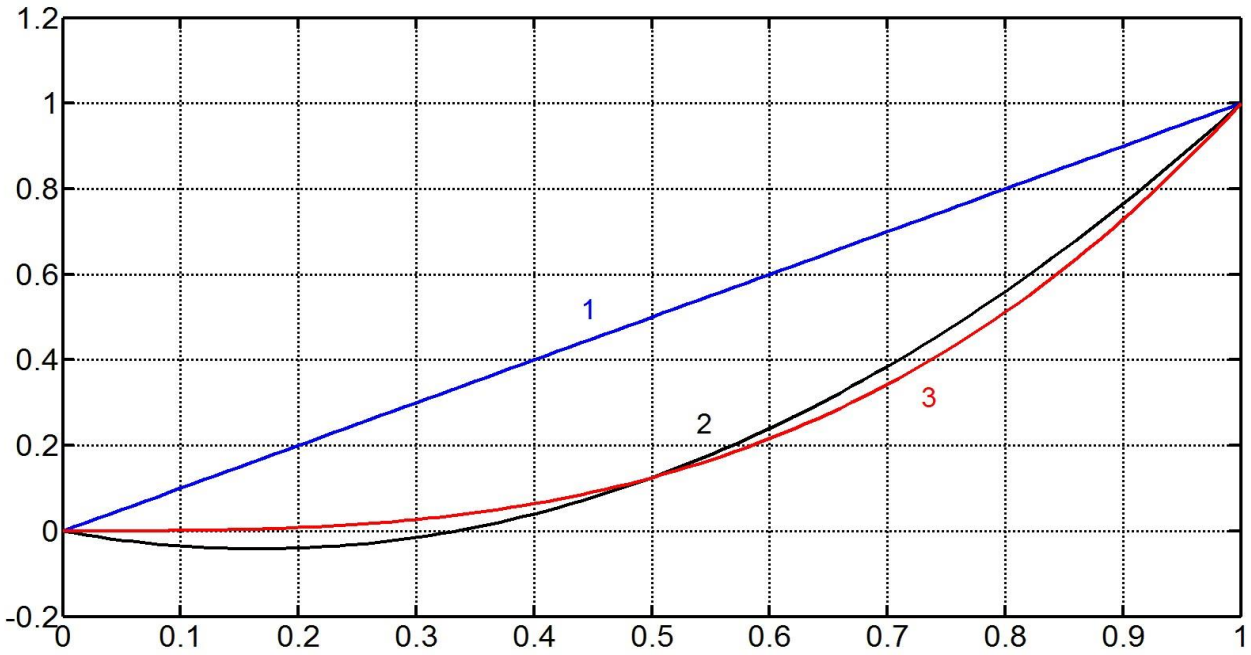


3) Now we consider $y=ax^3+bx^2+cx$ and select a, b, c so as to minimize the integral I under condition $a + b + c = 1$

$$I = \int_0^1 [\, 12\,(ax^4+bx^3+cx^2) + (3ax^2+2b\,x+c)^2\,]\,dx = \ldots\ldots\ldots\ldots\ldots$$

Using Excel or condition $\partial I/\partial a=\partial I/\partial b=0$ for finding a minimum, we get

$a=1$, $b=c=0$, $y= x^3$, $I_3 = 0.25$



Trigonometric functions can also be used in searches of minimizing function.

Example 3 **Find a minimum of functional**

$$I = \int_0^1 [\, x^2+y^2 + (y')^2 \,]\, dx \qquad y(0)=1 \qquad y(1)=2$$

1) $y = x+1$

$$I_1 = \int_0^1 [\, x^2+y^2 + (y')^2 \,]\, dx = \int [\, x^2+(x+1)^2 + 1 \,]\, dx = \int (2x^2+2x+2 \,]\, dx =$$

$$= 2/3 +1 +2 = 3.66666666$$

2) Now choose $y= x+1+ a \sin(\pi x)$ and select a to minimize I:

$$I_2 = \int_0^1 [\, x^2+ (x+1+ a \sin(\pi x))^2 + (1+ a \pi \cos(\pi x))^2 \,]\, dx =$$

$$= \int [\, x^2+ (x+1)^2+2(x+1)\, a \sin(\pi x) + a^2 \sin^2(\pi x) +1+2a \pi \cos(\pi x) + a^2 \pi^2 \cos^2(\pi x)]\, dx$$

$$dI/da = \int [\, 2(x+1) \sin(\pi x) + 2a \sin^2(\pi x) + 2a \pi^2 \cos^2(\pi x)]\, dx = 0$$

$$\int_0^1 [\, 2(x+1)\sin(\pi x) + a\,(1-\cos(2\pi x)) + a\,\pi^2(1+\cos(2\pi x))\,]\,dx = 0$$

$$\int_0^1 [\, 2(x+1)\sin(\pi x) + a + a\,\pi^2\,]\,dx = 0$$

$$\int_0^1 [\, 2(x+1)\sin(\pi x)\,]\,dx + a\,(1+\pi^2) = 0$$

Scilab:

```
integrate('2*(x+1)*sin(%pi*x)','x',0,1)
```

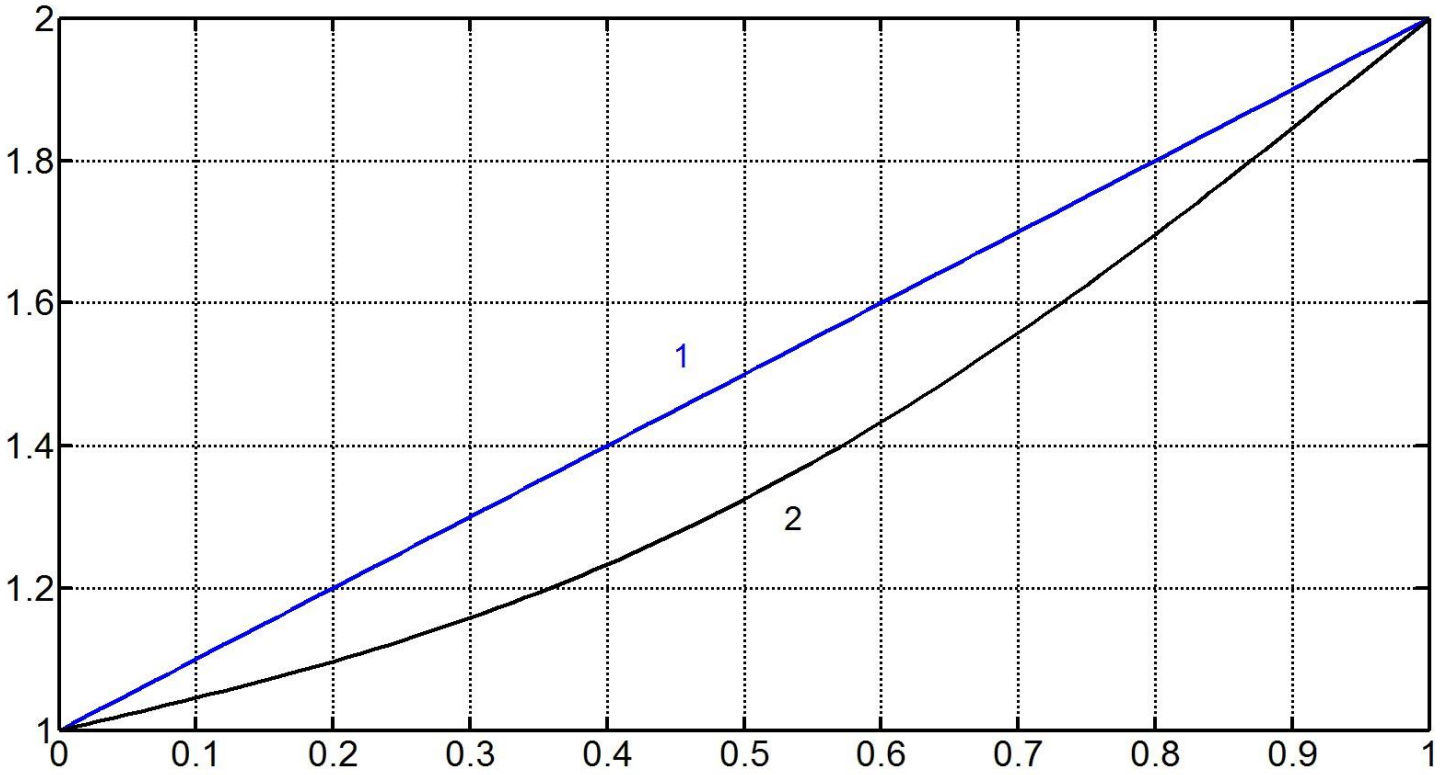
ans = 1.9098593

$$1.9098593 + a(1+\pi^2) = 0$$

$$a = -\,1.9098593/\,(1+\pi^2) = -0.17570642$$

Again Scilab: calculate the minimum

$$I_2 = \int_0^1 [\, x^2 + (x+1+ a \sin(\pi x))^2 + (1+ a\,\pi \cos(\pi x))^2 \,]\, dx = 3.4988794$$



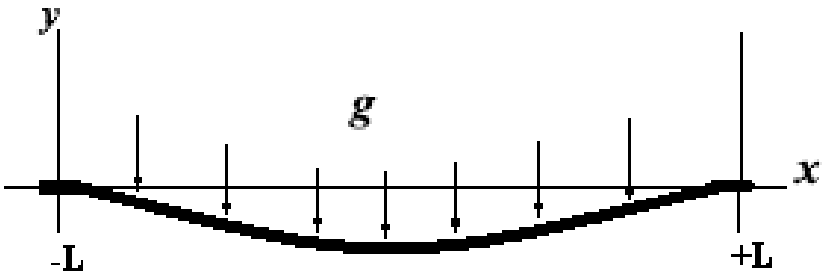
1: $y = x+1$ ➔ $I_1 = 3.6666666$ **2:** $y = x+1+a\sin(\pi x)$ ➔ $I_2 = 3.4988794$

3) Then we can try $y = x+1+a\sin(\pi x)+b\sin(2\pi x)$ **and select** a, b **so as to minimize functional** I**. Probably, the obtained value** I_3 **will be smaller than** I_2**.**

The more terms in the sum, the better.

A functional may depend on the second-order derivative:

Example. Equilibrium of a rigid beam



Beam is fixed at x=-L, x=L.

The potential energy depends on its shape

$$I[y(x)] = \int_{-L}^{L} (1/2\, EJy''^2 + \rho g y)dx, \quad y(L) = y(-L) = 0,\ y'(-L) = y'(L) = 0$$

First term is density of elastic energy of beam deformation (bending), second term is density of potential energy due to gravity.

Equilibrium corresponds to a minimum of potential energy (see a course of Physics). Using a polynomial of 4th degree for minimization of the functional, we can obtain the solution

$$y(x) = -\frac{\rho g}{24EJ}x^4 + c_1x^3 + c_2x^2 + c_3x + c_4$$

Constants c_1, c_2, c_3, c_4 **are determined by four boundary conditions at** $x = \pm L$ **(details are omitted).**

Finally:

$$y(x) = -\rho g/(24EJ)(x+L)^2(x-L)^2$$

Other conditions at endpoints:

The examples considered above demonstrated a Direct numerical method of functional minimization.

An idea of the direct methods is to create a sequence of approximate solutions (functions) which will converge to the exact solution.

The idea was suggested by Walter Ritz in 1908.

Walter Ritz 1878 — 1909 (Swiss mathematician and physicist)

General approach to minimization of functionals:

$$I[y(x)] = \int_a^b F(x, y(x), y'(x))\,dx$$

Ritz proposed to seek ***n***-the approximation to the minimizing function in the form

$$y_n(x) = \sum_{i=1}^{n} \alpha_i \varphi_i(x) \qquad (*)$$

with constant coefficients α_i and known functions $\varphi_i(x)$.
Then functional **becomes a function of *n* variables $\alpha_1, \alpha_2, \ldots, \alpha_n$**

Now in order to find a minimum, one needs either to find first-order derivatives and set them to zero, or use numerical methods.
Here the problem is unconstrained.

Functions $\varphi_i(x)$ belong to a set of functions which must be complete. This means that any continuous function $f(x)$ can be approximated by expression (*) accurately enough at sufficiently large n .

As n increases, the sequence (*) converges to a function that minimizes functional (proof is omitted).

In practice, engineers and researchers normally use polynomials

$$1,\ x,\ x^2, \ldots,\ x^n, \ldots$$

or systems of trigonometric functions.

For estimation of the error, in practice, after calculation of two approximate solutions $y_n(x)$ and $y_{n+1}(x)$, it makes sense to compare them at a few points of segment [a,b]. If the difference does not exceed the admitted tolerance, then $y_n(x)$ can be accepted as an approximate solution of the functional minimization problem.

Chapter 11. Numerical Differentiation.

Let a function be given in table

$$y_0 , y_1 , y_2 , \ldots, y_n$$

$$x_0 , x_1 , x_2 , \ldots, x_n$$

Assume $h = x_{i+1} - x_i = \text{const}$.

$$y'_i = \ ?$$ approximately

It is well known from Mathematical Analysis that

$$f(x_{i+1}) = f(x_i) + f'(x_i)\, h + f''(c)\, h^2/2 \,.$$

The same formula using notation y_i looks :

$$y_{i+1} = y_i + y'_i\, h + f''(c)\, h^2/2 \,,$$

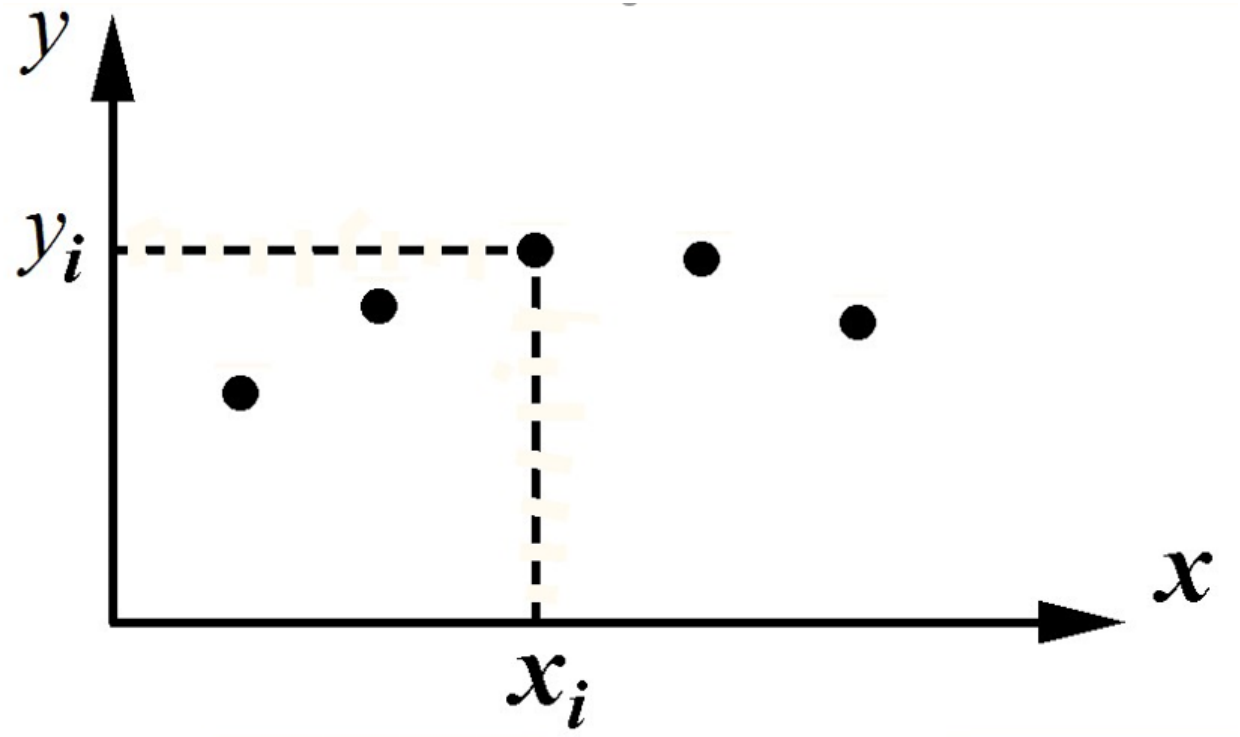
therefore,

$$y'_i = (y_{i+1} - y_i)/ h - f''(c)\cdot h/2$$

Similarly, for node i and previous node i-1 :

$$y_{i-1} = y_i - y'_i\, h + f''(c)\, h^2/2$$

$$y'_i = (y_i - y_{i-1})/ h + f''(c)\cdot h/2$$



$$y'_i \approx (y_{i+1} - y_i)/h$$

forward difference

$$y'_i \approx (y_i - y_{i-1})/h$$

backward difference

These expressions are of first-order accuracy, as the error involves the first degree of h (see previous page).

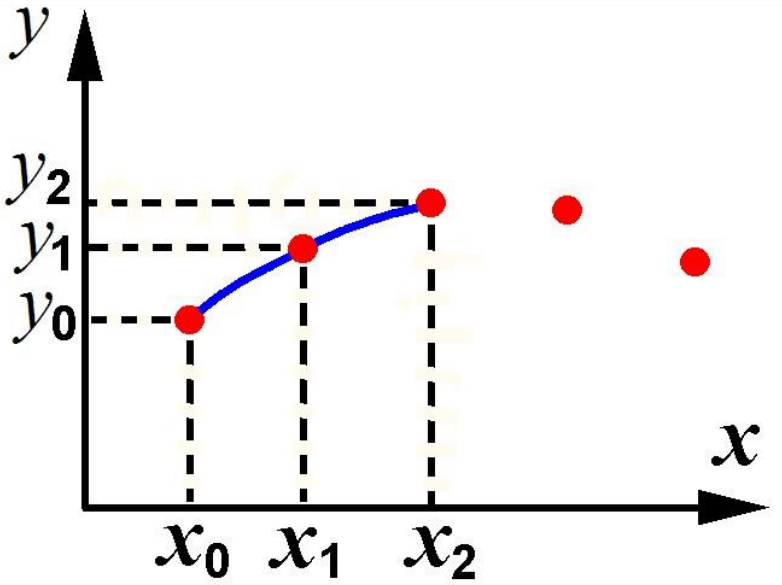
There exist formulae of higher accuracy for y'_i

Let us derive a formula of second-order accuracy for y'_0

at the left end x_0 of interval $[x_0, x_n]$

To do this, we write Lagrangian's polynomial, that passes through 3 points

(x_0, y_0) (x_1, y_1) (x_2, y_2)



$$P(x) = y_0 (x - x_1)(x - x_2)/(2h^2) -$$

$$- y_1 (x - x_0)(x - x_2)/h^2 +$$

$$+ y_2 (x - x_0)(x - x_1)/(2h^2)$$

$x_{i+1} - x_i = h$

The derivative of $P(x)$:

$$P'(x) = y_0 (x - x_1 + x - x_2)/(2h^2) -$$

$$- y_1 (x - x_0 + x - x_2)/h^2 +$$

$$+ y_2 (x - x_0 + x - x_1)/(2h^2)$$

Now we set $x = x_0$:

$$P'(x_0) = y_0 (x_0 - x_1 + x_0 - x_2)/ (2h^2) -$$
$$- y_1 (x_0 - x_2)/ h^2 +$$
$$+ y_2 (x_0 - x_1)/ (2h^2)$$

$$= y_0 (-h \ -2h)/ (2h^2) -$$
$$- y_1 (-2h)/ h^2 +$$
$$+ y_2 (-h)/ (2h^2)$$

$$= y_0 (-3) /(2h) - y_1 (-2)/ h + y_2 (-1)/ (2h)$$

$$= (-3y_0 +4y_1 -y_2) / (2h)$$

Thus,

$$y'_0 = f'(x_0) \approx P'(x_0) = (-3y_0 +4y_1 -y_2) / (2h)$$

This is formula of the second-order accuracy, as error involves the second degree of h:

$$y'_0 = (-3y_0 + 4y_1 - y_2) / (2h) + O(h^2)$$

Proof. **Theorem of Chapter 5, page 6, says:**

$$f(x) - P(x) = f^{(n+1)}(c) \cdot (x-x_0)(x-x_1) \ldots (x-x_n)/(n+1)!$$

For n=2:

$$f(x) - P(x) = f'''(c) \cdot (x-x_0)(x-x_1)(x-x_2)/3!$$

$$[f(x)-P(x)]' \Big|_{x=x0} = f'''(c) \cdot [(x-x_0)(x-x_1)(x-x_2)]'/6 \Big|_{x=x0}$$

$$= f'''(c) \cdot (x_0-x_1)(x_0-x_2)/6$$

$$= f'''(c) \cdot (-h)(-2h)/6 = f'''(c) \cdot h^2/3$$

For the last node x_n of given segment, in a similar way we obtain:

$$y'_n = (3y_n - 4y_{n-1} + y_{n-2}) / (2h) + O(h^2)$$

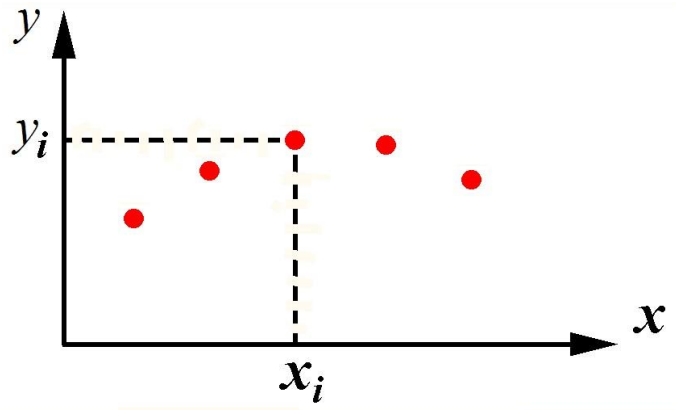
Second-order accuracy is much better than first-order,

as the error $O(h^2) \to 0$ much faster than $h \to 0$

Now let us derive the formula of second-order accuracy for y'_i at inner nodes i=1, 2, . . . , n-1.

Write Lagrange's polynomial that passes 3 points

(x_{i-1}, y_{i-1}) (x_i, y_i) (x_{i+1}, y_{i+1})



$$P(x) = y_{i-1}\,(x - x_i)(x - x_{i+1})/(2h^2) + y_i\,(x - x_{i-1})(x - x_{i+1})/(-h^2) + y_{i+1}(x - x_{i-1})(x - x_i)/(2h^2)$$

$$P'(x) = y_{i-1}\,(x - x_i + x - x_{i+1})/(2h^2) + y_i\,(x - x_{i-1} + x - x_{i+1})/(-h^2) + y_{i+1}(x - x_{i-1} + x - x_i)/(2h^2) \qquad (*)$$

Setting $x = x_i$, we obtain

$$P'(x_i) = y_{i-1}(-h)/(2h^2) + y_i(h-h)/(-h^2) + y_{i+1}(h)/(2h^2)$$

$$P'(x_i) = (y_{i+1} - y_{i-1})/(2h)$$

$$y'_i = f'(x_i) \approx P'(x_i) = (y_{i+1} - y_{i-1})/(2h)$$

Regarding the error:

$$y'_i = f'(x_i) = (y_{i+1} - y_{i-1})/(2h) - f'''(c)\, h^2/6$$

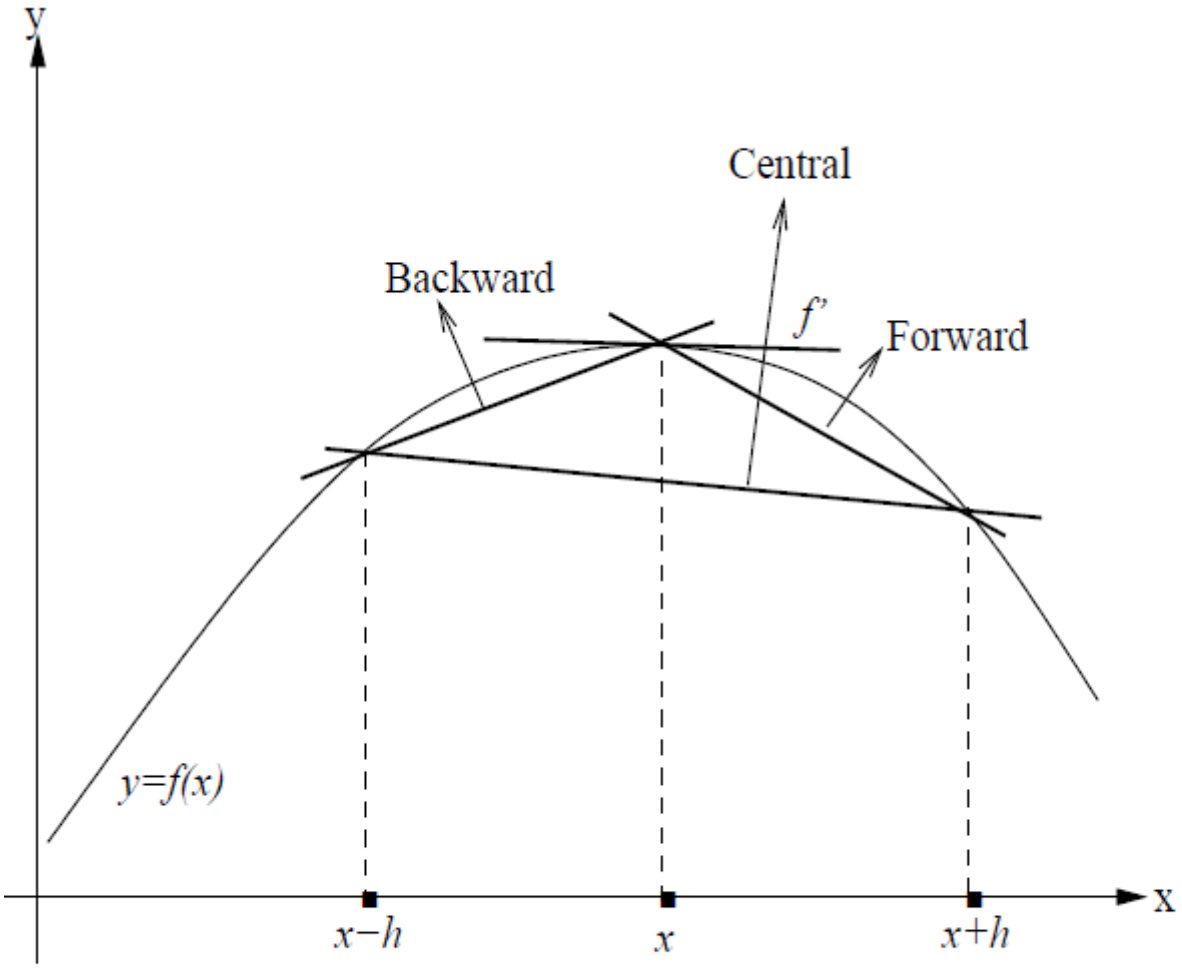
$$\boxed{y'_i = (y_{i+1} - y_{i-1})/(2h) + O(h^2)}$$

Proof:

$$[f(x)-P(x)]'\Big|_{x=x1} = f'''(c)\cdot[(x-x_0)(x-x_1)(x-x_2)]'/6\Big|_{x=x1}$$

$$= f'''(c)\cdot(x_1-x_0)(x_1-x_2)/6$$

$$= f'''(c)\cdot(h)(-h)]/6 = -f'''(c)\cdot h^2/6$$



Geometrical interpretation of difference formulae.

Illustration:

To find the value of derivative of $f(x)=\sin x$ at $x=1$

we use the three primitive difference formulas

$$f(x-h) = f(0.996094) = 0.839354, \qquad f(x) = f(1) = 0.841471,$$

$$f(x+h) = f(1.003906) = 0.843575.$$

$$h = 0.003906$$

I. Backward difference: $f'(x) = \frac{f(x)-f(x-h)}{h} = 0.541935.$

II. Central Difference: $f'(x) = \frac{f(x+h)-f(x-h)}{2h} = 0.540303.$

III. Forward Difference: $f'(x) = \frac{f(x+h)-f(x)}{h} = 0.538670.$

Note that the exact value is $f'(1) = \cos 1 = 0.540302.$

Formula of 4th-order accuracy for y'_i at inner nodes:

$$y'_i = (y_{i-2} - 8y_{i-1} + 8y_{i+1} - y_{i+2})/(12h) - f^{(5)}(c)h^4/30$$

It can be derived using polynomial of the degree n=4

For second-order derivative y''_i , differentiation of (*) gives at i=1, 2, . . . , n-1:

$$P''(x) = y_{i-1}(2)/(2h^2) + y_i(2)/(-h^2) + y_{i+1}(2)/(2h^2) = (y_{i-1} - 2y_i + y_{i+1})/h^2$$

$$y''_i = (y_{i-1} - 2y_i + y_{i+1})/h^2 + f^{(4)}(c)h^2/12$$

(formula of 2nd order accuracy). This will be of use for solving differential equations.

Formula of higher-order accuracy:

$$y''(x_i) = \frac{-y_{i+2} + 16y_{i+1} - 30y_i + 16y_{i-1} - y_{i-2}}{12h^2}$$

Calculation of derivatives with splines

Quadratic splines

$$S(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2$$

$$\text{at} \quad x_i \le x \le x_{i+1} \;, \qquad i = 0, 1, \ldots, n-1$$

coefficients a_i , b_i , c_i are calculated using 3 conditions:

1) The condition that $S(x)$ equals to given y_i at the left endpoint x_i :

$$S(x_i) = y_i \quad \Rightarrow \quad a_i = y_i \qquad i = 0, 1, \ldots, n-1$$

2)The condition that $S(x)$ equals to given value y_{i+1} at x_{i+1} :

$S(x_{i+1}) = y_{i+1}$ =>

$$\Rightarrow a_i + b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 = y_{i+1} \quad (1)$$

3) The condition that dS/dx is continuous at inner nodes x_{i+1} :

$$dS/dx = b_i + 2\,c_i \cdot (x - x_i) \qquad at \quad x_i \le x \le x_{i+1}$$

$$\downarrow$$

$$b_i + 2\,c_i \cdot (x_{i+1} - x_i) = b_{i+1} + 2\,c_{i+1} \cdot (x_{i+1} - x_{i+1})$$

$$\begin{cases} b_i + 2\,c_i \cdot (x_{i+1} - x_i) = b_{i+1} & (2) \quad i=0, 1, \ldots, n-2 \\ b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 = y_{i+1} - y_i & (1) \quad i=0, 1, \ldots, n-1 \end{cases}$$

In fact, coefficients b_i, c_i can be calculated <u>sequentially</u> for i=1,2, . . . , n :

b_1 , c_1 ,

then b_2 *from (1),*

then c_2 *from (2),*

then b_3 *from (1), . . .*

Therefore $y'_i \approx dS/dx\big|_{xi} = b_i$

Cubic spline

$$S_{cub}(x) = a_i + b_i \cdot (x - x_i) + c_i \cdot (x - x_i)^2 + d_i \cdot (x - x_i)^3$$

$$x_i \le x \le x_{i+1}\ , \qquad i = 0, 1, \ldots, n-1,$$

provides the continuity of both dS_{cub}/dx *and* d^2S_{cub}/dx^2

Coefficients of the spline are calculated using 4 conditions:

1) $S_{cub}(x)$ **is equal to** y_i **at left end of segment**

$$x_i \le x \le x_{i+1} : \qquad y_i = a_i$$

2) $S_{cub}(x)$ **is equal to** y_{i+1} **at right end of the segment**

$$y_{i+1} = a_i + b_i \cdot (x_{i+1} - x_i) + c_i \cdot (x_{i+1} - x_i)^2 + d_i \cdot (x_{i+1} - x_i)^3$$

3) First-order derivative:

$$dS_{cub}/dx = b_i + 2c_i \cdot (x-x_i) + 3d_i \cdot (x-x_i)^2$$

the condition of equal derivatives at the right endpoint:

$$b_i + 2c_i \cdot (x_{i+1}-x_i) + 3d_i \cdot (x_{i+1}-x_i)^2 = b_{i+1}$$

$$i=1,2,\ldots,n-1$$

4) Second-order derivative :

$$d^2S_{cub}/dx^2 = 2c_i + 6d_i \cdot (x-x_i)$$

the condition of equal second derivatives at the right endpoint:

$$2c_i + 6d_i \cdot (x_{i+1}-x_i) = 2c_{i+1} \qquad i=1,2,\ldots,n-1$$

We have got 2n+2(n-1) equations with respect to 4n coefficients of spline S_{cub} .

We add two conditions $d^2S_{cub}/dx^2 = 0$ at the first and last segments :

$$c_0 = 0, \qquad 2c_{n-1} + 6d_{n-1} \cdot (x_n - x_{n-1}) = 0$$

Therefore, we finally have system of 4n equations for finding 4n coefficients.

After calculation of the coefficients, we can find

$$dS_{cub}/dx = b_i + 2c_i \cdot (x - x_i) + 3d_i \cdot (x - x_i)^2$$

$$y'_i \approx dS_{cub}/dx \Big|_{xi} = b_i$$

$$d^2S_{cub}/dx^2 = 2\,c_i + 6d_i \cdot (x - x_i)$$

$$y''_i \approx d^2S_{cub}/dx^2 \Big|_{xi} = 2\,c_i$$